

XCPC Template Library

Morning Flower
and Evening Oath

Tongji University

双码风算法模板与数学速查

建设稿 · 2026-09-11

状态说明

本稿包含首批已进行本地验证的模块与完整来源覆盖表的入口。尚未完成 kuangbin、WIDA 全部条目的双码风迁移，尚未完成近三年中国赛站逐题覆盖审计，两套最大流已通过洛谷 P3376；其余模块尚无本项目 OJ AC 证据。本地对拍通过不代表对所有输入的绝对正确性证明；适用范围、数值上限与退化约定属于接口的一部分。

目录

使用与验证	v
第一部分 动态容器风格	1
第一章 数据结构	2
1.1 并查集	2
1.2 可撤销并查集	3
1.3 树状数组与第 k 小	4
1.4 区间加与区间和	4
1.5 64 位异或线性基	6
第二章 网络流	7
2.1 最大流与方案	7
2.2 势能最小费用最大流	9
2.3 上下界可行循环流	11
第三章 图论	13
3.1 非负权最短路	13
3.2 强连通分量	14
3.3 2-SAT 与任意可行方案	16
3.4 二分图最大匹配	16
3.5 割点与桥	19
第四章 树上算法	21
4.1 重链剖分与 LCA	21
第五章 字符串	24
5.1 前缀函数	24
5.2 KMP 匹配	24
5.3 Z 函数	25
5.4 Manacher 回文半径	25
5.5 最小循环表示	26
5.6 AC 自动机出现次数	26
5.7 倍增后缀数组与 LCP	29
5.8 后缀自动机	30
第六章 数论	32
6.1 模运算、素性、CRT 与类欧几里德	32
6.2 Pollard-Rho 因数分解	35
6.3 线性筛、欧拉函数与 Möbius 函数	36
6.4 静态模整数	37
6.5 阶乘组合数	38

第七章 多项式	39
7.1 NTT、形式幂级数、FWT 与递推	39
第八章 代数与数论进阶	44
8.1 模高斯消元、行列式、矩阵树定理	44
8.2 杜教筛	47
8.3 扩展 BSGS	48
第九章 计算几何	49
9.1 整数精确二维几何	49
9.2 浮点点线圆构造	52
第十章 优化与可持久化	56
10.1 Max-plus 矩阵幂	56
10.2 离散李超树与最优直线编号	56
10.3 主席树：静态区间第 k 小	58
第十一章 图论进阶	61
11.1 矩形二分图最小权匹配	61
11.2 有向最小树形图	62
11.3 无向全局最小割	64
第十二章 精确几何进阶	66
12.1 整数最近点对与 Minkowski 和	66
12.2 整数三维共线、共面与方向	69
第十三章 动态树	70
13.1 Link-cut tree：动态森林与路径异或	70
第十四章 随机平衡树	74
14.1 FHQ Treap：可重集合与顺序统计	74
14.2 隐式 Treap：序列翻转、加值与区间和	76
第十五章 伸展树	81
15.1 Splay：可重集合与顺序统计	81
第十六章 状态序列维护	86
16.1 状态序列 Treap：分组区间 GCD	86
第二部分 传统接口风格	90
第十七章 数据结构	91
17.1 并查集	91
17.2 可撤销并查集	92
17.3 树状数组与第 k 小	93
17.4 区间加与区间和	94
17.5 64 位异或线性基	95

第十八章 网络流	96
18.1 最大流与方案	96
18.2 势能最小费用最大流	98
18.3 上下界可行循环流	101
第十九章 图论	102
19.1 非负权最短路	102
19.2 强连通分量	103
19.3 2-SAT 与任意可行方案	105
19.4 二分图最大匹配	105
19.5 割点与桥	107
第二十章 树上算法	109
20.1 重链剖分与 LCA	109
第二十一章 字符串	112
21.1 前缀函数	112
21.2 KMP 匹配	112
21.3 Z 函数	113
21.4 Manacher 回文半径	113
21.5 最小循环表示	114
21.6 AC 自动机出现次数	114
21.7 倍增后缀数组与 LCP	117
21.8 后缀自动机	118
第二十二章 数论	120
22.1 模运算、素性、CRT 与类欧几里德	120
22.2 Pollard-Rho 因数分解	123
22.3 线性筛、欧拉函数与 Möbius 函数	124
22.4 静态模整数	125
22.5 阶乘组合数	126
第二十三章 多项式	127
23.1 NTT、形式幂级数、FWT 与递推	127
第二十四章 代数与数论进阶	132
24.1 模高斯消元、行列式、矩阵树定理	132
24.2 杜教筛	135
24.3 扩展 BSGS	136
第二十五章 计算几何	137
25.1 整数精确二维几何	137
25.2 浮点点线圆构造	140
第二十六章 优化与可持久化	144
26.1 Max-plus 矩阵幂	144
26.2 离散李超树与最优直线编号	144
26.3 主席树：静态区间第 k 小	146

第二十七章 图论进阶	149
27.1 矩形二分图最小权匹配	149
27.2 有向最小树形图	150
27.3 无向全局最小割	152
第二十八章 精确几何进阶	153
28.1 整数最近点对与 Minkowski 和	153
28.2 整数三维共线、共面与方向	156
第二十九章 动态树	157
29.1 Link-cut tree: 动态森林与路径异或	157
第三十章 随机平衡树	161
30.1 FHQ Treap: 可重集合与顺序统计	161
30.2 隐式 Treap: 序列翻转、加值与区间和	164
第三十一章 伸展树	168
31.1 Splay: 可重集合与顺序统计	168
第三十二章 状态序列维护	173
32.1 状态序列 Treap: 分组区间 GCD	173
第三部分 数学速查与维护	177
第三十三章 数学速查: 条件先于公式	178
33.1 整除、卷积与筛法	178
33.2 模运算的适用条件	178
33.3 组合计数	178
33.4 群作用计数	179
33.5 生成函数与递推	179
33.6 线性代数与生成树	179
33.7 概率、期望与博弈	179
33.8 数值算法与几何	179
第三十四章 来源与覆盖维护	180
34.1 参考基线	180
34.2 后续覆盖项	180

使用与验证

英文署名已于 QOJ 队伍账号 `ucup-team7502` 核对。两套风格依据训练提交 2932575、2930805、2931279 提取，不擅自将共享队号的提交归属到个人。

代码采用 C++20。以赛时快速抄写为先，一行一条语句，函数体展开。无状态的简单算法使用独立函数，有状态算法使用轻量 `struct`。动态版默认需要 `bits/stdc++.h` 和 `cassert`；传统版各头文件列明依赖。抄写单个函数或结构体时需同时抄写其依赖结构。同名的不同风格实现通常用于不同提交，不建议混在同一编译单元中。点号除特别注明外为 1-based，字符串和多项式为 0-based；所有区间约定应以条目说明为准。

测试源码位于 `tests/`，固定种子便于复现。验证包含独立暴力参考、边界案例、静态与动态实现、AddressSanitizer 和 UndefinedBehaviorSanitizer。`verification/` 保存实际运行记录与源码摘要；OJ 状态只在获得可查提交结果后登记。最大流记录为 <https://www.luogu.com.cn/record/297501480> 与 <https://www.luogu.com.cn/record/297502364>。

原始来源目录共登记 713 项，见 `docs/coverage.csv`。它包含不同版本、例题及公式，不应将其数目视为独立算法数，也不能将“已登记”视为“已完成”。

第一部分

动态容器风格

第一章 数据结构

1.1 并查集

`init(n)`、`find(x)`、`merge(x,y)`，点号 `1..n`；`siz[find(x)]` 为集合大小。路径压缩与按大小合并，均摊 $O(\alpha(n))$ 。

```
1 struct DSU
2 {
3     vector<int> fa, siz;
4
5     DSU(int n = 0)
6     {
7         init(n);
8     }
9
10    void init(int n)
11    {
12        fa.resize(n + 1);
13        iota(fa.begin(), fa.end(), 0);
14        siz.assign(n + 1, 1);
15    }
16
17    int find(int x)
18    {
19        while (x != fa[x])
20        {
21            fa[x] = fa[fa[x]];
22            x = fa[x];
23        }
24        return x;
25    }
26
27    bool merge(int x, int y)
28    {
29        x = find(x);
30        y = find(y);
31        if (x == y)
32            return false;
33        if (siz[x] < siz[y])
34            swap(x, y);
35        fa[y] = x;
36        siz[x] += siz[y];
37        return true;
38    }
39 };
```

1.2 可撤销并查集

构造后 `snapshot()` 保存历史长度, `rollback(t)` 回退。禁止路径压缩; `find/merge` 为 $O(\log n)$, 回退每次成功合并 $O(1)$ 。

```

1 struct RollbackDSU
2 {
3     vector<int> fa, siz;
4     vector<pair<int, int>> his;
5
6     RollbackDSU(int n) : fa(n + 1), siz(n + 1, 1)
7     {
8         iota(fa.begin(), fa.end(), 0);
9     }
10
11     int find(int x) const
12     {
13         while (x != fa[x])
14             x = fa[x];
15         return x;
16     }
17
18     int snapshot() const
19     {
20         return (int)his.size();
21     }
22
23     bool merge(int x, int y)
24     {
25         x = find(x);
26         y = find(y);
27         if (x == y)
28             return false;
29         if (siz[x] < siz[y])
30             swap(x, y);
31         his.push_back({y, siz[x]});
32         fa[y] = x;
33         siz[x] += siz[y];
34         return true;
35     }
36
37     void rollback(int t)
38     {
39         assert(0 <= t && t <= snapshot());
40         while (snapshot() > t)
41         {
42             auto [y, s] = his.back();
43             his.pop_back();
44             siz[fa[y]] = s;
45             fa[y] = y;
46         }
47     }
48 };

```

1.3 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。

```

1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1)
7     {
8     }
9
10    void add(int x, T v)
11    {
12        assert(x >= 1);
13        for (; x <= n; x += x & -x)
14            a[x] += v;
15    }
16
17    T sum(int x) const
18    {
19        T ans = 0;
20        for (; x; x -= x & -x)
21            ans += a[x];
22        return ans;
23    }
24
25    T query(int l, int r) const
26    {
27        return sum(r) - sum(l - 1);
28    }
29
30    // Nonnegative frequencies; returns n+1 if k exceeds total.
31    int kth(T k) const
32    {
33        assert(k > 0);
34        int x = 0, b = 1;
35        while (b <= n / 2)
36            b *= 2;
37        for (; b; b >>= 1)
38            if (x + b <= n && a[x + b] < k)
39            {
40                k -= a[x + b];
41                x += b;
42            }
43        return x + 1;
44    }
45 };

```

1.4 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1 \leq l \leq r \leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n;
4     vector<long long> sum, tag;
5
6     LazySeg(int n) : n(n), sum(4 * n + 4), tag(4 * n + 4)
7     {
8         assert(n > 0);
9     }
10
11     void apply(int p, int l, int r, long long v)
12     {
13         sum[p] += (r - l + 1) * v;
14         tag[p] += v;
15     }
16
17     void push(int p, int l, int r)
18     {
19         int m = (l + r) / 2;
20         apply(p * 2, l, m, tag[p]);
21         apply(p * 2 + 1, m + 1, r, tag[p]);
22         tag[p] = 0;
23     }
24
25     void add(int l, int r, long long v)
26     {
27         add(1, 1, n, l, r, v);
28     }
29
30     void add(int p, int l, int r, int ql, int qr, long long v)
31     {
32         if (ql <= l && r <= qr)
33         {
34             apply(p, l, r, v);
35             return;
36         }
37         push(p, l, r);
38         int m = (l + r) / 2;
39         if (ql <= m)
40             add(p * 2, l, m, ql, qr, v);
41         if (qr > m)
42             add(p * 2 + 1, m + 1, r, ql, qr, v);
43         sum[p] = sum[p * 2] + sum[p * 2 + 1];
44     }
45
46     long long query(int l, int r)
47     {
48         return query(1, 1, n, l, r);
49     }
50
51     long long query(int p, int l, int r, int ql, int qr)
52     {
53         if (ql <= l && r <= qr)
54             return sum[p];
55         push(p, l, r);

```

```

56     int m = (l + r) / 2;
57     long long ans = 0;
58     if (ql <= m)
59         ans += query(p * 2, l, m, ql, qr);
60     if (qr > m)
61         ans += query(p * 2 + 1, m + 1, r, ql, qr);
62     return ans;
63 }
64 };

```

1.5 64 位异或线性基

`insert(x)` 返回是否增加秩；`query(x)` 返回 `x` 与所张成空间异或后的最大无符号值。单次 $O(64)$ ，不提供第 k 小。

```

1 struct XorBasis
2 {
3     unsigned long long a[64]{};
4
5     bool insert(unsigned long long x)
6     {
7         for (int i = 63; i >= 0; i--)
8             if (x >> i & 1)
9             {
10                 if (!a[i])
11                 {
12                     a[i] = x;
13                     return true;
14                 }
15                 x ^= a[i];
16             }
17         return false;
18     }
19
20     unsigned long long query(unsigned long long x = 0) const
21     {
22         for (int i = 63; i >= 0; i--)
23             x = max(x, x ^ a[i]);
24         return x;
25     }
26 };

```

第二章 网络流

2.1 最大流与方案

Dinic(n), add(u,v,c) 返回偶数原边编号; flow(s,t,limit) 返回本次新增流; used(id) 查询原边净流, cut(s) 返回残量可达点集。允许重边、自环, 容量非负, $s \neq t$; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$, 递归增广深度 $O(V)$ 。总流不得超过 long long 上限。

```
1 struct Dinic
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll cap, initial;
9     };
10
11     int n;
12     vector<Edge> e;
13     vector<vector<int>>> g;
14     vector<int> dep, cur;
15
16     Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1)
17     {
18     }
19
20     int add(int u, int v, ll c)
21     {
22         assert(c >= 0);
23         int id = (int)e.size();
24         e.push_back({u, v, c, c});
25         e.push_back({v, u, 0, 0});
26         g[u].push_back(id);
27         g[v].push_back(id + 1);
28         return id;
29     }
30
31     bool bfs(int s, int t)
32     {
33         fill(dep.begin(), dep.end(), -1);
34         queue<int> q;
35         dep[s] = 0;
36         q.push(s);
37         while (!q.empty())
38         {
39             int u = q.front();
40             q.pop();
41             for (int id : g[u])
42                 if (e[id].cap && dep[e[id].to] < 0)
43                 {
44                     dep[e[id].to] = dep[u] + 1;
```

```

45         q.push(e[id].to);
46     }
47 }
48 return dep[t] >= 0;
49 }
50
51 ll dfs(int u, int t, ll f)
52 {
53     if (u == t)
54         return f;
55     for (int &i = cur[u]; i < (int)g[u].size(); i++)
56     {
57         int id = g[u][i], v = e[id].to;
58         if (!e[id].cap || dep[v] != dep[u] + 1)
59             continue;
60         ll d = dfs(v, t, min(f, e[id].cap));
61         if (d)
62         {
63             e[id].cap -= d;
64             e[id ^ 1].cap += d;
65             return d;
66         }
67     }
68     return 0;
69 }
70
71 // Returns additional flow; reuse only with the same source/sink.
72 ll flow(int s, int t, ll limit = LLONG_MAX)
73 {
74     assert(s != t && limit >= 0);
75     ll ans = 0;
76     while (ans < limit && bfs(s, t))
77     {
78         fill(cur.begin(), cur.end(), 0);
79         while (ans < limit)
80         {
81             ll d = dfs(s, t, limit - ans);
82             if (!d)
83                 break;
84             ans += d;
85         }
86     }
87     return ans;
88 }
89
90 ll used(int id) const
91 {
92     return e[id].initial - e[id].cap;
93 }
94
95 vector<int> cut(int s)

```

```

96     {
97         bfs(s, s);
98         vector<int> ans;
99         for (int u = 1; u <= n; u++)
100             if (dep[u] >= 0)
101                 ans.push_back(u);
102         return ans;
103     }
104 };

```

2.2 势能最小费用最大流

$\text{add}(u, v, \text{cap}, \text{cost})$ 返回边编号, $\text{flow}(s, t, \text{limit})$ 返回新增流量及 128 位费用, $\text{used}(id)$ 返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化 $O(VE)$, 每次增广 $O(E \log V)$; 增广次数不能简单假设为 $O(V)$ 。另要求 $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$, 防止总费用超过 int128 。

```

1 struct MinCostFlow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>> g;
15
16    MinCostFlow(int n) : n(n), g(n + 1)
17    {
18    }
19
20    int add(int u, int v, ll cap, ll cost)
21    {
22        assert(cap >= 0 && cost != LLONG_MIN);
23        int id = (int)e.size();
24        e.push_back({u, v, cap, cost, cap});
25        e.push_back({v, u, 0, -cost, 0});
26        g[u].push_back(id);
27        g[v].push_back(id + 1);
28        return id;
29    }
30
31    ll used(int id) const
32    {
33        return e[id].initial - e[id].cap;
34    }
35
36    // Negative costs allowed; initial residual graph must have no negative cycle.
37    pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
38    {
39        assert(s != t && limit >= 0);

```

```

40  const i128 inf = i128(1) << 120;
41  vector<i128> h(n + 1, 0), d(n + 1);
42  vector<int> pre(n + 1);
43  // Super-source Bellman-Ford: feasible potentials for every vertex.
44  for (int it = 0; it < n; it++)
45  {
46      bool changed = false;
47      for (auto &a : e)
48          if (a.cap && h[a.to] > h[a.from] + a.cost)
49          {
50              h[a.to] = h[a.from] + a.cost;
51              changed = true;
52          }
53      if (!changed)
54          break;
55      if (it == n - 1)
56          throw invalid_argument("negative cycle");
57  }
58  ll f = 0;
59  i128 cost = 0;
60  while (f < limit)
61  {
62      fill(d.begin(), d.end(), inf);
63      d[s] = 0;
64      priority_queue<pair<i128, int>,
65                    vector<pair<i128, int>>,
66                    greater<pair<i128, int>>>
67          q;
68      q.push({0, s});
69      while (!q.empty())
70      {
71          auto [du, u] = q.top();
72          q.pop();
73          if (du != d[u])
74              continue;
75          for (int id : g[u])
76          {
77              auto &a = e[id];
78              i128 nd = du + a.cost + h[u] - h[a.to];
79              if (a.cap && nd < d[a.to])
80              {
81                  d[a.to] = nd;
82                  pre[a.to] = id;
83                  q.push({nd, a.to});
84              }
85          }
86      }
87      if (d[t] == inf)
88          break;
89      for (int u = 1; u <= n; u++)
90          if (d[u] != inf)
91              h[u] += d[u];
92      ll take = limit - f;
93      for (int u = t; u != s; u = e[pre[u]].from)
94          take = min(take, e[pre[u]].cap);
95      for (int u = t; u != s; u = e[pre[u]].from)

```

```

96         {
97             int id = pre[u];
98             e[id].cap -= take;
99             e[id ^ 1].cap += take;
100             cost += i128(take) * e[id].cost;
101         }
102         f += take;
103     }
104     return {f, cost};
105 }
106 };

```

2.3 上下界可行循环流

$\text{add}(u, v, lo, hi)$, $0 \leq lo \leq hi$; $\text{solve}()$ 仅调用一次, 返回可行性, 仅成功后读取 $\text{used}(id)$ 。约束为每点流量守恒; 不是带源汇最大流。所有上下界及其总和须在 long long 范围内。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll = long long;
4     int n;
5     Dinic g;
6     vector<ll> balance, lower;
7     vector<int> ids;
8     bool solved = false, feasible = false;
9
10    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1)
11    {
12    }
13
14    int add(int u, int v, ll lo, ll hi)
15    {
16        assert(!solved && 0 <= lo && lo <= hi);
17        int id = ids.size();
18        ids.push_back(g.add(u, v, hi - lo));
19        lower.push_back(lo);
20        balance[u] -= lo;
21        balance[v] += lo;
22        return id;
23    }
24
25    bool solve()
26    {
27        assert(!solved);
28        solved = true;
29        ll need = 0;
30        for (int u = 1; u <= n; u++)
31            if (balance[u] > 0)
32            {
33                g.add(n + 1, u, balance[u]);
34                need += balance[u];
35            }
36        else if (balance[u] < 0)

```

```
37         g.add(u, n + 2, -balance[u]);
38     return feasible = (g.flow(n + 1, n + 2) == need);
39 }
40
41 ll used(int id) const
42 {
43     assert(feasible);
44     return lower[id] + g.used(ids[id]);
45 }
46 };
```

第三章 图论

3.1 非负权最短路

`add(u,v,w)`、`run(s)`，`dis` 为距离，`pre` 为前驱，`path(t)` 返回点序列。不可达用 `LLONG_MAX`；所有可表示的最短距离必须严格小于该哨兵， $w \geq 0$ 。 $O((V+E)\log V)$ 。

```
1 struct Dijkstra
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Dijkstra(int n) : n(n), g(n + 1)
11    {
12    }
13
14    void add(int u, int v, ll w)
15    {
16        assert(w >= 0);
17        g[u].push_back({v, w});
18    }
19
20    void run(int s)
21    {
22        dis.assign(n + 1, inf);
23        pre.assign(n + 1, -1);
24        dis[s] = 0;
25        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
26        q.push({0, s});
27        while (!q.empty())
28        {
29            auto [d, u] = q.top();
30            q.pop();
31            if (d != dis[u])
32                continue;
33            for (auto [v, w] : g[u])
34                if (w < inf - d && d + w < dis[v])
35                {
36                    dis[v] = d + w;
37                    pre[v] = u;
38                    q.push({dis[v], v});
39                }
40        }
41    }
42
43    vector<int> path(int t) const
44    {
```

```

45     if (dis[t] == inf)
46         return {};
47     vector<int> ans;
48     for (; t != -1; t = pre[t])
49         ans.push_back(t);
50     reverse(ans.begin(), ans.end());
51     return ans;
52 }
53 };

```

3.2 强连通分量

$\text{add}(u,v)$ 、 $\text{run}()$ 、 $\text{bel}[1..n]$ 为分量编号， cnt 为数量；分量编号按缩点图拓扑序递增。迭代 Kosaraju 避免递归爆栈； $O(V+E)$ ，同时存正反图。

```

1 struct SCC
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     SCC(int n) : n(n), g(n + 1), rg(n + 1)
8     {
9     }
10
11     void add(int u, int v)
12     {
13         g[u].push_back(v);
14         rg[v].push_back(u);
15     }
16
17     // Iterative Kosaraju; component IDs are in topological order, 1..cnt.
18     void run()
19     {
20         vector<int> vis(n + 1), order;
21         cnt = 0;
22         bel.assign(n + 1, 0);
23         for (int s = 1; s <= n; s++)
24             if (!vis[s])
25             {
26                 vector<pair<int, int>> st{{s, 0}};
27                 vis[s] = 1;
28                 while (!st.empty())
29                 {
30                     int u = st.back().first, &i = st.back().second;
31                     if (i == (int)g[u].size())
32                     {
33                         order.push_back(u);
34                         st.pop_back();
35                     }
36                     else
37                     {
38                         int v = g[u][i++];
39                         if (!vis[v])
40                             st.push_back({v, 0});

```

```
41         vis[v] = 1;
42         st.push_back({v, 0});
43     }
44 }
45 }
46 }
47 reverse(order.begin(), order.end());
48 for (int s : order)
49     if (!bel[s])
50     {
51         ++cnt;
52         vector<int> st{s};
53         bel[s] = cnt;
54         while (!st.empty())
55         {
56             int u = st.back();
57             st.pop_back();
58             for (int v : rg[u])
59                 if (!bel[v])
60                 {
61                     bel[v] = cnt;
62                     st.push_back(v);
63                 }
64         }
65     }
66 }
67 };
```

3.3 2-SAT 与任意可行方案

`add(x,a,y,b)` 添加 $(x=a)$ 或 $(y=b)$, 变量 $1..n$; `solve()` 返回可满足性, 仅成功时读取 `ans`。依赖同文件 `SCC`; $O(V+E)$, 不保证字典序最小。

```

1 struct TwoSAT
2 {
3     int n;
4     SCC g;
5     vector<int> ans;
6
7     TwoSAT(int n) : n(n), g(2 * n)
8     {
9     }
10
11     int id(int x, bool value) const
12     {
13         return 2 * x - 1 + value;
14     }
15
16     void add(int x, bool a, int y, bool b)
17     {
18         g.add(id(x, !a), id(y, b));
19         g.add(id(y, !b), id(x, a));
20     }
21
22     bool solve()
23     {
24         g.run();
25         ans.assign(n + 1, 0);
26         for (int x = 1; x <= n; x++)
27         {
28             if (g.bel[id(x, 0)] == g.bel[id(x, 1)])
29                 return false;
30             ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
31         }
32         return true;
33     }
34 };

```

3.4 二分图最大匹配

左右两侧分别编号 $1..n$ 、 $1..m$; `add(u,v)`, `solve()` 返回匹配数, $l[u]$ 与 $r[v]$ 为匹配端点, 0 表示未匹配。分层增广; 本版本保守按 $O(VE)$ 估计, 不宣称已经验证 Hopcroft-Karp 最紧复杂度。

```

1 struct BipartiteMatching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep;
6
7     BipartiteMatching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1)
9     {
10     }

```

```
11
12 void add(int u, int v)
13 {
14     g[u].push_back(v);
15 }
16
17 bool bfs()
18 {
19     queue<int> q;
20     fill(dep.begin(), dep.end(), -1);
21     for (int u = 1; u <= n; u++)
22         if (!l[u])
23         {
24             dep[u] = 0;
25             q.push(u);
26         }
27     bool ok = false;
28     while (!q.empty())
29     {
30         int u = q.front();
31         q.pop();
32         for (int v : g[u])
33             if (!r[v])
34                 ok = true;
35             else if (dep[r[v]] < 0)
36             {
37                 dep[r[v]] = dep[u] + 1;
38                 q.push(r[v]);
39             }
40     }
41     return ok;
42 }
43
44 bool dfs(int u)
45 {
46     for (int v : g[u])
47         if (!r[v] || (dep[r[v]] == dep[u] + 1 && dfs(r[v])))
48         {
49             l[u] = v;
50             r[v] = u;
51             return true;
52         }
53     dep[u] = -1;
54     return false;
55 }
56
57 int solve()
58 {
59     int ans = 0;
```

```
60     for (int u = 1; u <= n; u++)
61         ans += (l[u] != 0);
62     while (bfs())
63         for (int u = 1; u <= n; u++)
64             if (!l[u])
65                 ans += dfs(u);
66     return ans;
67 }
68 };
```

3.5 割点与桥

`add(u,v)` 返回无向边编号；`run()` 后 `cut[u]` 标识割点、`bridge[id]` 标识桥。按边编号跳过父边，支持重边、自环、不连通图。 $O(V+E)$ ，递归深度可能 $O(V)$ 。

```

1 struct Lowlink
2 {
3     int n, timer = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge;
6     int edges = 0;
7
8     Lowlink(int n) : n(n), g(n + 1)
9     {
10    }
11
12    int add(int u, int v)
13    {
14        g[u].push_back({v, edges});
15        g[v].push_back({u, edges});
16        return edges++;
17    }
18
19    void dfs(int u, int pe)
20    {
21        dfn[u] = low[u] = ++timer;
22        int children = 0;
23        for (auto [v, id] : g[u])
24        {
25            if (id == pe)
26                continue;
27            if (!dfn[v])
28            {
29                ++children;
30                dfs(v, id);
31                low[u] = min(low[u], low[v]);
32                if (low[v] > dfn[u])
33                    bridge[id] = 1;
34                if (pe != -1 && low[v] >= dfn[u])
35                    cut[u] = 1;
36            }
37            else
38                low[u] = min(low[u], dfn[v]);
39        }
40        if (pe == -1 && children > 1)
41            cut[u] = 1;
42    }
43
44    void run()
45    {

```

```
46     timer = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     cut = dfn;
50     bridge.assign(edges, 0);
51     for (int u = 1; u <= n; u++)
52         if (!dfn[u])
53             dfs(u, -1);
54     }
55 };
```

第四章 树上算法

4.1 重链剖分与 LCA

add(u,v) 建无向树; build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 $O(\log n)$, path(u,v,work,edge) 分解路径; work(l,r) 仅适用于可交换聚合, 边权模式排除 LCA 位置。输入必须是连通无环树, 建树 $O(n)$, 无递归。

```
1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     // Connected tree only; iterative build avoids chain-shaped DFS stack overflow.
27     void build(int root = 1)
28     {
29         fill(fa.begin(), fa.end(), 0);
30         fill(son.begin(), son.end(), 0);
31         vector<int> order{root};
32         fa[root] = root;
33         dep[root] = 0;
34         for (int i = 0; i < (int)order.size(); i++)
35         {
36             int u = order[i];
37             for (int v : g[u])
38                 if (v != fa[u])
39                 {
40                     fa[v] = u;
41                     dep[v] = dep[u] + 1;
42                     order.push_back(v);
43                 }
44         }
```

```

45     assert((int)order.size() == n);
46     for (int i = n - 1; i >= 0; i--)
47     {
48         int u = order[i];
49         siz[u] = 1;
50         for (int v : g[u])
51             if (fa[v] == u && v != u)
52             {
53                 siz[u] += siz[v];
54                 if (!son[u] || siz[v] > siz[son[u]])
55                     son[u] = v;
56             }
57     }
58     timer = 0;
59     vector<pair<int, int>> st{{root, root}};
60     while (!st.empty())
61     {
62         auto [u, t] = st.back();
63         st.pop_back();
64         for (; u; u = son[u])
65         {
66             top[u] = t;
67             dfn[u] = ++timer;
68             rk[timer] = u;
69             for (int v : g[u])
70                 if (fa[v] == u && v != u && v != son[u])
71                     st.push_back({v, v});
72         }
73     }
74 }
75
76 int lca(int u, int v) const
77 {
78     while (top[u] != top[v])
79     {
80         if (dep[top[u]] < dep[top[v]])
81             swap(u, v);
82         u = fa[top[u]];
83     }
84     return dep[u] < dep[v] ? u : v;
85 }
86
87 // Commutative operations only; edge=true excludes the LCA's position.
88 template <class F> void path(int u, int v, F work, bool edge = false) const
89 {
90     while (top[u] != top[v])
91     {
92         if (dep[top[u]] < dep[top[v]])

```

```
93         swap(u, v);
94         work(dfn[top[u]], dfn[u]);
95         u = fa[top[u]];
96     }
97     if (dep[u] > dep[v])
98         swap(u, v);
99     if (dfn[u] + edge <= dfn[v])
100         work(dfn[u] + edge, dfn[v]);
101 }
102 };
```

第五章 字符串

5.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 inline vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j])
9             j = p[j - 1];
10        p[i] = j + (s[i] == s[j]);
11    }
12    return p;
13 }
```

5.2 KMP 匹配

`kmp_match(s,t)` 返回非空模式串 `t` 在 `s` 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 inline vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j])
9             j = p[j - 1];
10        j += (s[i] == t[j]);
11        if (j == (int)t.size())
12        {
13            ans.push_back(i - j + 1);
14            j = p[j - 1];
15        }
16    }
17    return ans;
18 }
```

5.3 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP，约定 `z[0]=n`，空串返回空数组。 $O(n)$ 。

```

1 inline vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n)
6         a[0] = n;
7     for (int i = 1, l = 0, r = 0; i < n; i++)
8     {
9         if (i < r)
10            a[i] = min(r - i, a[i - l]);
11        while (i + a[i] < n && s[a[i]] == s[i + a[i]])
12            ++a[i];
13        if (i + a[i] > r)
14        {
15            l = i;
16            r = i + a[i];
17        }
18    }
19    return a;
20 }
```

5.4 Manacher 回文半径

`manacher(s)` 返回 `odd` 与 `even`；`odd[i]` 的半径包含中心，`even[i]` 的中心在 `i` 前。字符串及下标为 0-based，空串合法， $O(n)$ 。

```

1 // odd[i] includes the center; even[i] is centered before i.
2 inline pair<vector<int>, vector<int>> manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for (int i = 0, l = 0, r = -1; i < n; i++)
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
10             ++k;
11        odd[i] = k--;
12        if (i + k > r)
13        {
14            l = i - k;
15            r = i + k;
16        }
17    }
18    for (int i = 0, l = 0, r = -1; i < n; i++)
19    {
20        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
21        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
22            ++k;

```

```

23     even[i] = k--;
24     if (i + k > r)
25     {
26         l = i - k - 1;
27         r = i + k;
28     }
29 }
30 return {odd, even};
31 }

```

5.5 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。 $O(n)$ 。

```

1 inline int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n)
5         return 0;
6     while (i < n && j < n && k < n)
7     {
8         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
9         if (a == b)
10            {
11                ++k;
12                continue;
13            }
14         if (a > b)
15            {
16                i += k + 1;
17                if (i == j)
18                    ++i;
19            }
20         else
21            {
22                j += k + 1;
23                if (i == j)
24                    ++j;
25            }
26         k = 0;
27     }
28     return min(i, j);
29 }

```

5.6 AC 自动机出现次数

字母表 `a..z`, `add(s)` 返回终止状态, 要求模式非空; 全部插入后 `build()`, 只允许一次构建。`count(text)` 返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建 $O(26N)$, 查询 $O(|text|+N)$ 。

```

1 struct AhoCorasick
2 {
3     struct Node
4     {

```

```

5      array<int, 26> go{};
6      int fail = 0;
7  };
8
9  vector<Node> a{Node{}};
10 vector<int> order;
11 bool built = false;
12
13 int add(const string &s)
14 {
15     assert(!built && !s.empty());
16     int u = 0;
17     for (char ch : s)
18     {
19         int c = ch - 'a';
20         assert(0 <= c && c < 26);
21         if (!a[u].go[c])
22         {
23             int v = (int)a.size();
24             a[u].go[c] = v;
25             a.push_back(Node{});
26         }
27         u = a[u].go[c];
28     }
29     return u;
30 }
31
32 void build()
33 {
34     assert(!built);
35     built = true;
36     queue<int> q;
37     for (int v : a[0].go)
38         if (v)
39             q.push(v);
40     while (!q.empty())
41     {
42         int u = q.front();
43         q.pop();
44         order.push_back(u);
45         for (int c = 0; c < 26; c++)
46         {
47             int v = a[u].go[c];
48             if (v)
49             {
50                 a[v].fail = a[a[u].fail].go[c];
51                 q.push(v);
52             }
53             else
54                 a[u].go[c] = a[a[u].fail].go[c];
55         }
56     }
57 }
58
59 vector<long long> count(const string &s) const
60 {

```

```
61     assert(built);
62     vector<long long> ans(a.size());
63     int u = 0;
64     for (char ch : s)
65     {
66         u = a[u].go[ch - 'a'];
67         ++ans[u];
68     }
69     for (int i = (int)order.size() - 1; i >= 0; i--)
70         ans[a[order[i]].fail] += ans[order[i]];
71     return ans;
72 }
73 };
```

5.7 倍增后缀数组与 LCP

$sa[k]$ 为第 k 个后缀起点, $rk[i]$ 为排名, $lcp[k]$ 为 $sa[k-1]$ 与 $sa[k]$ 的 LCP ($lcp[0]=0$)。空串合法; 当前排序比较版 $O(n \log^2 n)$, 不能替代待补的线性 SA/DC3 条目。

```

1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         int n = (int)s.size();
8         sa.resize(n);
9         rk.resize(n);
10        lcp.assign(n, 0);
11        for (int i = 0; i < n; i++)
12        {
13            sa[i] = i;
14            rk[i] = (unsigned char)s[i];
15        }
16        vector<int> tmp(n);
17        for (int k = 1; k < n; k *= 2)
18        {
19            auto key = [&](int i)
20            {
21                return pair{rk[i], i + k < n ? rk[i + k] : -1};
22            };
23            sort(sa.begin(),
24                sa.end(),
25                [&](int i, int j)
26                {
27                    return key(i) < key(j);
28                });
29            tmp[sa[0]] = 0;
30            for (int i = 1; i < n; i++)
31                tmp[sa[i]] = tmp[sa[i - 1]] + (key(sa[i]) != key(sa[i - 1]));
32            rk = tmp;
33            if (rk[sa.back()] == n - 1)
34                break;
35        }
36        for (int i = 0; i < n; i++)
37            rk[sa[i]] = i;
38        for (int i = 0, k = 0; i < n; i++)
39        {
40            if (!rk[i])
41            {
42                k = 0;
43                continue;
44            }

```

```

45     int j = sa[rk[i] - 1];
46     while (i + k < n && j + k < n && s[i + k] == s[j + k])
47         ++k;
48     lcp[rk[i]] = k;
49     if (k)
50         --k;
51 }
52 }
53 };

```

5.8 后缀自动机

extend(c) 接收整数 0..25; distinct() 返回不同非空子串数, counts() 返回 endpos 次数且不修改结构, 可重复调用。状态 a, 初始状态 0, link 根为 -1; 空间 $O(26n)$, 构建 $O(26n)$ 。

```

1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while (p != -1 && !a[p].go[c])
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if (p == -1)
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if (a[p].len + 1 == a[q].len)
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while (p != -1 && a[p].go[c] == q)

```

```

41         {
42             a[p].go[c] = clone;
43             p = a[p].link;
44         }
45         a[q].link = a[cur].link = clone;
46     }
47 }
48 last = cur;
49 }
50
51 vector<long long> counts() const
52 {
53     vector<int> bucket(a[last].len + 1), order(a.size());
54     vector<long long> ans(a.size());
55     for (auto &v : a)
56         ++bucket[v.len];
57     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
58     for (int i = 0; i < (int)a.size(); i++)
59     {
60         order[--bucket[a[i].len]] = i;
61         ans[i] = a[i].occ;
62     }
63     for (int i = (int)order.size() - 1; i > 0; i--)
64         ans[a[order[i]].link] += ans[order[i]];
65     return ans;
66 }
67
68 long long distinct() const
69 {
70     long long ans = 0;
71     for (int i = 1; i < (int)a.size(); i++)
72         ans += a[i].len - a[a[i].link].len;
73     return ans;
74 }
75 };

```

第六章 数论

6.1 模运算、素性、CRT 与类欧几里德

mul/power 支持无符号 64 位正模数，乘积用 uint128。prime 使用确定性 64 位底数。inverse(a,m) 无逆元返回 -1。crt(r,m,b,n) 合并同余式，无解返回 false，模数溢出抛异常。floor_sum 为 $i=0..n-1$ 的 $\text{floor}((ai+b)/m)$ 之和，支持负 a、b，返回 int128；约定 $n \leq 10^9$ 、 $m \leq 10^9$ ，a、b 为有符号 64 位。

```
1 struct NumberTheory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7
8     static ull mul(ull a, ull b, ull m)
9     {
10         return u128(a) * b % m;
11     }
12
13     static ull power(ull a, ull b, ull m)
14     {
15         assert(m);
16         ull r = 1 % m;
17         a %= m;
18         for (; b; b >>= 1, a = mul(a, a, m))
19             if (b & 1)
20                 r = mul(r, a, m);
21         return r;
22     }
23
24     static bool prime(ull n)
25     {
26         if (n < 2)
27             return false;
28         for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
29             if (n % p == 0)
30                 return n == p;
31         ull d = n - 1;
32         int s = 0;
33         while (!(d & 1))
34         {
35             d >>= 1;
36             ++s;
37         }
38         for (ull a :
39             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
40         {
41             if (a % n == 0)
42                 continue;
43             ull x = power(a, d, n);
44             if (x == 1 || x == n - 1)
```

```

45         continue;
46         bool ok = false;
47         for (int j = 1; j < s; j++)
48         {
49             x = mul(x, x, n);
50             if (x == n - 1)
51             {
52                 ok = true;
53                 break;
54             }
55         }
56         if (!ok)
57             return false;
58     }
59     return true;
60 }
61
62 static i128 exgcd(i128 a, i128 b, i128 &x, i128 &y)
63 {
64     if (!b)
65     {
66         x = 1;
67         y = 0;
68         return a;
69     }
70     i128 d = exgcd(b, a % b, y, x);
71     y -= a / b * x;
72     return d;
73 }
74
75 static ll inverse(ll a, ll m)
76 {
77     assert(m > 0);
78     a %= m;
79     if (a < 0)
80         a += m;
81     i128 x, y;
82     if (exgcd(a, m, x, y) != 1)
83         return -1;
84     return (x % m + m) % m;
85 }
86
87 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
88 static bool crt(ll &r, ll &m, ll b, ll n)
89 {
90     assert(m > 0 && n > 0);
91     r %= m;
92     if (r < 0)
93         r += m;
94     b %= n;
95     if (b < 0)
96         b += n;
97     i128 x, y;
98     ll g = (ll)exgcd(m, n, x, y);
99     i128 diff = i128(b) - r;
100    if (diff % g)

```

```

101         return false;
102     i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
103     if (mod > LLONG_MAX)
104         throw overflow_error("CRT modulus");
105     r = (r + i128(m) * k) % mod;
106     m = (ll)mod;
107     return true;
108 }
109
110 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
111 static i128 floor_sum(ll n, ll m, ll a, ll b)
112 {
113     assert(n >= 0 && m > 0);
114     i128 ans = 0;
115     auto norm = [&](ll &v)
116     {
117         ll q = v / m;
118         if (v % m < 0)
119             --q;
120         v = (ll)(i128(v) - i128(q) * m);
121         return q;
122     };
123     ans += i128(norm(a)) * n * (n - 1) / 2;
124     ans += i128(norm(b)) * n;
125     while (true)
126     {
127         ans += i128(n) * (n - 1) / 2 * (a / m);
128         a %= m;
129         ans += i128(n) * (b / m);
130         b %= m;
131         i128 y = i128(a) * n + b;
132         if (y < m)
133             break;
134         n = (ll)(y / m);
135         b = (ll)(y % m);
136         swap(a, m);
137     }
138     return ans;
139 }
140 };

```

6.2 Pollard–Rho 因数分解

`factor(n)` 返回升序质因子（含重数）， $n \geq 1$ ；依赖 `NumberTheory`。返回因子经过整除和素性检查；Las Vegas 随机运行时间，固定默认种子方便复现，可由调用者更换。无确定性最坏时间保证。

```

1 struct PollardRho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     PollardRho(ull seed = 712367821) : rng(seed)
8     {
9     }
10
11     ull rho(ull n)
12     {
13         if (n % 2 == 0)
14             return 2;
15         for (;;)
16         {
17             ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
18             auto f = [&](ull v)
19             {
20                 return (NumberTheory::mul(v, v, n) + u128(c)) % n;
21             };
22             // Retry bounded attempts; Las Vegas: only exact divisors returned.
23             for (int i = 0; i < 200000 && d == 1; i++)
24             {
25                 x = f(x);
26                 y = f(f(y));
27                 d = gcd(x > y ? x - y : y - x, n);
28             }
29             if (1 < d && d < n)
30                 return d;
31         }
32     }
33
34     void split(ull n, vector<ull> &a)
35     {
36         if (n == 1)
37             return;
38         if (NumberTheory::prime(n))
39         {
40             a.push_back(n);
41             return;
42         }
43         ull d = rho(n);
44         split(d, a);
45         split(n / d, a);
46     }

```

```

47
48     vector<ull> factor(ull n)
49     {
50         assert(n >= 1);
51         vector<ull> a;
52         split(n, a);
53         sort(a.begin(), a.end());
54         return a;
55     }
56 };

```

6.3 线性筛、欧拉函数与 Möbius 函数

构造参数 $n \geq 0$, prime 为质数表, lp/phi/mu 为 $0..n$ 数组。phi[1]=mu[1]=1, $O(n)$ 时间和空间。

```

1 struct LinearSieve
2 {
3     vector<int> prime, lp, phi, mu;
4
5     LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if (n)
8             phi[1] = mu[1] = 1;
9         for (int i = 2; i <= n; i++)
10         {
11             if (!lp[i])
12             {
13                 lp[i] = i;
14                 prime.push_back(i);
15                 phi[i] = i - 1;
16                 mu[i] = -1;
17             }
18             for (int p : prime)
19             {
20                 if (p > n / i)
21                     break;
22                 int j = i * p;
23                 lp[j] = p;
24                 if (i % p == 0)
25                 {
26                     phi[j] = phi[i] * p;
27                     mu[j] = 0;
28                     break;
29                 }
30                 phi[j] = phi[i] * (p - 1);
31                 mu[j] = -mu[i];
32             }
33         }
34     }
35 };

```

6.4 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。
复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。

```

1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod)
6     {
7     }
8
9     ModInt operator+(ModInt b) const
10    {
11        return ModInt((long long)v + b.v);
12    }
13
14    ModInt operator-(ModInt b) const
15    {
16        return ModInt((long long)v - b.v);
17    }
18
19    ModInt operator*(ModInt b) const
20    {
21        return ModInt(1LL * v * b.v);
22    }
23
24    ModInt pow(long long e) const
25    {
26        assert(e >= 0);
27        ModInt a = *this, r = 1;
28        for (; e >>= 1, a = a * a)
29            if (e & 1)
30                r = r * a;
31        return r;
32    }
33
34    ModInt inv() const
35    {
36        assert(v);
37        return pow(mod - 2);
38    } // prime modulus
39
40    ModInt operator/(ModInt b) const
41    {
42        return *this * b.inv();
43    }
44 };

```

6.5 阶乘组合数

$\text{Binomial}<\text{prime}>(n)$, 要求 $0 \leq n < \text{prime}$; $\text{choose}(n, k)$ 越界 k 返回 0。预处理 $O(n + \log \text{prime})$, 单次 $O(1)$ 。
尚不能替代合数模数 exLucas 。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac, ifac;
5
6     Binomial(int n) : fac(n + 1), ifac(n + 1)
7     {
8         assert(0 <= n && n < mod);
9         fac[0] = 1;
10        for (int i = 1; i <= n; i++)
11            fac[i] = fac[i - 1] * i;
12        ifac[n] = fac[n].inv();
13        for (int i = n; i; i--)
14            ifac[i - 1] = ifac[i] * i;
15    }
16
17    Z choose(int n, int k) const
18    {
19        if (k < 0 || k > n)
20            return 0;
21        assert(n < (int)fac.size());
22        return fac[n] * ifac[k] * ifac[n - k];
23    }
24 };
```

第七章 多项式

7.1 NTT、形式幂级数、FWT 与递推

依赖 number_theory.hpp。模数固定 998244353，乘法结果长度 $\leq 2^{23}$ ；inverse 要求 $a[0] \neq 0$ ，log 要求 $a[0]=1$ ，exp 要求 $a[0]=0$ ，截断长度 $n>0$ 。朴素逐层 NTT 实现的 FPS 为 $O(n \log n)$ 。FWT 的 op 为 $\wedge$ 、 $|$ 、 $\&$ ；BM 返回 $s[n]=\sum c[i-1]s[n-i]$ 的系数；recurrence 用 $O(k^2 \log n)$ 求第 n 项。BM 对有限样本给出的递推不自动保证未知后续项成立。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void ntt(Poly &a, bool invert = false)
8     {
9         int n = (int)a.size();
10        assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
11        for (int i = 1, j = 0; i < n; i++)
12        {
13            int bit = n >> 1;
14            for (; j & bit; bit >>= 1)
15                j ^= bit;
16            j ^= bit;
17            if (i < j)
18                swap(a[i], a[j]);
19        }
20        for (int len = 2; len <= n; len *= 2)
21        {
22            Z step = Z(3).pow((mod - 1) / len);
23            if (invert)
24                step = step.inv();
25            for (int i = 0; i < n; i += len)
26            {
27                Z w = 1;
28                for (int j = 0; j < len / 2; j++, w = w * step)
29                {
30                    Z u = a[i + j], v = a[i + j + len / 2] * w;
31                    a[i + j] = u + v;
32                    a[i + j + len / 2] = u - v;
33                }
34            }
35        }
36        if (invert)
37        {
38            Z inv = Z(n).inv();
39            for (auto &x : a)
40                x = x * inv;
41        }
42    }
43 }
```

```

44 static Poly multiply(Poly a, Poly b)
45 {
46     if (a.empty() || b.empty())
47         return {};
48     int size = (int)a.size() + (int)b.size() - 1, n = 1;
49     assert(size <= (1 << 23));
50     while (n < size)
51         n *= 2;
52     a.resize(n);
53     b.resize(n);
54     ntt(a);
55     ntt(b);
56     for (int i = 0; i < n; i++)
57         a[i] = a[i] * b[i];
58     ntt(a, true);
59     a.resize(size);
60     return a;
61 }
62
63 static Poly inverse(const Poly &a, int n)
64 {
65     assert(n > 0 && !a.empty() && a[0].v);
66     Poly b{a[0].inv()};
67     while ((int)b.size() < n)
68     {
69         int m = min(n, 2 * (int)b.size());
70         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
71         auto t = multiply(f, b);
72         t.resize(m);
73         for (auto &x : t)
74             x = Z(0) - x;
75         t[0] = t[0] + 2;
76         b = multiply(b, t);
77         b.resize(m);
78     }
79     return b;
80 }
81
82 static Poly derivative(const Poly &a)
83 {
84     Poly b;
85     for (int i = 1; i < (int)a.size(); i++)
86         b.push_back(a[i] * i);
87     return b;
88 }
89
90 static Poly integral(const Poly &a)
91 {
92     Poly b(a.size() + 1);
93     vector<Z> inv(a.size() + 1);
94     if (!a.empty())
95         inv[1] = 1;
96     for (int i = 2; i <= (int)a.size(); i++)
97         inv[i] = Z(mod - mod / i) * inv[mod % i];
98     for (int i = 0; i < (int)a.size(); i++)
99         b[i + 1] = a[i] * inv[i + 1];

```

```

100     return b;
101 }
102
103 static Poly log(const Poly &a, int n)
104 {
105     assert(n > 0 && !a.empty() && a[0].v == 1);
106     auto b = multiply(derivative(a), inverse(a, n));
107     b.resize(n - 1);
108     return integral(b);
109 }
110
111 static Poly exp(const Poly &a, int n)
112 {
113     assert(n > 0 && (a.empty() || a[0].v == 0));
114     Poly b{1};
115     while ((int)b.size() < n)
116     {
117         int m = min(n, 2 * (int)b.size());
118         auto t = log(b, m);
119         for (int i = 0; i < m; i++)
120             t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
121         t[0] = t[0] + 1;
122         b = multiply(b, t);
123         b.resize(m);
124     }
125     return b;
126 }
127
128 static void fwt(Poly &a, char op, bool invert = false)
129 {
130     int n = (int)a.size();
131     assert(n > 0 && (n & (n - 1)) == 0);
132     const Z inv2 = (mod + 1) / 2;
133     for (int len = 1; len < n; len *= 2)
134         for (int i = 0; i < n; i += len * 2)
135             for (int j = 0; j < len; j++)
136             {
137                 Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
138                 if (op == '^')
139                 {
140                     u = x + y;
141                     v = x - y;
142                     if (invert)
143                     {
144                         u = u * inv2;
145                         v = v * inv2;
146                     }
147                 }
148                 else if (op == '|')
149                     v = invert ? y - x : y + x;
150                 else
151                 {
152                     assert(op == '&');
153                     u = invert ? x - y : x + y;
154                 }
155             }

```

```

156     }
157
158     // c: s[n]=sum c[i-1]*s[n-i], i=1..L.
159     static Poly berlekamp_massey(const Poly &s)
160     {
161         Poly c{1}, b{1};
162         int l = 0, m = 1;
163         Z last = 1;
164         for (int n = 0; n < (int)s.size(); n++)
165         {
166             Z d = s[n];
167             for (int i = 1; i <= l; i++)
168                 d = d + c[i] * s[n - i];
169             if (!d.v)
170             {
171                 ++m;
172                 continue;
173             }
174             auto old = c;
175             Z coef = d / last;
176             if (c.size() < b.size() + m)
177                 c.resize(b.size() + m);
178             for (int j = 0; j < (int)b.size(); j++)
179                 c[j + m] = c[j + m] - coef * b[j];
180             if (2 * l <= n)
181             {
182                 l = n + 1 - l;
183                 b = old;
184                 last = d;
185                 m = 1;
186             }
187             else
188                 ++m;
189         }
190         c.erase(c.begin());
191         for (auto &x : c)
192             x = Z(0) - x;
193         return c;
194     }
195
196     static Z recurrence(Poly init, const Poly &c, unsigned long long n)
197     {
198         int k = (int)c.size();
199         if (n < init.size())
200             return init[n];
201         if (!k)
202             return 0;
203         assert((int)init.size() >= k);
204         auto combine = [&](const Poly &a, const Poly &b)
205         {
206             Poly t(2 * k - 1);
207             for (int i = 0; i < k; i++)
208                 for (int j = 0; j < k; j++)
209                     t[i + j] = t[i + j] + a[i] * b[j];
210             for (int i = 2 * k - 2; i >= k; i--)
211                 for (int j = 1; j <= k; j++)

```

```
212         t[i - j] = t[i - j] + t[i] * c[j - 1];
213     t.resize(k);
214     return t;
215 };
216 Poly a(k), r(k);
217 r[0] = 1;
218 if (k == 1)
219     a[0] = c[0];
220 else
221     a[1] = 1;
222 for (; n; n >= 1, a = combine(a, a))
223     if (n & 1)
224         r = combine(r, a);
225 Z ans = 0;
226 for (int i = 0; i < k; i++)
227     ans = ans + r[i] * init[i];
228 return ans;
229 }
230 };
```

第八章 代数与数论进阶

8.1 模高斯消元、行列式、矩阵树定理

素数模数。solve(a,n) 输入增广矩阵，返回 consistent、rank、特解 particular 和零空间基 kernel。determinant(empty)=1。spanning_trees(n,edges) 使用 0-based 无向图，允许重边，忽略自环；返回生成树数模 p。高斯与行列式 $O(n^3)$ ，矩阵幂 $O(n^3 \log e)$ 。

```
1 template <int mod> struct LinearAlgebra
2 {
3     using Z = ModInt<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for (const auto &v : a)
20            assert((int)v.size() == n + 1);
21        for (int col = 0; col < n && row < m; col++)
22        {
23            int p = row;
24            while (p < m && !a[p][col].v)
25                ++p;
26            if (p == m)
27                continue;
28            swap(a[p], a[row]);
29            Z inv = a[row][col].inv();
30            for (int j = col; j <= n; j++)
31                a[row][j] = a[row][j] * inv;
32            for (int i = 0; i < m; i++)
33                if (i != row && a[i][col].v)
34                {
35                    Z f = a[i][col];
36                    for (int j = col; j <= n; j++)
37                        a[i][j] = a[i][j] - f * a[row][j];
38                }
39            where[col] = row++;
40        }
41        for (int i = row; i < m; i++)
42            if (a[i][n].v)
43                return {false, row, {}, {}};
44        Solution ans{true, row, vector<Z>(n), {}};
```

```

45     for (int j = 0; j < n; j++)
46         if (where[j] != -1)
47             ans.particular[j] = a[where[j]][n];
48     for (int j = 0; j < n; j++)
49         if (where[j] == -1)
50         {
51             vector<Z> v(n);
52             v[j] = 1;
53             for (int k = 0; k < n; k++)
54                 if (where[k] != -1)
55                     v[k] = Z(0) - a[where[k]][j];
56             ans.kernel.push_back(v);
57         }
58     return ans;
59 }
60
61 static Z determinant(Matrix a)
62 {
63     int n = (int)a.size();
64     Z ans = 1;
65     for (int i = 0; i < n; i++)
66     {
67         assert((int)a[i].size() == n);
68         int p = i;
69         while (p < n && !a[p][i].v)
70             ++p;
71         if (p == n)
72             return 0;
73         if (p != i)
74         {
75             swap(a[p], a[i]);
76             ans = Z(0) - ans;
77         }
78         ans = ans * a[i][i];
79         Z inv = a[i][i].inv();
80         for (int j = i + 1; j < n; j++)
81         {
82             Z f = a[j][i] * inv;
83             for (int k = i; k < n; k++)
84                 a[j][k] = a[j][k] - f * a[i][k];
85         }
86     }
87     return ans;
88 }
89
90 static Matrix multiply(const Matrix &a, const Matrix &b)
91 {
92     assert(!a.empty() && !b.empty());
93     int n = a.size(), m = b[0].size(), k = b.size();
94     assert((int)a[0].size() == k);
95     Matrix c(n, vector<Z>(m));
96     for (int i = 0; i < n; i++)
97         for (int t = 0; t < k; t++)
98             for (int j = 0; j < m; j++)
99                 c[i][j] = c[i][j] + a[i][t] * b[t][j];
100     return c;

```

```

101     }
102
103     static Matrix power(Matrix a, unsigned long long e)
104     {
105         int n = a.size();
106         assert(n > 0 && (int)a[0].size() == n);
107         Matrix r(n, vector<Z>(n));
108         for (int i = 0; i < n; i++)
109             r[i][i] = 1;
110         for (; e >>= 1, a = multiply(a, a))
111             if (e & 1)
112                 r = multiply(r, a);
113         return r;
114     }
115
116     // Undirected multigraph, vertices 0..n-1; loops ignored.
117     static Z spanning_trees(int n, const vector<pair<int, int>> &edges)
118     {
119         assert(n > 0);
120         Matrix lap(n, vector<Z>(n));
121         for (auto [u, v] : edges)
122             if (u != v)
123             {
124                 lap[u][u] = lap[u][u] + 1;
125                 lap[v][v] = lap[v][v] + 1;
126                 lap[u][v] = lap[u][v] - 1;
127                 lap[v][u] = lap[v][u] - 1;
128             }
129         lap.pop_back();
130         for (auto &row : lap)
131             row.pop_back();
132         return determinant(lap);
133     }
134 };

```

8.2 杜教筛

构造预筛上限 $B \geq 1$, $\text{mertens}(n)$ 返回 μ 的前缀和, $\text{totient_sum}(n)$ 返回 φ 的前缀和 (int128)。约定 $0 \leq n \leq 10^{11}$, 按整除分块递归并缓存; 大输入需要合理 B , 不能把常数 B 当作性能保证。

```

1 struct DuJiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        LinearSieve s(limit);
14        for (int i = 1; i <= limit; i++)
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll mertens(ll n)
22    {
23        if (n <= limit)
24            return pmu[n];
25        if (mmu.count(n))
26            return mmu[n];
27        ll ans = 1;
28        for (ll l = 2, r; l <= n; l = r + 1)
29        {
30            r = n / (n / l);
31            ans -= (r - l + 1) * mertens(n / l);
32        }
33        return mmu[n] = ans;
34    }
35
36    I totient_sum(ll n)
37    {
38        if (n <= limit)
39            return pphi[n];
40        if (mphi.count(n))
41            return mphi[n];
42        I ans = I(n) * (n + 1) / 2;
43        for (ll l = 2, r; l <= n; l = r + 1)
44        {
45            r = n / (n / l);
46            ans -= I(r - l + 1) * totient_sum(n / l);
47        }
48        return mphi[n] = ans;
49    }
50 };

```

8.3 扩展 BSGS

`solve(a,b,m)` 返回最小 $x \geq 0$ 使 $a^x \equiv b \pmod{m}$, 无解 -1; $1 \leq m \leq 10^{12}$, $0^0=1$ 。时间与内存 $O(\sqrt{m})$, `unordered_map` 复杂度为期望界。

```

1 struct DiscreteLog
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; O(sqrt(m)) time/storage. Smallest x>=0, or -1.
6     static ll solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if (m == 1)
12            return 0;
13        ll offset = 0, k = 1;
14        for (ll g; (g = gcd(a, m)) > 1;)
15        {
16            if (b == k)
17                return offset;
18            if (b % g)
19                return -1;
20            b /= g;
21            m /= g;
22            k = (__int128)k * (a / g) % m;
23            ++offset;
24        }
25        ll target = (__int128)b * NumberTheory::inverse(k, m) % m;
26        ll step = sqrtl(m) + 1;
27        unordered_map<ll, ll> baby;
28        ll cur = 1 % m;
29        for (ll j = 0; j < step; j++)
30        {
31            if (!baby.count(cur))
32                baby[cur] = j;
33            cur = (__int128)cur * a % m;
34        }
35        ll inv = NumberTheory::inverse(NumberTheory::power(a, step, m), m);
36        cur = target;
37        for (ll i = 0; i <= step; i++)
38        {
39            auto it = baby.find(cur);
40            if (it != baby.end())
41                return offset + i * step + it->second;
42            cur = (__int128)cur * inv % m;
43        }
44        return -1;
45    }
46 };

```

第九章 计算几何

9.1 整数精确二维几何

坐标绝对值 $\leq 10^{12}$ ，点数 $\leq 10^6$ 。叉积、点积、线段相交、凸包、点在多边形、点在凸包、旋转卡壳直径。凸包逆时针，不保留共线内点，不重复首点。contains 返回 0 外部、1 边界、2 内部。convex_contains/diameter2 输入必须为本模板 strict hull。PolarLess 要求排除零向量。

```
1 struct IntegerGeometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const
12         {
13             return tie(x, y) < tie(b.x, b.y);
14         }
15
16         bool operator==(Point b) const
17         {
18             return x == b.x && y == b.y;
19         }
20     };
21
22     static I cross(Point a, Point b, Point c)
23     {
24         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
25     }
26
27     static I dot(Point a, Point b, Point c)
28     {
29         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
30     }
31
32     static I dist2(Point a, Point b)
33     {
34         return dot(a, b, b);
35     }
36
37     static int sign(I x)
38     {
39         return (x > 0) - (x < 0);
40     }
41
42     static bool on_segment(Point p, Point a, Point b)
43     {
44         return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
```

```

45     }
46
47     static bool intersect(Point a, Point b, Point c, Point d)
48     {
49         int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
50         int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
51         if (!x && on_segment(c, a, b))
52             return true;
53         if (!y && on_segment(d, a, b))
54             return true;
55         if (!z && on_segment(a, c, d))
56             return true;
57         if (!w && on_segment(b, c, d))
58             return true;
59         return x * y < 0 && z * w < 0;
60     }
61
62     static vector<Point> hull(vector<Point> p)
63     {
64         sort(p.begin(), p.end());
65         p.erase(unique(p.begin(), p.end(), p.end()));
66         int n = (int)p.size();
67         if (n < 3)
68             return p;
69         vector<Point> h;
70         for (Point v : p)
71         {
72             while (h.size() > 1 && cross(h[h.size() - 2], h.back(), v) <= 0)
73                 h.pop_back();
74             h.push_back(v);
75         }
76         int low = (int)h.size();
77         for (int i = n - 2; i >= 0; i--)
78         {
79             while ((int)h.size() > low && cross(h[h.size() - 2], h.back(), p[i]) <= 0)
80                 h.pop_back();
81             h.push_back(p[i]);
82         }
83         h.pop_back();
84         return h; // CCW, no repeated endpoint, no collinear interior points.
85     }
86
87     static I area2(const vector<Point> &p)
88     {
89         I ans = 0;
90         int n = (int)p.size();
91         for (int i = 0; i < n; i++)
92             ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
93         return ans;
94     }
95
96     // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
97     static int contains(const vector<Point> &p, Point q)
98     {
99         int winding = 0, n = (int)p.size();
100         for (int i = 0; i < n; i++)

```

```

101     {
102         Point a = p[i], b = p[(i + 1) % n];
103         if (on_segment(q, a, b))
104             return 1;
105         I c = cross(a, b, q);
106         if (a.y <= q.y && b.y > q.y && c > 0)
107             ++winding;
108         if (a.y > q.y && b.y <= q.y && c < 0)
109             --winding;
110     }
111     return winding ? 2 : 0;
112 }
113
114 // Input must be the strict CCW hull above. O(log n).
115 static int convex_contains(const vector<Point> &p, Point q)
116 {
117     int n = (int)p.size();
118     if (n < 3)
119         return contains(p, q);
120     I a = cross(p[0], p[1], q), b = cross(p[0], p[n - 1], q);
121     if (a < 0 || b > 0)
122         return 0;
123     if (!a)
124         return on_segment(q, p[0], p[1]) ? 1 : 0;
125     if (!b)
126         return on_segment(q, p[0], p[n - 1]) ? 1 : 0;
127     int l = 1, r = n - 1;
128     while (l + 1 < r)
129     {
130         int m = (l + r) / 2;
131         if (cross(p[0], p[m], q) >= 0)
132             l = m;
133         else
134             r = m;
135     }
136     I c = cross(p[l], p[r], q);
137     return c < 0 ? 0 : c == 0 ? 1 : 2;
138 }
139
140 static I diameter2(const vector<Point> &p)
141 {
142     int n = (int)p.size();
143     if (n < 2)
144         return 0;
145     if (n == 2)
146         return dist2(p[0], p[1]);
147     I ans = 0;
148     int j = 1;
149     for (int i = 0; i < n; i++)
150     {
151         int k = (i + 1) % n;
152         while (cross(p[i], p[k], p[(j + 1) % n]) > cross(p[i], p[k], p[j]))
153             j = (j + 1) % n;
154         ans = max({ans, dist2(p[i], p[j]), dist2(p[k], p[j])});
155     }
156     return ans;

```

```

157 }
158
159 struct PolarLess
160 {
161     static int half(Point p)
162     {
163         return p.y < 0 || (p.y == 0 && p.x < 0);
164     }
165
166     bool operator()(Point a, Point b) const
167     {
168         if (half(a) != half(b))
169             return half(a) < half(b);
170         I c = cross({0, 0}, a, b);
171         if (c)
172             return c > 0;
173         return dist2({0, 0}, a) < dist2({0, 0}, b);
174     }
175 };
176 };

```

9.2 浮点点线圆构造

long double, 显式 eps 默认 10^{-12} 。要求有限输入及适当尺度；接近退化时按容差分类，不能提供任意实数输入的精确拓扑保证。Result 区分 none/one/two/infinite/degenerate。零长度线返回 degenerate；投影与点到线段距离将零线段视为点。坐标缩放和误差预算应按具体题面复核。

```

1 struct RealGeometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     RealGeometry(R eps = 1e-12L) : eps(eps)
8     {
9         assert(eps > 0);
10    }
11
12    struct Point
13    {
14        R x = 0, y = 0;
15
16        Point operator+(Point b) const
17        {
18            return {x + b.x, y + b.y};
19        }
20
21        Point operator-(Point b) const
22        {
23            return {x - b.x, y - b.y};
24        }
25
26        Point operator*(R k) const
27        {
28            return {x * k, y * k};

```

```

29     }
30
31     Point operator/(R k) const
32     {
33         return {x / k, y / k};
34     }
35 };
36
37 struct Circle
38 {
39     Point o;
40     R r;
41 };
42 enum class Kind
43 {
44     none,
45     one,
46     two,
47     infinite,
48     degenerate
49 };
50
51 struct Result
52 {
53     Kind kind;
54     vector<Point> p;
55 };
56
57 static R dot(Point a, Point b)
58 {
59     return a.x * b.x + a.y * b.y;
60 }
61
62 static R cross(Point a, Point b)
63 {
64     return a.x * b.y - a.y * b.x;
65 }
66
67 static R norm(Point a)
68 {
69     return hypotl(a.x, a.y);
70 }
71
72 static Point perp(Point a)
73 {
74     return {-a.y, a.x};
75 }
76
77 Point projection(Point p, Point a, Point b) const
78 {
79     Point v = b - a;
80     R d = dot(v, v);
81     if (d == 0)
82         return a;
83     return a + v * (dot(p - a, v) / d);
84 }

```

```

85
86 R segment_distance(Point p, Point a, Point b) const
87 {
88     Point v = b - a;
89     R d = dot(v, v);
90     if (d == 0)
91         return norm(p - a);
92     R t = clamp(dot(p - a, v) / d, R(0), R(1));
93     return norm(p - (a + v * t));
94 }
95
96 Result line_intersection(Point a, Point b, Point c, Point d) const
97 {
98     Point u = b - a, v = d - c;
99     R nu = norm(u), nv = norm(v);
100    if (nu == 0 || nv == 0)
101        return {Kind::degenerate, {}};
102    R det = cross(u, v);
103    if (fabsl(det) <= eps * nu * nv)
104        return {fabsl(cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
105                ? Kind::infinite
106                : Kind::none,
107                {}};
108    return {Kind::one, {a + u * (cross(c - a, v) / det)}};
109 }
110
111 Result line_circle(Point a, Point b, Circle c) const
112 {
113     assert(c.r >= 0);
114     Point v = b - a;
115     R len = norm(v);
116     if (len == 0)
117         return {Kind::degenerate, {}};
118     Point h = projection(c.o, a, b);
119     R d = norm(h - c.o), tol = eps * max({R(1), d, c.r});
120     if (d > c.r + tol)
121         return {Kind::none, {}};
122     if (fabsl(d - c.r) <= tol)
123         return {Kind::one, {h}};
124     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
125     Point w = v * (t / len);
126     return {Kind::two, {h - w, h + w}};
127 }
128
129 Result circles(Circle a, Circle b) const
130 {
131     assert(a.r >= 0 && b.r >= 0);
132     Point v = b.o - a.o;
133     R d = norm(v);
134     R tol = eps * max({R(1), d, a.r, b.r});
135     if (d <= tol)
136     {
137         if (fabsl(a.r - b.r) > tol)
138             return {Kind::none, {}};
139         if (a.r == 0 && b.r == 0)
140             return {Kind::one, {a.o}};

```

```

141         return {Kind::infinite, {}};
142     }
143     if (d > a.r + b.r + tol || d < fabs1(a.r - b.r) - tol)
144         return {Kind::none, {}};
145     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
146     Point h = a.o + v * (x / d);
147     if (fabs1(d - (a.r + b.r)) <= tol || fabs1(d - fabs1(a.r - b.r)) <= tol)
148         return {Kind::one, {h}};
149     R y = sqrt1(max(R(0), (a.r - x) * (a.r + x)));
150     Point w = perp(v) * (y / d);
151     return {Kind::two, {h - w, h + w}};
152 }
153
154 R overlap(Circle a, Circle b) const
155 {
156     R d = norm(a.o - b.o), pi = acos1(-1.L);
157     if (d >= a.r + b.r)
158         return 0;
159     if (d <= fabs1(a.r - b.r))
160         return pi * min(a.r, b.r) * min(a.r, b.r);
161     R x =
162         acos1(clamp((d * d + a.r * a.r - b.r * b.r) / (2 * d * a.r), R(-1), R(1)));
163     R y =
164         acos1(clamp((d * d + b.r * b.r - a.r * a.r) / (2 * d * b.r), R(-1), R(1)));
165     return a.r * a.r * (x - sin1(2 * x) / 2) + b.r * b.r * (y - sin1(2 * y) / 2);
166 }
167 };

```

第十章 优化与可持久化

10.1 Max-plus 矩阵幂

$a[i][j]$ 为转移收益，不存在的边为 neg ； $power(k)$ 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^{119} 。 $O(n^3 \log k)$ ；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
1 struct MaxPlusMatrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if (identity)
11             for (int i = 0; i < n; i++)
12                 a[i][i] = 0;
13     }
14
15     MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
16     {
17         assert(n == b.n);
18         MaxPlusMatrix c(n);
19         for (int i = 0; i < n; i++)
20             for (int k = 0; k < n; k++)
21                 if (a[i][k] != neg)
22                     for (int j = 0; j < n; j++)
23                         if (b.a[k][j] != neg)
24                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
25         return c;
26     }
27
28     MaxPlusMatrix power(unsigned long long e) const
29     {
30         MaxPlusMatrix b = *this, r(n, true);
31         for (; e >>= 1, b = b * b)
32             if (e & 1)
33                 r = r * b;
34         return r;
35     }
36 };
```

10.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标， $add(\{k,b,id\})$ 插入直线， $query(x)$ 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 $int128$ ， k 、 b 、 x 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```
1 struct LiChao
2 {
```

```

3  using ll = long long;
4  using I = __int128_t;
5
6  struct Line
7  {
8      ll k = 0, b = 0;
9      int id = -1;
10
11      I value(ll x) const
12      {
13          return I(k) * x + b;
14      }
15  };
16
17  struct Node
18  {
19      Line line;
20      int left = -1, right = -1;
21  };
22
23  vector<ll> xs;
24  vector<Node> a{Node{}};
25
26  LiChao(vector<ll> x) : xs(x)
27  {
28      sort(xs.begin(), xs.end());
29      xs.erase(unique(xs.begin(), xs.end()), xs.end());
30      assert(!xs.empty());
31  }
32
33  bool better(Line u, Line v, ll x) const
34  {
35      return v.id == -1 ||
36             (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
37  }
38
39  void add(Line line)
40  {
41      assert(line.id >= 0);
42      add(0, 0, (int)xs.size() - 1, line);
43  }
44
45  void add(int p, int l, int r, Line line)
46  {
47      int m = (l + r) / 2;
48      if (better(line, a[p].line, xs[m]))
49          swap(line, a[p].line);
50      if (l == r || line.id == -1)
51          return;
52      bool left = better(line, a[p].line, xs[l]);
53      bool right = better(line, a[p].line, xs[r]);
54      if (!left && !right)
55          return;
56      int next = left ? a[p].left : a[p].right;
57      if (next == -1)
58          {

```

```

59     next = (int)a.size();
60     a.push_back(Node{});
61     if (left)
62         a[p].left = next;
63     else
64         a[p].right = next;
65 }
66 if (left)
67     add(next, l, m, line);
68 else
69     add(next, m + 1, r, line);
70 }
71
72 pair<I, int> query(ll x) const
73 {
74     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
75     assert(idx < (int)xs.size() && xs[idx] == x);
76     int p = 0, l = 0, r = (int)xs.size() - 1;
77     Line best;
78     while (p != -1)
79     {
80         if (better(a[p].line, best, x))
81             best = a[p].line;
82         if (l == r)
83             break;
84         int m = (l + r) / 2;
85         if (idx <= m)
86         {
87             p = a[p].left;
88             r = m;
89         }
90         else
91         {
92             p = a[p].right;
93             l = m + 1;
94         }
95     }
96     assert(best.id != -1);
97     return {best.value(x), best.id};
98 }
99 };

```

10.3 主席树：静态区间第 k 小

构造传入 0-based 数组， $\text{kth}(l, r, k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r - l + 1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```

1 struct PersistentKth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};

```

```

9     vector<int> root{0};
10    vector<long long> vals;
11
12    PersistentKth(const vector<long long> &a) : vals(a)
13    {
14        sort(vals.begin(), vals.end());
15        vals.erase(unique(vals.begin(), vals.end()), vals.end());
16        for (long long x : a)
17        {
18            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19            root.push_back(add(root.back(), 0, (int)vals.size() - 1, p));
20        }
21    }
22
23    int add(int old, int l, int r, int x)
24    {
25        int p = t.size();
26        Node copy = t[old];
27        t.push_back(copy);
28        ++t[p].sum;
29        if (l < r)
30        {
31            int m = (l + r) / 2;
32            if (x <= m)
33            {
34                int v = add(t[old].l, l, m, x);
35                t[p].l = v;
36            }
37            else
38            {
39                int v = add(t[old].r, m + 1, r, x);
40                t[p].r = v;
41            }
42        }
43        return p;
44    }
45
46    long long kth(int l, int r, int k) const
47    {
48        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50        while (L < R)
51        {
52            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53            if (k <= c)
54            {
55                u = t[u].l;
56                v = t[v].l;
57                R = m;
58            }
59            else

```

```
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };
```

第十一章 图论进阶

11.1 矩形二分图最小权匹配

`solve(cost)` 输入 $n \times m$ 完整权值矩阵, $0 \leq n \leq m$; 每行匹配不同列, 返回最小费用及列号 (0-based)。允许负费用, 使用 `int128` 势能; $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代, 当前接口仅表示完整二分图。

```
1 struct Hungarian
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if (!n)
10             return {0, {}};
11         int m = cost[0].size();
12         assert(n <= m);
13         vector<I> u(n + 1), v(m + 1);
14         vector<int> p(m + 1), way(m + 1);
15         const I inf = I(1) << 120;
16         for (int i = 1; i <= n; i++)
17         {
18             assert((int)cost[i - 1].size() == m);
19             p[0] = i;
20             int j0 = 0;
21             vector<I> dis(m + 1, inf);
22             vector<bool> used(m + 1);
23             do
24             {
25                 used[j0] = true;
26                 int i0 = p[j0], j1 = 0;
27                 I delta = inf;
28                 for (int j = 1; j <= m; j++)
29                     if (!used[j])
30                     {
31                         I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
32                         if (cur < dis[j])
33                         {
34                             dis[j] = cur;
35                             way[j] = j0;
36                         }
37                         if (dis[j] < delta)
38                         {
39                             delta = dis[j];
40                             j1 = j;
41                         }
42                     }
43                 for (int j = 0; j <= m; j++)
44                     if (used[j])
45                     {
```

```

46         u[p[j]] += delta;
47         v[j] -= delta;
48     }
49     else
50         dis[j] -= delta;
51     j0 = j1;
52 } while (p[j0]);
53 do
54 {
55     int j1 = way[j0];
56     p[j0] = p[j1];
57     j0 = j1;
58 } while (j0);
59 }
60 vector<int> ans(n);
61 for (int j = 1; j <= m; j++)
62     if (p[j])
63         ans[p[j] - 1] = j - 1;
64 return {-v[0], ans};
65 }
66 };

```

11.2 有向最小树形图

`solve(n, root, edges)`, 点号 $0..n-1$, $n \geq 1$; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边、自环 (自环不选); $O(VE)$ 。当前仅返回费用, 不恢复边方案。所有中间值须远小于 2^{120} 。

```

1 struct Arborescence
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while (true)
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for (auto a : e)
21                 if (a.u != a.v && a.w < in[a.v])
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for (I x : in)
28                 {

```

```

29         if (x == inf)
30             return nullopt;
31         ans += x;
32     }
33     int cnt = 0;
34     for (int i = 0; i < n; i++)
35     {
36         int u = i;
37         while (vis[u] != i && id[u] == -1 && u != root)
38         {
39             vis[u] = i;
40             u = pre[u];
41         }
42         if (u != root && id[u] == -1)
43         {
44             id[u] = cnt;
45             for (int v = pre[u]; v != u; v = pre[v])
46                 id[v] = cnt;
47             ++cnt;
48         }
49     }
50     if (!cnt)
51         return ans;
52     for (int i = 0; i < n; i++)
53         if (id[i] == -1)
54             id[i] = cnt++;
55     for (auto &a : e)
56     {
57         int v = a.v;
58         a.u = id[a.u];
59         a.v = id[a.v];
60         if (a.u != a.v)
61             a.w -= in[v];
62     }
63     root = id[root];
64     n = cnt;
65 }
66 }
67 };

```

11.3 无向全局最小割

`solve(w)`, $n \geq 2$; w 是对称非负权矩阵, 重边需先累加, 主对角置零。返回割权与一侧顶点编号 (0-based), 允许不连通图。 $O(n^3)$, 总权值小于 2^{120} 。

```

1 struct StoerWagner
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for (int i = 0; i < n; i++)
14            group[i] = {i};
15        I best = I(1) << 120;
16        vector<int> side;
17        while (active.size() > 1)
18        {
19            vector<I> dis(n);
20            vector<bool> used(n);
21            int prev = -1;
22            for (int step = 0; step < (int)active.size(); step++)
23            {
24                int sel = -1;
25                for (int v : active)
26                    if (!used[v] && (sel == -1 || dis[v] > dis[sel]))
27                        sel = v;
28                if (step + 1 == (int)active.size())
29                {
30                    if (dis[sel] < best)
31                    {
32                        best = dis[sel];
33                        side = group[sel];
34                    }
35                    for (int v : active)
36                        if (v != prev && v != sel)
37                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
38                    group[prev].insert(
39                        group[prev].end(), group[sel].begin(), group[sel].end());
40                    active.erase(find(active.begin(), active.end(), sel));
41                    break;
42                }

```

```
43         used[sel] = true;
44         prev = sel;
45         for (int v : active)
46             if (!used[v])
47                 dis[v] += w[sel][v];
48     }
49 }
50 return {best, side};
51 }
52 };
```

第十二章 精确几何进阶

12.1 整数最近点对与 Minkowski 和

`closest_pair(points)` 返回最短距离平方，少于两个点为 `nullopt`，重复点返回 0， $O(n \log n)$ ，全程精确整数。
`minkowski` 输入 strict CCW 凸包，返回不重复首点的凸包；非退化 $O(n+m)$ ，退化时 $O((n+m)\log(n+m))$ 。坐标及坐标和须满足整数几何范围。

```
1 struct GeometryExtra
2 {
3     using G = IntegerGeometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance, 0(n log n).
8     static optional<I> closest_pair(vector<P> p)
9     {
10         int n = p.size();
11         if (n < 2)
12             return nullopt;
13         sort(p.begin(), p.end());
14         for (int i = 1; i < n; i++)
15             if (p[i] == p[i - 1])
16                 return I(0);
17         vector<P> tmp(n);
18         auto by_y = [](P a, P b)
19         {
20             return tie(a.y, a.x) < tie(b.y, b.x);
21         };
22         function<I(int, int)> solve = [&](int l, int r) -> I
23         {
24             if (r - l <= 3)
25             {
26                 I best = I(1) << 120;
27                 for (int i = l; i < r; i++)
28                     for (int j = i + 1; j < r; j++)
29                         best = min(best, G::dist2(p[i], p[j]));
30                 sort(p.begin() + l, p.begin() + r, by_y);
31                 return best;
32             }
33             int m = (l + r) / 2;
34             long long x = p[m].x;
35             I best = min(solve(l, m), solve(m, r));
36             merge(p.begin() + l,
37                 p.begin() + m,
38                 p.begin() + m,
39                 p.begin() + r,
40                 tmp.begin() + l,
41                 by_y);
42             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
43             vector<P> strip;
44             for (int i = l; i < r; i++)
```

```

45         if ((I(p[i].x) - x) * (I(p[i].x) - x) < best)
46         {
47             for (int j = (int)strip.size() - 1; j >= 0; j--)
48             {
49                 I dy = I(p[i].y) - strip[j].y;
50                 if (dy * dy >= best)
51                     break;
52                 best = min(best, G::dist2(p[i], strip[j]));
53             }
54             strip.push_back(p[i]);
55         }
56         return best;
57     };
58     return solve(0, n);
59 }
60
61 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
62 static vector<P> minkowski(vector<P> a, vector<P> b)
63 {
64     if (a.empty() || b.empty())
65         return {};
66     auto plus = [](P u, P v)
67     {
68         return P{u.x + v.x, u.y + v.y};
69     };
70     if (a.size() < 3 || b.size() < 3)
71     {
72         vector<P> p;
73         for (P u : a)
74             for (P v : b)
75                 p.push_back(plus(u, v));
76         return G::hull(p);
77     }
78     auto start = [](vector<P> &p)
79     {
80         auto it = min_element(p.begin(),
81                               p.end(),
82                               [](P u, P v)
83                               {
84                                   return tie(u.y, u.x) < tie(v.y, v.x);
85                               });
86         rotate(p.begin(), it, p.end());
87     };
88     start(a);
89     start(b);
90     int n = a.size(), m = b.size(), i = 0, j = 0;
91     vector<P> result;
92     while (i < n || j < m)
93     {
94         result.push_back(plus(a[i % n], b[j % m]));
95         if (i == n)
96         {
97             ++j;
98             continue;
99         }
100         if (j == m)

```

```
101         {
102             ++i;
103             continue;
104         }
105         P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
106         P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
107         I c = G::cross({0, 0}, u, v);
108         if (c >= 0)
109             ++i;
110         if (c <= 0)
111             ++j;
112     }
113     return result;
114 }
115 };
```

12.2 整数三维共线、共面与方向

坐标绝对值 $\leq 10^9$; $\text{orient}(a,b,c,d)$ 为有向六倍四面体体积, 0 表示共面; collinear 判三点共线, on_segment 支持退化线段。int128 精确判定, 不包含三维凸包构造。

```

1 struct IntegerGeometry3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I dot(Vector a, Vector b)
27     {
28         return a.x * b.x + a.y * b.y + a.z * b.z;
29     }
30
31     static I orient(Point a, Point b, Point c, Point d)
32     {
33         return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
34     }
35
36     static bool collinear(Point a, Point b, Point c)
37     {
38         auto v = cross(diff(b, a), diff(c, a));
39         return !v.x && !v.y && !v.z;
40     }
41
42     static bool on_segment(Point p, Point a, Point b)
43     {
44         return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
45     }
46 };

```

第十三章 动态树

13.1 Link-cut tree: 动态森林与路径异或

构造 LinkCutTree(n), 点号 1..n, 初始权值为零。set(x,value) 修改点权; link(x,y) 仅在不同连通块之间连边并返回是否成功; cut(x,y) 只删除真实存在的边, 否则返回 false。connected 查询连通性; query(x,y) 返回路径点权异或, 要求两点连通, 支持 x=y。make_root/find_root 维护与查询当前表示树根。每次操作均摊 $O(\log n)$, $O(n)$ 空间, splay 下传使用显式栈, 支持长链。权值为 unsigned long long。仅维护路径异或, 不包含虚子树聚合或动态直径。

```
1 struct LinkCutTree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    vector<Node> a;
13    vector<int> st;
14
15    LinkCutTree(int n) : a(n + 1), st(n + 1)
16    {
17    }
18
19    bool is_root(int x) const
20    {
21        int f = a[x].fa;
22        return a[f].ch[0] != x && a[f].ch[1] != x;
23    }
24
25    void pull(int x)
26    {
27        a[x].sum = a[a[x].ch[0]].sum ^ a[x].val ^ a[a[x].ch[1]].sum;
28    }
29
30    void reverse(int x)
31    {
32        if (!x)
33            return;
34        swap(a[x].ch[0], a[x].ch[1]);
35        a[x].rev = !a[x].rev;
36    }
37
38    void push(int x)
39    {
40        if (!a[x].rev)
41            return;
42        reverse(a[x].ch[0]);
```

```

43     reverse(a[x].ch[1]);
44     a[x].rev = false;
45 }
46
47 void rotate(int x)
48 {
49     int y = a[x].fa, z = a[y].fa;
50     int k = a[y].ch[1] == x;
51     int w = a[x].ch[k ^ 1];
52     if (!is_root(y))
53         a[z].ch[a[z].ch[1] == y] = x;
54     a[x].fa = z;
55     a[y].ch[k] = w;
56     if (w)
57         a[w].fa = y;
58     a[x].ch[k ^ 1] = y;
59     a[y].fa = x;
60     pull(y);
61     pull(x);
62 }
63
64 void splay(int x)
65 {
66     int top = 0, y = x;
67     st[top++] = y;
68     while (!is_root(y))
69     {
70         y = a[y].fa;
71         st[top++] = y;
72     }
73     while (top)
74         push(st[--top]);
75     while (!is_root(x))
76     {
77         int y = a[x].fa, z = a[y].fa;
78         if (!is_root(y))
79         {
80             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
81             rotate(same ? y : x);
82         }
83         rotate(x);
84     }
85 }
86
87 void access(int x)
88 {
89     int last = 0;
90     for (int y = x; y; y = a[y].fa)
91     {
92         splay(y);
93         // The old right child's fa remains y: it becomes a path-parent.
94         a[y].ch[1] = last;
95         if (last)
96             a[last].fa = y;
97         pull(y);
98         last = y;

```

```

99     }
100     splay(x);
101 }
102
103 void make_root(int x)
104 {
105     access(x);
106     reverse(x);
107 }
108
109 int find_root(int x)
110 {
111     access(x);
112     push(x);
113     while (a[x].ch[0])
114     {
115         x = a[x].ch[0];
116         push(x);
117     }
118     splay(x);
119     return x;
120 }
121
122 bool connected(int x, int y)
123 {
124     return x == y || find_root(x) == find_root(y);
125 }
126
127 bool link(int x, int y)
128 {
129     make_root(x);
130     if (find_root(y) == x)
131         return false;
132     a[x].fa = y;
133     return true;
134 }
135
136 bool cut(int x, int y)
137 {
138     make_root(x);
139     access(y);
140     if (a[y].ch[0] != x || a[x].ch[1])
141         return false;
142     a[y].ch[0] = 0;
143     a[x].fa = 0;
144     pull(y);
145     return true;
146 }
147
148 void set(int x, U value)
149 {
150     access(x);
151     a[x].val = value;
152     pull(x);

```

```
153     }
154
155     // x and y must be connected. This is a path aggregate, not a subtree sum.
156     U query(int x, int y)
157     {
158         make_root(x);
159         access(y);
160         return a[y].sum;
161     }
162 };
```

第十四章 随机平衡树

14.1 FHQ Treap: 可重集合与顺序统计

insert(x) 插入一个副本, erase(x) 删除一个副本并返回是否存在; rank(x) 为严格小于 x 的元素数加一, x 不必存在; kth(k) 要求 $1 \leq k \leq \text{size}()$ 。prev/next 为严格前驱/后继, 不存在时返回 nullopt。支持重复值、LLONG_MIN 和 LLONG_MAX, 分裂不使用 x-1。随机优先级下单次期望 $O(\log n)$, 并非确定性最坏界。节点池不回收已删除节点, 空间按累计插入次数计算; 可在构造时指定随机种子。

```
1 struct OrderedTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    mt19937_64 rng;
14    int root = 0;
15
16    OrderedTreap(unsigned long long seed = 712367821) : rng(seed)
17    {
18    }
19
20    void pull(int p)
21    {
22        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
23    }
24
25    int node(ll x)
26    {
27        int p = a.size();
28        a.push_back({0, 0, 1, x, rng()});
29        return p;
30    }
31
32    // left: val < x, or val <= x when equal is true. No x-1 overflow.
33    void split(int p, ll x, int &l, int &r, bool equal = false)
34    {
35        if (!p)
36        {
37            l = 0;
38            r = 0;
39            return;
40        }
41        if (a[p].val < x || (equal && a[p].val == x))
42        {
43            l = p;
```

```

44         split(a[p].r, x, a[p].r, r, equal);
45     }
46     else
47     {
48         r = p;
49         split(a[p].l, x, l, a[p].l, equal);
50     }
51     pull(p);
52 }
53
54 int merge(int l, int r)
55 {
56     if (!l || !r)
57         return l ? l : r;
58     if (a[l].pri > a[r].pri)
59     {
60         a[l].r = merge(a[l].r, r);
61         pull(l);
62         return l;
63     }
64     a[r].l = merge(l, a[r].l);
65     pull(r);
66     return r;
67 }
68
69 void insert(ll x)
70 {
71     int p = node(x), l, r;
72     split(root, x, l, r);
73     root = merge(merge(l, p), r);
74 }
75
76 bool erase(ll x)
77 {
78     int l, mid, r;
79     split(root, x, l, mid);
80     split(mid, x, mid, r, true);
81     bool found = mid != 0;
82     if (mid)
83         mid = merge(a[mid].l, a[mid].r);
84     root = merge(l, merge(mid, r));
85     return found;
86 }
87
88 int size() const
89 {
90     return a[root].siz;
91 }
92
93 int less(ll x, bool equal = false) const
94 {
95     int p = root, ans = 0;
96     while (p)
97     {
98         if (a[p].val < x || (equal && a[p].val == x))
99             {

```

```

100         ans += a[a[p].l].siz + 1;
101         p = a[p].r;
102     }
103     else
104         p = a[p].l;
105     }
106     return ans;
107 }
108
109 int rank(ll x) const
110 {
111     return less(x) + 1;
112 }
113
114 ll kth(int k) const
115 {
116     assert(1 <= k && k <= size());
117     int p = root;
118     while (true)
119     {
120         int left = a[a[p].l].siz;
121         if (k == left + 1)
122             return a[p].val;
123         if (k <= left)
124             p = a[p].l;
125         else
126         {
127             k -= left + 1;
128             p = a[p].r;
129         }
130     }
131 }
132
133 optional<ll> prev(ll x) const
134 {
135     int k = less(x);
136     if (!k)
137         return nullopt;
138     return kth(k);
139 }
140
141 optional<ll> next(ll x) const
142 {
143     int k = less(x, true);
144     if (k == size())
145         return nullopt;
146     return kth(k + 1);
147 }
148 };

```

14.2 隐式 Treap：序列翻转、加值与区间和

`insert(k,value)` 在前 k 个元素后插入， $0 \leq k \leq \text{size}()$ ；`erase(l,r)`、`reverse(l,r)`、`add(l,r,value)`、`query(l,r)` 均使用 1-based 非空闭区间。`values()` 返回当前完整序列， $O(n)$ ，使用显式中序栈。其余操作在随机优先级下期望 $O(\log$

n); 翻转与加法懒标记同时维护。所有值、和、标记及 $\text{value} \times \text{长度}$ 中间结果必须落在 `long long` 范围内。删除不回收节点，内存按累计插入数计算。

```

1 struct SequenceTreap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0, sum = 0, tag = 0;
9         unsigned long long pri = 0;
10        bool rev = false;
11    };
12
13    vector<Node> a{Node{}};
14    mt19937_64 rng;
15    int root = 0;
16
17    SequenceTreap(unsigned long long seed = 712367821) : rng(seed)
18    {
19    }
20
21    int size() const
22    {
23        return a[root].siz;
24    }
25
26    void pull(int p)
27    {
28        a[p].siz = a[a[p].l].siz + a[a[p].r].siz + 1;
29        a[p].sum = a[a[p].l].sum + a[p].val + a[a[p].r].sum;
30    }
31
32    void apply_add(int p, ll x)
33    {
34        if (!p)
35            return;
36        a[p].val += x;
37        a[p].sum += x * a[p].siz;
38        a[p].tag += x;
39    }
40
41    void apply_reverse(int p)
42    {
43        if (!p)
44            return;
45        swap(a[p].l, a[p].r);
46        a[p].rev = !a[p].rev;
47    }
48
49    void push(int p)
50    {
51        if (a[p].rev)
52        {
53            apply_reverse(a[p].l);

```

```

54         apply_reverse(a[p].r);
55         a[p].rev = false;
56     }
57     if (a[p].tag)
58     {
59         apply_add(a[p].l, a[p].tag);
60         apply_add(a[p].r, a[p].tag);
61         a[p].tag = 0;
62     }
63 }
64
65 // Split into the first k elements and the rest.
66 void split(int p, int k, int &l, int &r)
67 {
68     if (!p)
69     {
70         l = 0;
71         r = 0;
72         return;
73     }
74     push(p);
75     int left = a[a[p].l].siz;
76     if (k > left)
77     {
78         l = p;
79         split(a[p].r, k - left - 1, a[p].r, r);
80     }
81     else
82     {
83         r = p;
84         split(a[p].l, k, l, a[p].l);
85     }
86     pull(p);
87 }
88
89 int merge(int l, int r)
90 {
91     if (!l || !r)
92         return l ? l : r;
93     if (a[l].pri > a[r].pri)
94     {
95         push(l);
96         a[l].r = merge(a[l].r, r);
97         pull(l);
98         return l;
99     }
100    push(r);
101    a[r].l = merge(l, a[r].l);
102    pull(r);
103    return r;
104 }
105
106 // Insert after the first k elements, 0 <= k <= size().
107 void insert(int k, ll value)
108 {
109     assert(0 <= k && k <= size());

```

```

110     int p = a.size(), l, r;
111     a.push_back({0, 0, 1, value, value, 0, rng(), false});
112     split(root, k, l, r);
113     root = merge(merge(l, p), r);
114 }
115
116 // All range operations use 1-based inclusive intervals.
117 void erase(int l, int r)
118 {
119     assert(1 <= l && l <= r && r <= size());
120     int x, y, z;
121     split(root, r, y, z);
122     split(y, l - 1, x, y);
123     root = merge(x, z);
124 }
125
126 void reverse(int l, int r)
127 {
128     assert(1 <= l && l <= r && r <= size());
129     int x, y, z;
130     split(root, r, y, z);
131     split(y, l - 1, x, y);
132     apply_reverse(y);
133     root = merge(merge(x, y), z);
134 }
135
136 void add(int l, int r, ll value)
137 {
138     assert(1 <= l && l <= r && r <= size());
139     int x, y, z;
140     split(root, r, y, z);
141     split(y, l - 1, x, y);
142     apply_add(y, value);
143     root = merge(merge(x, y), z);
144 }
145
146 ll query(int l, int r)
147 {
148     assert(1 <= l && l <= r && r <= size());
149     int x, y, z;
150     split(root, r, y, z);
151     split(y, l - 1, x, y);
152     ll ans = a[y].sum;
153     root = merge(merge(x, y), z);
154     return ans;
155 }
156
157 vector<ll> values()
158 {
159     vector<ll> ans;
160     vector<int> st;
161     int p = root;
162     while (p || !st.empty())
163     {
164         while (p)
165         {

```

```
166         push(p);
167         st.push_back(p);
168         p = a[p].l;
169     }
170     p = st.back();
171     st.pop_back();
172     ans.push_back(a[p].val);
173     p = a[p].r;
174 }
175 return ans;
176 }
177 };
```

第十五章 伸展树

15.1 Splay: 可重集合与顺序统计

insert/erase/rank/kth/prev/next 与有序 Treap 同义, 严格前驱后继不存在返回 nullopt。重复键计数存储; 失败查询也伸展最后访问节点, 单次均摊 $O(\log n)$ 。不回收节点, 空间按累计新建节点数计算。支持 long long 全范围; 不包含序列 Splay、虚子树信息或矩阵动态 DP。

```
1 struct OrderedSplay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}, fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     vector<Node> a{Node{}};
12     int root = 0;
13
14     int size() const
15     {
16         return a[root].siz;
17     }
18
19     void pull(int x)
20     {
21         a[x].siz = a[a[x].ch[0]].siz + a[x].cnt + a[a[x].ch[1]].siz;
22     }
23
24     int node(ll value, int parent)
25     {
26         int x = a.size();
27         a.push_back({{0, 0}, parent, 1, 1, value});
28         return x;
29     }
30
31     void rotate(int x)
32     {
33         int y = a[x].fa, z = a[y].fa;
34         int k = a[y].ch[1] == x;
35         int w = a[x].ch[k ^ 1];
36         if (z)
37             a[z].ch[a[z].ch[1] == y] = x;
38         else
39             root = x;
40         a[x].fa = z;
41         a[y].ch[k] = w;
42         if (w)
43             a[w].fa = y;
44         a[x].ch[k ^ 1] = y;
```

```

45     a[y].fa = x;
46     pull(y);
47     pull(x);
48 }
49
50 void splay(int x)
51 {
52     while (a[x].fa)
53     {
54         int y = a[x].fa, z = a[y].fa;
55         if (z)
56         {
57             bool same = (a[y].ch[1] == x) == (a[z].ch[1] == y);
58             rotate(same ? y : x);
59         }
60         rotate(x);
61     }
62 }
63
64 // Splay the last visited node even when the value does not exist.
65 int find(ll value)
66 {
67     int x = root, last = 0;
68     while (x && a[x].val != value)
69     {
70         last = x;
71         x = a[x].ch[value > a[x].val];
72     }
73     if (x)
74         splay(x);
75     else if (last)
76         splay(last);
77     return x;
78 }
79
80 void insert(ll value)
81 {
82     int x = root, parent = 0;
83     while (x && a[x].val != value)
84     {
85         parent = x;
86         x = a[x].ch[value > a[x].val];
87     }
88     if (x)
89     {
90         ++a[x].cnt;
91         pull(x);
92     }
93     else
94     {
95         x = node(value, parent);
96         if (parent)
97             a[parent].ch[value > a[parent].val] = x;
98         else
99             root = x;
100    }

```

```

101     splay(x);
102 }
103
104 bool erase(ll value)
105 {
106     int x = find(value);
107     if (!x)
108         return false;
109     if (a[x].cnt > 1)
110     {
111         --a[x].cnt;
112         pull(x);
113         return true;
114     }
115     int l = a[x].ch[0], r = a[x].ch[1];
116     if (l)
117         a[l].fa = 0;
118     if (r)
119         a[r].fa = 0;
120     if (!l)
121     {
122         root = r;
123         return true;
124     }
125     root = l;
126     x = l;
127     while (a[x].ch[1])
128         x = a[x].ch[1];
129     splay(x);
130     a[x].ch[1] = r;
131     if (r)
132         a[r].fa = x;
133     pull(x);
134     return true;
135 }
136
137 int rank(ll value)
138 {
139     int x = root, last = 0, ans = 1;
140     while (x)
141     {
142         last = x;
143         if (value <= a[x].val)
144             x = a[x].ch[0];
145         else
146         {
147             ans += a[a[x].ch[0]].siz + a[x].cnt;
148             x = a[x].ch[1];
149         }
150     }
151     if (last)
152         splay(last);
153     return ans;
154 }
155
156 ll kth(int k)

```

```

157 {
158     assert(1 <= k && k <= size());
159     int x = root;
160     while (true)
161     {
162         int left = a[a[x].ch[0]].siz;
163         if (k <= left)
164             x = a[x].ch[0];
165         else if (k <= left + a[x].cnt)
166         {
167             splay(x);
168             return a[x].val;
169         }
170         else
171         {
172             k -= left + a[x].cnt;
173             x = a[x].ch[1];
174         }
175     }
176 }
177
178 optional<ll> prev(ll value)
179 {
180     int x = root, last = 0, answer = 0;
181     while (x)
182     {
183         last = x;
184         if (a[x].val < value)
185         {
186             answer = x;
187             x = a[x].ch[1];
188         }
189         else
190             x = a[x].ch[0];
191     }
192     if (last)
193         splay(last);
194     if (!answer)
195         return nullopt;
196     splay(answer);
197     return a[answer].val;
198 }
199
200 optional<ll> next(ll value)
201 {
202     int x = root, last = 0, answer = 0;
203     while (x)
204     {
205         last = x;
206         if (a[x].val > value)
207         {
208             answer = x;
209             x = a[x].ch[0];
210         }
211         else

```

```
212         x = a[x].ch[1];
213     }
214     if (last)
215         splay(last);
216     if (!answer)
217         return nullopt;
218     splay(answer);
219     return a[answer].val;
220 }
221 };
```

第十六章 状态序列维护

16.1 状态序列 Treap: 分组区间 GCD

build(v) 用 (unsigned long long 值, 0/1 状态) 序列替换当前内容; insert(k, value, tag) 在前 k 项后插入。erase(l, r), query(l, r, tag) 使用 1-based 非空闭区间; set(k, value) 修改值, toggle(k) 翻转状态。query 返回选中状态的 GCD, 无对应元素为 nullopt, 实际全零则返回 0。回收删除节点, 空间按峰值长度计算。建树 $O(n)$ 次 GCD, 随机优先级下单次期望 $O(\log n)$ 次 GCD; 删除另需 $O(\text{删除长度})$ 回收。一次 GCD 为 $O(\log U)$ 。覆盖 kuangbin 3.7 的序列操作, 尚无该原题 OJ 记录。

```
1 struct GcdSequenceTreap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    vector<Node> a{Node{}};
13    vector<int> recycled;
14    mt19937_64 rng;
15    int root = 0;
16
17    GcdSequenceTreap(U seed = 712367821) : rng(seed)
18    {
19    }
20
21    int size() const
22    {
23        return a[root].siz;
24    }
25
26    int node(U value, int tag)
27    {
28        assert(tag == 0 || tag == 1);
29        int p;
30        if (recycled.empty())
31        {
32            p = a.size();
33            a.push_back(Node{});
34        }
35        else
36        {
37            p = recycled.back();
38            recycled.pop_back();
39            a[p] = Node{};
40        }
41        a[p].siz = 1;
42        a[p].val = value;
```

```

43     a[p].tag = tag;
44     a[p].cnt[tag] = 1;
45     a[p].g[tag] = value;
46     a[p].pri = rng();
47     return p;
48 }
49
50 void pull(int p)
51 {
52     int l = a[p].l, r = a[p].r;
53     a[p].siz = a[l].siz + a[r].siz + 1;
54     for (int t = 0; t < 2; t++)
55     {
56         a[p].cnt[t] = a[l].cnt[t] + a[r].cnt[t] + (a[p].tag == t);
57         a[p].g[t] = gcd(a[l].g[t], a[r].g[t]);
58         if (a[p].tag == t)
59             a[p].g[t] = gcd(a[p].g[t], a[p].val);
60     }
61 }
62
63 void split(int p, int k, int &l, int &r)
64 {
65     if (!p)
66     {
67         l = 0;
68         r = 0;
69         return;
70     }
71     int left = a[a[p].l].siz;
72     if (k > left)
73     {
74         l = p;
75         split(a[p].r, k - left - 1, a[p].r, r);
76     }
77     else
78     {
79         r = p;
80         split(a[p].l, k, l, a[p].l);
81     }
82     pull(p);
83 }
84
85 int merge(int l, int r)
86 {
87     if (!l || !r)
88         return l ? l : r;
89     if (a[l].pri > a[r].pri)
90     {
91         a[l].r = merge(a[l].r, r);
92         pull(l);
93         return l;
94     }
95     a[r].l = merge(l, a[r].l);
96     pull(r);
97     return r;
98 }

```

```

99
100 // O(n) GCD calls; replaces the current sequence.
101 void build(const vector<pair<U, int>> &v)
102 {
103     a.assign(1, Node{});
104     recycled.clear();
105     root = 0;
106     vector<int> st;
107     for (auto [value, tag] : v)
108     {
109         int p = node(value, tag), last = 0;
110         while (!st.empty() && a[st.back()].pri < a[p].pri)
111         {
112             last = st.back();
113             st.pop_back();
114         }
115         a[p].l = last;
116         if (!st.empty())
117             a[st.back()].r = p;
118         st.push_back(p);
119     }
120     if (st.empty())
121         return;
122     root = st.front();
123     vector<int> order{root};
124     for (int i = 0; i < (int)order.size(); i++)
125     {
126         int p = order[i];
127         if (a[p].l)
128             order.push_back(a[p].l);
129         if (a[p].r)
130             order.push_back(a[p].r);
131     }
132     for (int i = (int)order.size() - 1; i >= 0; i--)
133         pull(order[i]);
134 }
135
136 // Insert after the first k entries. Range operations below are 1-based.
137 void insert(int k, U value, int tag)
138 {
139     assert(0 <= k && k <= size());
140     int p = node(value, tag), l, r;
141     split(root, k, l, r);
142     root = merge(merge(l, p), r);
143 }
144
145 void erase(int l, int r)
146 {
147     assert(1 <= l && l <= r && r <= size());
148     int x, y, z;
149     split(root, r, y, z);
150     split(y, l - 1, x, y);
151     root = merge(x, z);
152     vector<int> st{y};
153     while (!st.empty())
154     {

```

```

155         int p = st.back();
156         st.pop_back();
157         if (a[p].l)
158             st.push_back(a[p].l);
159         if (a[p].r)
160             st.push_back(a[p].r);
161         recycled.push_back(p);
162     }
163 }
164
165 void set(int k, U value)
166 {
167     assert(1 <= k && k <= size());
168     int x, y, z;
169     split(root, k, y, z);
170     split(y, k - 1, x, y);
171     a[y].val = value;
172     pull(y);
173     root = merge(merge(x, y), z);
174 }
175
176 void toggle(int k)
177 {
178     assert(1 <= k && k <= size());
179     int x, y, z;
180     split(root, k, y, z);
181     split(y, k - 1, x, y);
182     a[y].tag ^= 1;
183     pull(y);
184     root = merge(merge(x, y), z);
185 }
186
187 optional<U> query(int l, int r, int tag)
188 {
189     assert(1 <= l && l <= r && r <= size());
190     assert(tag == 0 || tag == 1);
191     int x, y, z;
192     split(root, r, y, z);
193     split(y, l - 1, x, y);
194     optional<U> answer;
195     if (a[y].cnt[tag])
196         answer = a[y].g[tag];
197     root = merge(merge(x, y), z);
198     return answer;
199 }
200 };

```

第二部分

传统接口风格

第十七章 数据结构

17.1 并查集

算法约束见动态版本第 2 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template <int N> struct Disjoint_Set
2 {
3     int n, fa[N + 1], siz[N + 1];
4
5     void Init(int size)
6     {
7         assert(size <= N);
8         n = size;
9         iota(fa, fa + n + 1, 0);
10        fill(siz, siz + n + 1, 1);
11    }
12
13    int Find(int x)
14    {
15        int root = x;
16        while ( root != fa[root] )
17            root = fa[root];
18        while ( x != root )
19        {
20            int next = fa[x];
21            fa[x] = root;
22            x = next;
23        }
24        return root;
25    }
26
27    bool Merge(int x, int y)
28    {
29        x = Find(x);
30        y = Find(y);
31        if ( x == y )
32            return false;
33        if ( siz[x] < siz[y] )
34            swap(x, y);
35        fa[y] = x;
36        siz[x] += siz[y];
37        return true;
38    }
39 };
```

17.2 可撤销并查集

算法约束见动态版本第 3 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int N> struct Rollback_DSU
2 {
3     int fa[N + 1], siz[N + 1];
4     vector<pair<int, int>> history;
5
6     void Init(int n)
7     {
8         assert(n <= N);
9         history.clear();
10        iota(fa, fa + n + 1, 0);
11        fill(siz, siz + n + 1, 1);
12    }
13
14    int Find(int x) const
15    {
16        while ( fa[x] != x )
17            x = fa[x];
18        return x;
19    }
20
21    int Snapshot() const
22    {
23        return (int)history.size();
24    }
25
26    bool Merge(int x, int y)
27    {
28        x = Find(x);
29        y = Find(y);
30        if ( x == y )
31            return false;
32        if ( siz[x] < siz[y] )
33            swap(x, y);
34        history.push_back({y, siz[x]});
35        fa[y] = x;
36        siz[x] += siz[y];
37        return true;
38    }
39
40    void Rollback(int time)
41    {
42        assert(0 <= time && time <= Snapshot());
43        while ( Snapshot() > time )
44        {
45            auto [y, size] = history.back();
46            history.pop_back();
47            siz[fa[y]] = size;
48            fa[y] = y;
49        }
50    }
51 };

```

17.3 树状数组与第 k 小

算法约束见动态版本第 4 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```

1 template <int N, class T = long long> struct Binary_Indexed_Tree
2 {
3     int n;
4     T tree[N + 1];
5
6     void Init(int size)
7     {
8         assert(size <= N);
9         n = size;
10        fill(tree, tree + n + 1, T(0));
11    }
12
13    void Insert(int x, T value)
14    {
15        assert(x >= 1);
16        for ( ; x <= n; x += x & -x )
17            tree[x] += value;
18    }
19
20    T Sum(int x) const
21    {
22        T answer = 0;
23        for ( ; x; x -= x & -x )
24            answer += tree[x];
25        return answer;
26    }
27
28    T Query(int L, int R) const
29    {
30        return Sum(R) - Sum(L - 1);
31    }
32
33    int Kth(T k) const
34    {
35        assert(k > 0);
36        int now = 0, step = 1;
37        while ( step <= n / 2 )
38            step <<= 1;
39        for ( ; step; step >>= 1 )
40            if ( now + step <= n && tree[now + step] < k )
41            {
42                now += step;
43                k -= tree[now];
44            }
45        return now + 1;
46    }
47 };

```

17.4 区间加与区间和

算法约束见动态版本第 4 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```

1  template <int N> struct Segment_Tree
2  {
3      int n;
4      long long tree[4 * N + 5], tag[4 * N + 5];
5
6      void Init(int size)
7      {
8          assert(size > 0 && size <= N);
9          n = size;
10         fill(tree, tree + 4 * n + 5, 0);
11         fill(tag, tag + 4 * n + 5, 0);
12     }
13
14     void Update(int now, int L, int R, long long value)
15     {
16         tree[now] += (R - L + 1) * value;
17         tag[now] += value;
18     }
19
20     void Push_Up(int now)
21     {
22         tree[now] = tree[now << 1] + tree[now << 1 | 1];
23     }
24
25     void Push_Down(int now, int L, int R)
26     {
27         int mid = (L + R) >> 1;
28         Update(now << 1, L, mid, tag[now]);
29         Update(now << 1 | 1, mid + 1, R, tag[now]);
30         tag[now] = 0;
31     }
32
33     void Insert(int ql, int qr, long long value)
34     {
35         Insert(1, 1, n, ql, qr, value);
36     }
37
38     void Insert(int now, int L, int R, int ql, int qr, long long value)
39     {
40         if (ql <= L && R <= qr)
41             return Update(now, L, R, value);
42         Push_Down(now, L, R);
43         int mid = (L + R) >> 1;
44         if (ql <= mid)
45             Insert(now << 1, L, mid, ql, qr, value);
46         if (qr > mid)
47             Insert(now << 1 | 1, mid + 1, R, ql, qr, value);
48         Push_Up(now);
49     }
50
51     long long Query(int ql, int qr)

```

```

52 {
53     return Query(1, 1, n, ql, qr);
54 }
55
56 long long Query(int now, int L, int R, int ql, int qr)
57 {
58     if ( ql <= L && R <= qr )
59         return tree[now];
60     Push_Down(now, L, R);
61     int mid = (L + R) >> 1;
62     long long answer = 0;
63     if ( ql <= mid )
64         answer += Query(now << 1, L, mid, ql, qr);
65     if ( qr > mid )
66         answer += Query(now << 1 | 1, mid + 1, R, ql, qr);
67     return answer;
68 }
69 };

```

17.5 64 位异或线性基

算法约束见动态版本第 6 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Xor_Basis
2 {
3     unsigned long long base[64]{};
4
5     bool Insert(unsigned long long x)
6     {
7         for ( int i = 63; i >= 0; i-- )
8         {
9             if ( !(x >> i & 1) )
10                 continue;
11             if ( !base[i] )
12             {
13                 base[i] = x;
14                 return true;
15             }
16             x ^= base[i];
17         }
18         return false;
19     }
20
21     unsigned long long Query(unsigned long long x = 0) const
22     {
23         for ( int i = 63; i >= 0; i-- )
24             x = max(x, x ^ base[i]);
25         return x;
26     }
27 };

```

第十八章 网络流

18.1 最大流与方案

算法约束见动态版本第 7 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template <int N, int M> struct Network_Flow
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int to, next;
8         ll cap, initial;
9     } edge[2 * M];
10
11     int n, tot, head[N + 1], deep[N + 1], cur[N + 1], que[N + 1];
12
13     void Init(int size)
14     {
15         assert(size <= N);
16         n = size;
17         tot = 0;
18         fill(head, head + n + 1, -1);
19     }
20
21     int Add_Edge(int u, int v, ll cap)
22     {
23         assert(cap >= 0 && tot + 2 <= 2 * M);
24         int id = tot;
25         edge[tot] = {v, head[u], cap, cap};
26         head[u] = tot++;
27         edge[tot] = {u, head[v], 0, 0};
28         head[v] = tot++;
29         return id;
30     }
31
32     bool Bfs(int s, int t)
33     {
34         fill(deep, deep + n + 1, -1);
35         int L = 0, R = 0;
36         que[R++] = s;
37         deep[s] = 0;
38         while ( L < R )
39         {
40             int now = que[L++];
41             for ( int i = head[now]; i != -1; i = edge[i].next )
42             {
43                 int to = edge[i].to;
44                 if ( edge[i].cap && deep[to] == -1 )
45                 {
```

```

46         deep[to] = deep[now] + 1;
47         que[R++] = to;
48     }
49 }
50 }
51 return deep[t] != -1;
52 }
53
54 ll Dfs(int now, int t, ll flow)
55 {
56     if ( now == t )
57         return flow;
58     for ( int &i = cur[now]; i != -1; i = edge[i].next )
59     {
60         int to = edge[i].to;
61         if ( !edge[i].cap || deep[to] != deep[now] + 1 )
62             continue;
63         ll delta = Dfs(to, t, min(flow, edge[i].cap));
64         if ( delta )
65         {
66             edge[i].cap -= delta;
67             edge[i ^ 1].cap += delta;
68             return delta;
69         }
70     }
71     return 0;
72 }
73
74 ll Max_Flow(int s, int t, ll limit = LLONG_MAX)
75 {
76     assert(s != t && limit >= 0);
77     ll answer = 0;
78     while ( answer < limit && Bfs(s, t) )
79     {
80         copy(head, head + n + 1, cur);
81         while ( answer < limit )
82         {
83             ll delta = Dfs(s, t, limit - answer);
84             if ( !delta )
85                 break;
86             answer += delta;
87         }
88     }
89     return answer;
90 }
91
92 ll Used(int id) const
93 {
94     return edge[id].initial - edge[id].cap;
95 }
96
97 vector<int> Cut(int s)

```

```

98     {
99         Bfs(s, s);
100         vector<int> answer;
101         for ( int i = 1; i <= n; i++ )
102             if ( deep[i] != -1 )
103                 answer.push_back(i);
104         return answer;
105     }
106 };

```

18.2 势能最小费用最大流

算法约束见动态版本第 9 页。本节代码独立可抄；采用下列实际接口名。
使用 Insert 加边、Flow 求流、Used 查看方案。

```

1  struct Min_Cost_Flow
2  {
3      using ll = long long;
4      using i128 = __int128_t;
5
6      struct Edge
7      {
8          int from, to;
9          ll cap, cost, initial;
10     };
11
12     int n;
13     vector<Edge> e;
14     vector<vector<int>>> g;
15
16     Min_Cost_Flow(int n) : n(n), g(n + 1)
17     {
18     }
19
20     int Insert(int u, int v, ll cap, ll cost)
21     {
22         assert(cap >= 0 && cost != LLONG_MIN);
23         int id = (int)e.size();
24         e.push_back({u, v, cap, cost, cap});
25         e.push_back({v, u, 0, -cost, 0});
26         g[u].push_back(id);
27         g[v].push_back(id + 1);
28         return id;
29     }
30
31     ll Used(int id) const
32     {
33         return e[id].initial - e[id].cap;
34     }
35
36     // Negative costs allowed; initial residual graph must have no negative cycle.
37     pair<ll, i128> Flow(int s, int t, ll limit = LLONG_MAX)
38     {
39         assert(s != t && limit >= 0);
40         const i128 inf = i128(1) << 120;

```

```

41     vector<i128> h(n + 1, 0), d(n + 1);
42     vector<int> pre(n + 1);
43     // Super-source Bellman-Ford: feasible potentials for every vertex.
44     for ( int it = 0; it < n; it++ )
45     {
46         bool changed = false;
47         for ( auto &a : e )
48             if ( a.cap && h[a.to] > h[a.from] + a.cost )
49             {
50                 h[a.to] = h[a.from] + a.cost;
51                 changed = true;
52             }
53         if ( !changed )
54             break;
55         if ( it == n - 1 )
56             throw invalid_argument("negative cycle");
57     }
58     ll f = 0;
59     i128 cost = 0;
60     while ( f < limit )
61     {
62         fill(d.begin(), d.end(), inf);
63         d[s] = 0;
64         priority_queue<pair<i128, int>,
65                       vector<pair<i128, int>>,
66                       greater<pair<i128, int>>>
67             q;
68         q.push({0, s});
69         while ( !q.empty() )
70         {
71             auto [du, u] = q.top();
72             q.pop();
73             if ( du != d[u] )
74                 continue;
75             for ( int id : g[u] )
76             {
77                 auto &a = e[id];
78                 i128 nd = du + a.cost + h[u] - h[a.to];
79                 if ( a.cap && nd < d[a.to] )
80                 {
81                     d[a.to] = nd;
82                     pre[a.to] = id;
83                     q.push({nd, a.to});
84                 }
85             }
86         }
87         if ( d[t] == inf )
88             break;
89         for ( int u = 1; u <= n; u++ )
90             if ( d[u] != inf )
91                 h[u] += d[u];
92         ll take = limit - f;
93         for ( int u = t; u != s; u = e[pre[u]].from )
94             take = min(take, e[pre[u]].cap);
95         for ( int u = t; u != s; u = e[pre[u]].from )
96         {

```

```
97         int id = pre[u];
98         e[id].cap -= take;
99         e[id ^ 1].cap += take;
100        cost += i128(take) * e[id].cost;
101    }
102    f += take;
103 }
104 return {f, cost};
105 }
106 };
```

18.3 上下界可行循环流

算法约束见动态版本第 11 页。本节代码独立可抄；采用下列实际接口名。

```

1  template <int N, int M> struct Bounded_Circulation
2  {
3      int n, tot;
4      Network_Flow<N + 2, M + N> graph;
5      long long balance[N + 1], lower[M];
6      int id[M];
7      bool solved, feasible;
8
9      void Init(int size)
10     {
11         assert(size <= N);
12         n = size;
13         tot = 0;
14         solved = feasible = false;
15         graph.Init(n + 2);
16         fill(balance, balance + n + 1, 0);
17     }
18
19     int Insert(int u, int v, long long L, long long R)
20     {
21         assert(!solved && 0 <= L && L <= R && tot < M);
22         id[tot] = graph.Add_Edge(u, v, R - L);
23         lower[tot] = L;
24         balance[u] -= L;
25         balance[v] += L;
26         return tot++;
27     }
28
29     bool Solve()
30     {
31         assert(!solved);
32         solved = true;
33         long long need = 0;
34         for ( int u = 1; u <= n; u++ )
35         {
36             if ( balance[u] > 0 )
37             {
38                 graph.Add_Edge(n + 1, u, balance[u]);
39                 need += balance[u];
40             }
41             else if ( balance[u] < 0 )
42                 graph.Add_Edge(u, n + 2, -balance[u]);
43         }
44         return feasible = (graph.Max_Flow(n + 1, n + 2) == need);
45     }
46
47     long long Used(int index) const
48     {
49         assert(feasible);
50         return lower[index] + graph.Used(id[index]);
51     }
52 };

```

第十九章 图论

19.1 非负权最短路

算法约束见动态版本第 13 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Shortest_Path
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Shortest_Path(int n) : n(n), g(n + 1)
11    {
12    }
13
14    void Insert(int u, int v, ll w)
15    {
16        assert(w >= 0);
17        g[u].push_back({v, w});
18    }
19
20    void Run(int s)
21    {
22        dis.assign(n + 1, inf);
23        pre.assign(n + 1, -1);
24        dis[s] = 0;
25        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
26        q.push({0, s});
27        while ( !q.empty() )
28        {
29            auto [d, u] = q.top();
30            q.pop();
31            if ( d != dis[u] )
32                continue;
33            for ( auto [v, w] : g[u] )
34                if ( w < inf - d && d + w < dis[v] )
35                {
36                    dis[v] = d + w;
37                    pre[v] = u;
38                    q.push({dis[v], v});
39                }
40        }
41    }
42
43    vector<int> Path(int t) const
44    {
```

```

45     if ( dis[t] == inf )
46         return {};
47     vector<int> ans;
48     for ( ; t != -1; t = pre[t] )
49         ans.push_back(t);
50     reverse(ans.begin(), ans.end());
51     return ans;
52 }
53 };

```

19.2 强连通分量

算法约束见动态版本第 14 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Strong_Component
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     Strong_Component(int n) : n(n), g(n + 1), rg(n + 1)
8     {
9     }
10
11     void Insert(int u, int v)
12     {
13         g[u].push_back(v);
14         rg[v].push_back(u);
15     }
16
17     // Iterative Kosaraju; component IDs are in topological order, 1..cnt.
18     void Run()
19     {
20         vector<int> vis(n + 1), order;
21         cnt = 0;
22         bel.assign(n + 1, 0);
23         for ( int s = 1; s <= n; s++ )
24             if ( !vis[s] )
25             {
26                 vector<pair<int, int>> st{{s, 0}};
27                 vis[s] = 1;
28                 while ( !st.empty() )
29                 {
30                     int u = st.back().first, &i = st.back().second;
31                     if ( i == (int)g[u].size() )
32                     {
33                         order.push_back(u);
34                         st.pop_back();
35                     }
36                     else
37                     {
38                         int v = g[u][i++];
39                         if ( !vis[v] )
40                         {
41                             vis[v] = 1;

```

```
42         st.push_back({v, 0});
43     }
44 }
45 }
46 }
47 reverse(order.begin(), order.end());
48 for ( int s : order )
49     if ( !bel[s] )
50     {
51         ++cnt;
52         vector<int> st{s};
53         bel[s] = cnt;
54         while ( !st.empty() )
55         {
56             int u = st.back();
57             st.pop_back();
58             for ( int v : rg[u] )
59                 if ( !bel[v] )
60                 {
61                     bel[v] = cnt;
62                     st.push_back(v);
63                 }
64             }
65         }
66     }
67 };
```

19.3 2-SAT 与任意可行方案

算法约束见动态版本第 16 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Two_SAT
2 {
3     int n;
4     Strong_Component g;
5     vector<int> ans;
6
7     Two_SAT(int n) : n(n), g(2 * n)
8     {
9     }
10
11     int id(int x, bool value) const
12     {
13         return 2 * x - 1 + value;
14     }
15
16     void Insert(int x, bool a, int y, bool b)
17     {
18         g.Insert(id(x, !a), id(y, b));
19         g.Insert(id(y, !b), id(x, a));
20     }
21
22     bool Solve()
23     {
24         g.Run();
25         ans.assign(n + 1, 0);
26         for ( int x = 1; x <= n; x++ )
27         {
28             if ( g.bel[id(x, 0)] == g.bel[id(x, 1)] )
29                 return false;
30             ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
31         }
32         return true;
33     }
34 };

```

19.4 二分图最大匹配

算法约束见动态版本第 16 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Bipartite_Matching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep;
6
7     Bipartite_Matching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1)
9     {
10     }
11
12     void Insert(int u, int v)
13     {

```

```

14     g[u].push_back(v);
15 }
16
17 bool Bfs()
18 {
19     queue<int> q;
20     fill(dep.begin(), dep.end(), -1);
21     for ( int u = 1; u <= n; u++ )
22         if ( !l[u] )
23         {
24             dep[u] = 0;
25             q.push(u);
26         }
27     bool ok = false;
28     while ( !q.empty() )
29     {
30         int u = q.front();
31         q.pop();
32         for ( int v : g[u] )
33             if ( !r[v] )
34                 ok = true;
35             else if ( dep[r[v]] < 0 )
36             {
37                 dep[r[v]] = dep[u] + 1;
38                 q.push(r[v]);
39             }
40     }
41     return ok;
42 }
43
44 bool Dfs(int u)
45 {
46     for ( int v : g[u] )
47         if ( !r[v] || (dep[r[v]] == dep[u] + 1 && Dfs(r[v])) )
48         {
49             l[u] = v;
50             r[v] = u;
51             return true;
52         }
53     dep[u] = -1;
54     return false;
55 }
56
57 int Solve()
58 {
59     int ans = 0;
60     for ( int u = 1; u <= n; u++ )
61         ans += (l[u] != 0);
62     while ( Bfs() )
63         for ( int u = 1; u <= n; u++ )
64             if ( !l[u] )
65                 ans += Dfs(u);
66     return ans;
67 }
68 };

```

19.5 割点与桥

算法约束见动态版本第 19 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Low_Link
2 {
3     int n, timer = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge;
6     int edges = 0;
7
8     Low_Link(int n) : n(n), g(n + 1)
9     {
10    }
11
12    int Insert(int u, int v)
13    {
14        g[u].push_back({v, edges});
15        g[v].push_back({u, edges});
16        return edges++;
17    }
18
19    void Dfs(int u, int pe)
20    {
21        dfn[u] = low[u] = ++timer;
22        int children = 0;
23        for ( auto [v, id] : g[u] )
24        {
25            if ( id == pe )
26                continue;
27            if ( !dfn[v] )
28            {
29                ++children;
30                Dfs(v, id);
31                low[u] = min(low[u], low[v]);
32                if ( low[v] > dfn[u] )
33                    bridge[id] = 1;
34                if ( pe != -1 && low[v] >= dfn[u] )
35                    cut[u] = 1;
36            }
37            else
38                low[u] = min(low[u], dfn[v]);
39        }
40        if ( pe == -1 && children > 1 )
41            cut[u] = 1;
42    }
43
44    void Run()
45    {

```

```
46     timer = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     cut = dfn;
50     bridge.assign(edges, 0);
51     for ( int u = 1; u <= n; u++ )
52         if ( !dfn[u] )
53             Dfs(u, -1);
54     }
55 };
```

第二十章 树上算法

20.1 重链剖分与 LCA

算法约束见动态版本第 21 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Heavy_Light-Decomposition
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     Heavy_Light-Decomposition(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void Insert(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     // Connected tree only; iterative build avoids chain-shaped DFS stack overflow.
27     void Build(int root = 1)
28     {
29         fill(fa.begin(), fa.end(), 0);
30         fill(son.begin(), son.end(), 0);
31         vector<int> order{root};
32         fa[root] = root;
33         dep[root] = 0;
34         for (int i = 0; i < (int)order.size(); i++)
35         {
36             int u = order[i];
37             for (int v : g[u])
38                 if (v != fa[u])
39                 {
40                     fa[v] = u;
41                     dep[v] = dep[u] + 1;
42                     order.push_back(v);
43                 }
44         }
45         assert((int)order.size() == n);
46         for (int i = n - 1; i >= 0; i-- )
```

```

47     {
48         int u = order[i];
49         siz[u] = 1;
50         for ( int v : g[u] )
51             if ( fa[v] == u && v != u )
52             {
53                 siz[u] += siz[v];
54                 if ( !son[u] || siz[v] > siz[son[u]] )
55                     son[u] = v;
56             }
57     }
58     timer = 0;
59     vector<pair<int, int>> st{{root, root}};
60     while ( !st.empty() )
61     {
62         auto [u, t] = st.back();
63         st.pop_back();
64         for ( ; u; u = son[u] )
65         {
66             top[u] = t;
67             dfn[u] = ++timer;
68             rk[timer] = u;
69             for ( int v : g[u] )
70                 if ( fa[v] == u && v != u && v != son[u] )
71                     st.push_back({v, v});
72         }
73     }
74 }
75
76 int Lca(int u, int v) const
77 {
78     while ( top[u] != top[v] )
79     {
80         if ( dep[top[u]] < dep[top[v]] )
81             swap(u, v);
82         u = fa[top[u]];
83     }
84     return dep[u] < dep[v] ? u : v;
85 }
86
87 // Commutative operations only; edge=true excludes the LCA's position.
88 template <class F> void Path(int u, int v, F work, bool edge = false) const
89 {
90     while ( top[u] != top[v] )
91     {
92         if ( dep[top[u]] < dep[top[v]] )

```

```
93         swap(u, v);
94         work(dfn[top[u]], dfn[u]);
95         u = fa[top[u]];
96     }
97     if ( dep[u] > dep[v] )
98         swap(u, v);
99     if ( dfn[u] + edge <= dfn[v] )
100         work(dfn[u] + edge, dfn[v]);
101 }
102 };
```

第二十一章 字符串

21.1 前缀函数

算法约束见动态版本第 24 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Prefix_Function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for ( int i = 1; i < n; i++ )
6     {
7         int j = p[i - 1];
8         while ( j && s[i] != s[j] )
9             j = p[j - 1];
10        p[i] = j + (s[i] == s[j]);
11    }
12    return p;
13 }
```

21.2 KMP 匹配

算法约束见动态版本第 24 页。本节代码独立可抄；采用下列实际接口名。

```
1 inline vector<int> Kmp_Match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = Prefix_Function(t);
5     vector<int> ans;
6     for ( int i = 0, j = 0; i < (int)s.size(); i++ )
7     {
8         while ( j && s[i] != t[j] )
9             j = p[j - 1];
10        j += (s[i] == t[j]);
11        if ( j == (int)t.size() )
12        {
13            ans.push_back(i - j + 1);
14            j = p[j - 1];
15        }
16    }
17    return ans;
18 }
```

21.3 Z 函数

算法约束见动态版本第 25 页。本节代码独立可抄；采用下列实际接口名。

```

1 inline vector<int> Z_Function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if ( n )
6         a[0] = n;
7     for ( int i = 1, l = 0, r = 0; i < n; i++ )
8     {
9         if ( i < r )
10            a[i] = min(r - i, a[i - l]);
11        while ( i + a[i] < n && s[a[i]] == s[i + a[i]] )
12            ++a[i];
13        if ( i + a[i] > r )
14        {
15            l = i;
16            r = i + a[i];
17        }
18    }
19    return a;
20 }
```

21.4 Manacher 回文半径

算法约束见动态版本第 25 页。本节代码独立可抄；采用下列实际接口名。

```

1 // odd[i] includes the center; even[i] is centered before i.
2 inline pair<vector<int>, vector<int>> Manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for ( int i = 0, l = 0, r = -1; i < n; i++ )
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while ( i - k >= 0 && i + k < n && s[i - k] == s[i + k] )
10            ++k;
11        odd[i] = k--;
12        if ( i + k > r )
13        {
14            l = i - k;
15            r = i + k;
16        }
17    }
18    for ( int i = 0, l = 0, r = -1; i < n; i++ )
19    {
20        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
21        while ( i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k] )
22            ++k;
```

```

23     even[i] = k--;
24     if ( i + k > r )
25     {
26         l = i - k - 1;
27         r = i + k;
28     }
29 }
30 return {odd, even};
31 }

```

21.5 最小循环表示

算法约束见动态版本第 26 页。本节代码独立可抄；采用下列实际接口名。

```

1 inline int Minimum_Rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if ( !n )
5         return 0;
6     while ( i < n && j < n && k < n )
7     {
8         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
9         if ( a == b )
10            {
11                ++k;
12                continue;
13            }
14         if ( a > b )
15            {
16                i += k + 1;
17                if ( i == j )
18                    ++i;
19            }
20         else
21            {
22                j += k + 1;
23                if ( i == j )
24                    ++j;
25            }
26         k = 0;
27     }
28     return min(i, j);
29 }

```

21.6 AC 自动机出现次数

算法约束见动态版本第 26 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct AC_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;

```

```

7     };
8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    int Insert(const string &s)
14    {
15        assert(!built && !s.empty());
16        int u = 0;
17        for ( char ch : s )
18        {
19            int c = ch - 'a';
20            assert(0 <= c && c < 26);
21            if ( !a[u].go[c] )
22            {
23                int v = (int)a.size();
24                a[u].go[c] = v;
25                a.push_back(Node{});
26            }
27            u = a[u].go[c];
28        }
29        return u;
30    }
31
32    void Build()
33    {
34        assert(!built);
35        built = true;
36        queue<int> q;
37        for ( int v : a[0].go )
38            if ( v )
39                q.push(v);
40        while ( !q.empty() )
41        {
42            int u = q.front();
43            q.pop();
44            order.push_back(u);
45            for ( int c = 0; c < 26; c++ )
46            {
47                int v = a[u].go[c];
48                if ( v )
49                {
50                    a[v].fail = a[a[u].fail].go[c];
51                    q.push(v);
52                }
53                else
54                    a[u].go[c] = a[a[u].fail].go[c];
55            }
56        }
57    }
58
59    vector<long long> count(const string &s) const
60    {
61        assert(built);
62        vector<long long> ans(a.size());

```

```
63     int u = 0;
64     for ( char ch : s )
65     {
66         u = a[u].go[ch - 'a'];
67         ++ans[u];
68     }
69     for ( int i = (int)order.size() - 1; i >= 0; i-- )
70         ans[a[order[i]].fail] += ans[order[i]];
71     return ans;
72 }
73 };
```

21.7 倍增后缀数组与 LCP

算法约束见动态版本第 29 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Array
2 {
3     vector<int> sa, rk, lcp;
4
5     Suffix_Array(const string &s)
6     {
7         int n = (int)s.size();
8         sa.resize(n);
9         rk.resize(n);
10        lcp.assign(n, 0);
11        for ( int i = 0; i < n; i++ )
12        {
13            sa[i] = i;
14            rk[i] = (unsigned char)s[i];
15        }
16        vector<int> tmp(n);
17        for ( int k = 1; k < n; k *= 2 )
18        {
19            auto key = [&](int i)
20            {
21                return pair{rk[i], i + k < n ? rk[i + k] : -1};
22            };
23            sort(sa.begin(),
24                sa.end(),
25                [&](int i, int j)
26                {
27                    return key(i) < key(j);
28                });
29            tmp[sa[0]] = 0;
30            for ( int i = 1; i < n; i++ )
31                tmp[sa[i]] = tmp[sa[i - 1]] + (key(sa[i]) != key(sa[i - 1]));
32            rk = tmp;
33            if ( rk[sa.back()] == n - 1 )
34                break;
35        }
36        for ( int i = 0; i < n; i++ )
37            rk[sa[i]] = i;
38        for ( int i = 0, k = 0; i < n; i++ )
39        {
40            if ( !rk[i] )
41            {
42                k = 0;
43                continue;
44            }

```

```

45     int j = sa[rk[i] - 1];
46     while ( i + k < n && j + k < n && s[i + k] == s[j + k] )
47         ++k;
48     lcp[rk[i]] = k;
49     if ( k )
50         --k;
51 }
52 }
53 };

```

21.8 后缀自动机

算法约束见动态版本第 30 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
12
13    void Extend(int c)
14    {
15        assert(0 <= c && c < 26);
16        int cur = (int)a.size();
17        a.push_back(Node{});
18        a[cur].len = a[last].len + 1;
19        a[cur].occ = 1;
20        int p = last;
21        while ( p != -1 && !a[p].go[c] )
22        {
23            a[p].go[c] = cur;
24            p = a[p].link;
25        }
26        if ( p == -1 )
27            a[cur].link = 0;
28        else
29        {
30            int q = a[p].go[c];
31            if ( a[p].len + 1 == a[q].len )
32                a[cur].link = q;
33            else
34            {
35                int clone = (int)a.size();
36                Node copy = a[q];
37                a.push_back(copy);
38                a[clone].len = a[p].len + 1;
39                a[clone].occ = 0;
40                while ( p != -1 && a[p].go[c] == q )
41                {

```

```

42         a[p].go[c] = clone;
43         p = a[p].link;
44     }
45     a[q].link = a[cur].link = clone;
46 }
47 }
48 last = cur;
49 }
50
51 vector<long long> Counts() const
52 {
53     vector<int> bucket(a[last].len + 1), order(a.size());
54     vector<long long> ans(a.size());
55     for ( auto &v : a )
56         ++bucket[v.len];
57     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
58     for ( int i = 0; i < (int)a.size(); i++ )
59     {
60         order[--bucket[a[i].len]] = i;
61         ans[i] = a[i].occ;
62     }
63     for ( int i = (int)order.size() - 1; i > 0; i-- )
64         ans[a[order[i]].link] += ans[order[i]];
65     return ans;
66 }
67
68 long long Distinct() const
69 {
70     long long ans = 0;
71     for ( int i = 1; i < (int)a.size(); i++ )
72         ans += a[i].len - a[a[i].link].len;
73     return ans;
74 }
75 };

```

第二十二章 数论

22.1 模运算、素性、CRT 与类欧几里德

算法约束见动态版本第 32 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Number_Theory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7
8     static ull Mul(ull a, ull b, ull m)
9     {
10         return u128(a) * b % m;
11     }
12
13     static ull Power(ull a, ull b, ull m)
14     {
15         assert(m);
16         ull r = 1 % m;
17         a %= m;
18         for ( ; b; b >>= 1, a = Mul(a, a, m) )
19             if ( b & 1 )
20                 r = Mul(r, a, m);
21         return r;
22     }
23
24     static bool Prime(ull n)
25     {
26         if ( n < 2 )
27             return false;
28         for ( ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37} )
29             if ( n % p == 0 )
30                 return n == p;
31         ull d = n - 1;
32         int s = 0;
33         while ( !(d & 1) )
34         {
35             d >>= 1;
36             ++s;
37         }
38         for ( ull a :
39             {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL} )
40         {
41             if ( a % n == 0 )
42                 continue;
43             ull x = Power(a, d, n);
44             if ( x == 1 || x == n - 1 )
45                 continue;
46             bool ok = false;
```

```

47     for ( int j = 1; j < s; j++ )
48     {
49         x = Mul(x, x, n);
50         if ( x == n - 1 )
51         {
52             ok = true;
53             break;
54         }
55     }
56     if ( !ok )
57         return false;
58 }
59 return true;
60 }
61
62 static i128 Exgcd(i128 a, i128 b, i128 &x, i128 &y)
63 {
64     if ( !b )
65     {
66         x = 1;
67         y = 0;
68         return a;
69     }
70     i128 d = Exgcd(b, a % b, y, x);
71     y -= a / b * x;
72     return d;
73 }
74
75 static ll Inverse(ll a, ll m)
76 {
77     assert(m > 0);
78     a %= m;
79     if ( a < 0 )
80         a += m;
81     i128 x, y;
82     if ( Exgcd(a, m, x, y) != 1 )
83         return -1;
84     return (x % m + m) % m;
85 }
86
87 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
88 static bool Crt(ll &r, ll &m, ll b, ll n)
89 {
90     assert(m > 0 && n > 0);
91     r %= m;
92     if ( r < 0 )
93         r += m;
94     b %= n;
95     if ( b < 0 )
96         b += n;
97     i128 x, y;
98     ll g = (ll)Exgcd(m, n, x, y);
99     i128 diff = i128(b) - r;
100     if ( diff % g )
101         return false;
102     i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;

```

```

103     if ( mod > LLONG_MAX )
104         throw overflow_error("CRT modulus");
105     r = (r + i128(m) * k) % mod;
106     m = (ll)mod;
107     return true;
108 }
109
110 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
111 static i128 Floor_Sum(ll n, ll m, ll a, ll b)
112 {
113     assert(n >= 0 && m > 0);
114     i128 ans = 0;
115     auto norm = [&](ll &v)
116     {
117         ll q = v / m;
118         if ( v % m < 0 )
119             --q;
120         v = (ll)(i128(v) - i128(q) * m);
121         return q;
122     };
123     ans += i128(norm(a)) * n * (n - 1) / 2;
124     ans += i128(norm(b)) * n;
125     while ( true )
126     {
127         ans += i128(n) * (n - 1) / 2 * (a / m);
128         a %= m;
129         ans += i128(n) * (b / m);
130         b %= m;
131         i128 y = i128(a) * n + b;
132         if ( y < m )
133             break;
134         n = (ll)(y / m);
135         b = (ll)(y % m);
136         swap(a, m);
137     }
138     return ans;
139 }
140 };

```

22.2 Pollard-Rho 因数分解

算法约束见动态版本第 35 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Pollard_Rho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     Pollard_Rho(ull seed = 712367821) : rng(seed)
8     {
9     }
10
11     ull Rho(ull n)
12     {
13         if ( n % 2 == 0 )
14             return 2;
15         for ( ;; )
16         {
17             ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
18             auto f = [&](ull v)
19             {
20                 return (Number_Theory::Mul(v, v, n) + u128(c)) % n;
21             };
22             // Retry bounded attempts; Las Vegas: only exact divisors returned.
23             for ( int i = 0; i < 200000 && d == 1; i++ )
24             {
25                 x = f(x);
26                 y = f(f(y));
27                 d = gcd(x > y ? x - y : y - x, n);
28             }
29             if ( 1 < d && d < n )
30                 return d;
31         }
32     }
33
34     void Split(ull n, vector<ull> &a)
35     {
36         if ( n == 1 )
37             return;
38         if ( Number_Theory::Prime(n) )
39         {
40             a.push_back(n);
41             return;
42         }
43         ull d = Rho(n);
44         Split(d, a);
45         Split(n / d, a);
46     }

```

```

47
48     vector<ull> Factor(ull n)
49     {
50         assert(n >= 1);
51         vector<ull> a;
52         Split(n, a);
53         sort(a.begin(), a.end());
54         return a;
55     }
56 };

```

22.3 线性筛、欧拉函数与 Möbius 函数

算法约束见动态版本第 36 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Linear_Sieve
2 {
3     vector<int> Prime, lp, phi, mu;
4
5     Linear_Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if ( n )
8             phi[1] = mu[1] = 1;
9         for ( int i = 2; i <= n; i++ )
10            {
11                if ( !lp[i] )
12                {
13                    lp[i] = i;
14                    Prime.push_back(i);
15                    phi[i] = i - 1;
16                    mu[i] = -1;
17                }
18                for ( int p : Prime )
19                {
20                    if ( p > n / i )
21                        break;
22                    int j = i * p;
23                    lp[j] = p;
24                    if ( i % p == 0 )
25                    {
26                        phi[j] = phi[i] * p;
27                        mu[j] = 0;
28                        break;
29                    }
30                    phi[j] = phi[i] * (p - 1);
31                    mu[j] = -mu[i];
32                }
33            }
34     }
35 };

```

22.4 静态模整数

算法约束见动态版本第 37 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int mod> struct Mod_Int
2 {
3     int v;
4
5     Mod_Int(long long x = 0) : v((x % mod + mod) % mod)
6     {
7     }
8
9     Mod_Int operator+(Mod_Int b) const
10    {
11        return Mod_Int((long long)v + b.v);
12    }
13
14    Mod_Int operator-(Mod_Int b) const
15    {
16        return Mod_Int((long long)v - b.v);
17    }
18
19    Mod_Int operator*(Mod_Int b) const
20    {
21        return Mod_Int(1LL * v * b.v);
22    }
23
24    Mod_Int Pow(long long e) const
25    {
26        assert(e >= 0);
27        Mod_Int a = *this, r = 1;
28        for ( ; e; e >>= 1, a = a * a )
29            if ( e & 1 )
30                r = r * a;
31        return r;
32    }
33
34    Mod_Int Inv() const
35    {
36        assert(v);
37        return Pow(mod - 2);
38    } // prime modulus
39
40    Mod_Int operator/(Mod_Int b) const
41    {
42        return *this * b.Inv();
43    }
44 };

```

22.5 阶乘组合数

算法约束见动态版本第 38 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod> struct Combination
2 {
3     using Z = Mod_Int<mod>;
4     vector<Z> fac, ifac;
5
6     Combination(int n) : fac(n + 1), ifac(n + 1)
7     {
8         assert(0 <= n && n < mod);
9         fac[0] = 1;
10        for ( int i = 1; i <= n; i++ )
11            fac[i] = fac[i - 1] * i;
12        ifac[n] = fac[n].Inv();
13        for ( int i = n; i; i-- )
14            ifac[i - 1] = ifac[i] * i;
15    }
16
17    Z Choose(int n, int k) const
18    {
19        if ( k < 0 || k > n )
20            return 0;
21        assert(n < (int)fac.size());
22        return fac[n] * ifac[k] * ifac[n - k];
23    }
24 };
```

第二十三章 多项式

23.1 NTT、形式幂级数、FWT 与递推

算法约束见动态版本第 39 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = Mod_Int<mod>;
5     using Poly = vector<Z>;
6
7     static void Ntt(Poly &a, bool invert = false)
8     {
9         int n = (int)a.size();
10        assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
11        for ( int i = 1, j = 0; i < n; i++ )
12        {
13            int bit = n >> 1;
14            for ( ; j & bit; bit >>= 1 )
15                j ^= bit;
16            j ^= bit;
17            if ( i < j )
18                swap(a[i], a[j]);
19        }
20        for ( int len = 2; len <= n; len *= 2 )
21        {
22            Z step = Z(3).Pow((mod - 1) / len);
23            if ( invert )
24                step = step.Inv();
25            for ( int i = 0; i < n; i += len )
26            {
27                Z w = 1;
28                for ( int j = 0; j < len / 2; j++, w = w * step )
29                {
30                    Z u = a[i + j], v = a[i + j + len / 2] * w;
31                    a[i + j] = u + v;
32                    a[i + j + len / 2] = u - v;
33                }
34            }
35        }
36        if ( invert )
37        {
38            Z Inv = Z(n).Inv();
39            for ( auto &x : a )
40                x = x * Inv;
41        }
42    }
43
44    static Poly Multiply(Poly a, Poly b)
45    {
46        if ( a.empty() || b.empty() )
```

```

47     return {};
48     int size = (int)a.size() + (int)b.size() - 1, n = 1;
49     assert(size <= (1 << 23));
50     while ( n < size )
51         n *= 2;
52     a.resize(n);
53     b.resize(n);
54     Ntt(a);
55     Ntt(b);
56     for ( int i = 0; i < n; i++ )
57         a[i] = a[i] * b[i];
58     Ntt(a, true);
59     a.resize(size);
60     return a;
61 }
62
63 static Poly Inverse(const Poly &a, int n)
64 {
65     assert(n > 0 && !a.empty() && a[0].v);
66     Poly b{a[0].Inv()};
67     while ( (int)b.size() < n )
68     {
69         int m = min(n, 2 * (int)b.size());
70         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
71         auto t = Multiply(f, b);
72         t.resize(m);
73         for ( auto &x : t )
74             x = Z(0) - x;
75         t[0] = t[0] + 2;
76         b = Multiply(b, t);
77         b.resize(m);
78     }
79     return b;
80 }
81
82 static Poly Derivative(const Poly &a)
83 {
84     Poly b;
85     for ( int i = 1; i < (int)a.size(); i++ )
86         b.push_back(a[i] * i);
87     return b;
88 }
89
90 static Poly Integral(const Poly &a)
91 {
92     Poly b(a.size() + 1);
93     vector<Z> Inv(a.size() + 1);
94     if ( !a.empty() )
95         Inv[1] = 1;
96     for ( int i = 2; i <= (int)a.size(); i++ )
97         Inv[i] = Z(mod - mod / i) * Inv[mod % i];
98     for ( int i = 0; i < (int)a.size(); i++ )
99         b[i + 1] = a[i] * Inv[i + 1];
100     return b;
101 }
102

```

```

103 static Poly Log(const Poly &a, int n)
104 {
105     assert(n > 0 && !a.empty() && a[0].v == 1);
106     auto b = Multiply(Derivative(a), Inverse(a, n));
107     b.resize(n - 1);
108     return Integral(b);
109 }
110
111 static Poly Exp(const Poly &a, int n)
112 {
113     assert(n > 0 && (a.empty() || a[0].v == 0));
114     Poly b{1};
115     while ( (int)b.size() < n )
116     {
117         int m = min(n, 2 * (int)b.size());
118         auto t = Log(b, m);
119         for ( int i = 0; i < m; i++ )
120             t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
121         t[0] = t[0] + 1;
122         b = Multiply(b, t);
123         b.resize(m);
124     }
125     return b;
126 }
127
128 static void Fwt(Poly &a, char op, bool invert = false)
129 {
130     int n = (int)a.size();
131     assert(n > 0 && (n & (n - 1)) == 0);
132     const Z inv2 = (mod + 1) / 2;
133     for ( int len = 1; len < n; len *= 2 )
134         for ( int i = 0; i < n; i += len * 2 )
135             for ( int j = 0; j < len; j++ )
136             {
137                 Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
138                 if ( op == '^' )
139                 {
140                     u = x + y;
141                     v = x - y;
142                     if ( invert )
143                     {
144                         u = u * inv2;
145                         v = v * inv2;
146                     }
147                 }
148                 else if ( op == '|' )
149                     v = invert ? y - x : y + x;
150                 else
151                 {
152                     assert(op == '&');
153                     u = invert ? x - y : x + y;
154                 }
155             }
156 }
157
158 // c: s[n]=sum c[i-1]*s[n-i], i=1..L.

```

```

159 static Poly Berlekamp_Massey(const Poly &s)
160 {
161     Poly c{1}, b{1};
162     int l = 0, m = 1;
163     Z last = 1;
164     for ( int n = 0; n < (int)s.size(); n++ )
165     {
166         Z d = s[n];
167         for ( int i = 1; i <= l; i++ )
168             d = d + c[i] * s[n - i];
169         if ( !d.v )
170         {
171             ++m;
172             continue;
173         }
174         auto old = c;
175         Z coef = d / last;
176         if ( c.size() < b.size() + m )
177             c.resize(b.size() + m);
178         for ( int j = 0; j < (int)b.size(); j++ )
179             c[j + m] = c[j + m] - coef * b[j];
180         if ( 2 * l <= n )
181         {
182             l = n + 1 - l;
183             b = old;
184             last = d;
185             m = 1;
186         }
187         else
188             ++m;
189     }
190     c.erase(c.begin());
191     for ( auto &x : c )
192         x = Z(0) - x;
193     return c;
194 }
195
196 static Z Recurrence(Poly init, const Poly &c, unsigned long long n)
197 {
198     int k = (int)c.size();
199     if ( n < init.size() )
200         return init[n];
201     if ( !k )
202         return 0;
203     assert((int)init.size() >= k);
204     auto combine = [&](const Poly &a, const Poly &b)
205     {
206         Poly t(2 * k - 1);
207         for ( int i = 0; i < k; i++ )
208             for ( int j = 0; j < k; j++ )
209                 t[i + j] = t[i + j] + a[i] * b[j];
210         for ( int i = 2 * k - 2; i >= k; i-- )
211             for ( int j = 1; j <= k; j++ )
212                 t[i - j] = t[i - j] + t[i] * c[j - 1];
213         t.resize(k);
214         return t;

```

```
215     };
216     Poly a(k), r(k);
217     r[0] = 1;
218     if ( k == 1 )
219         a[0] = c[0];
220     else
221         a[1] = 1;
222     for ( ; n; n >>= 1, a = combine(a, a) )
223         if ( n & 1 )
224             r = combine(r, a);
225     Z ans = 0;
226     for ( int i = 0; i < k; i++ )
227         ans = ans + r[i] * init[i];
228     return ans;
229 }
230 };
```

第二十四章 代数与数论进阶

24.1 模高斯消元、行列式、矩阵树定理

算法约束见动态版本第 44 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod> struct Linear_Algebra
2 {
3     using Z = Mod_Int<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution Solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for ( const auto &v : a )
20            assert((int)v.size() == n + 1);
21        for ( int col = 0; col < n && row < m; col++ )
22        {
23            int p = row;
24            while ( p < m && !a[p][col].v )
25                ++p;
26            if ( p == m )
27                continue;
28            swap(a[p], a[row]);
29            Z Inv = a[row][col].Inv();
30            for ( int j = col; j <= n; j++ )
31                a[row][j] = a[row][j] * Inv;
32            for ( int i = 0; i < m; i++ )
33                if ( i != row && a[i][col].v )
34                {
35                    Z f = a[i][col];
36                    for ( int j = col; j <= n; j++ )
37                        a[i][j] = a[i][j] - f * a[row][j];
38                }
39            where[col] = row++;
40        }
41        for ( int i = row; i < m; i++ )
42            if ( a[i][n].v )
43                return {false, row, {}, {}};
44        Solution ans{true, row, vector<Z>(n), {}};
45        for ( int j = 0; j < n; j++ )
46            if ( where[j] != -1 )
```

```

47         ans.particular[j] = a[where[j]][n];
48     for ( int j = 0; j < n; j++ )
49         if ( where[j] == -1 )
50         {
51             vector<Z> v(n);
52             v[j] = 1;
53             for ( int k = 0; k < n; k++ )
54                 if ( where[k] != -1 )
55                     v[k] = Z(0) - a[where[k]][j];
56             ans.kernel.push_back(v);
57         }
58     return ans;
59 }
60
61 static Z Determinant(Matrix a)
62 {
63     int n = (int)a.size();
64     Z ans = 1;
65     for ( int i = 0; i < n; i++ )
66     {
67         assert((int)a[i].size() == n);
68         int p = i;
69         while ( p < n && !a[p][i].v )
70             ++p;
71         if ( p == n )
72             return 0;
73         if ( p != i )
74         {
75             swap(a[p], a[i]);
76             ans = Z(0) - ans;
77         }
78         ans = ans * a[i][i];
79         Z Inv = a[i][i].Inv();
80         for ( int j = i + 1; j < n; j++ )
81         {
82             Z f = a[j][i] * Inv;
83             for ( int k = i; k < n; k++ )
84                 a[j][k] = a[j][k] - f * a[i][k];
85         }
86     }
87     return ans;
88 }
89
90 static Matrix Multiply(const Matrix &a, const Matrix &b)
91 {
92     assert(!a.empty() && !b.empty());
93     int n = a.size(), m = b[0].size(), k = b.size();
94     assert((int)a[0].size() == k);
95     Matrix c(n, vector<Z>(m));
96     for ( int i = 0; i < n; i++ )
97         for ( int t = 0; t < k; t++ )
98             for ( int j = 0; j < m; j++ )
99                 c[i][j] = c[i][j] + a[i][t] * b[t][j];
100     return c;
101 }
102

```

```

103 static Matrix Power(Matrix a, unsigned long long e)
104 {
105     int n = a.size();
106     assert(n > 0 && (int)a[0].size() == n);
107     Matrix r(n, vector<Z>(n));
108     for ( int i = 0; i < n; i++ )
109         r[i][i] = 1;
110     for ( ; e; e >>= 1, a = Multiply(a, a) )
111         if ( e & 1 )
112             r = Multiply(r, a);
113     return r;
114 }
115
116 // Undirected multigraph, vertices 0..n-1; loops ignored.
117 static Z Spanning_Trees(int n, const vector<pair<int, int>> &edges)
118 {
119     assert(n > 0);
120     Matrix lap(n, vector<Z>(n));
121     for ( auto [u, v] : edges )
122         if ( u != v )
123         {
124             lap[u][u] = lap[u][u] + 1;
125             lap[v][v] = lap[v][v] + 1;
126             lap[u][v] = lap[u][v] - 1;
127             lap[v][u] = lap[v][u] - 1;
128         }
129     lap.pop_back();
130     for ( auto &row : lap )
131         row.pop_back();
132     return Determinant(lap);
133 }
134 };

```

24.2 杜教筛

算法约束见动态版本第 47 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Du_Jiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    Du_Jiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        Linear_Sieve s(limit);
14        for ( int i = 1; i <= limit; i++ )
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll Mertens(ll n)
22    {
23        if ( n <= limit )
24            return pmu[n];
25        if ( mmu.count(n) )
26            return mmu[n];
27        ll ans = 1;
28        for ( ll l = 2, r; l <= n; l = r + 1 )
29        {
30            r = n / (n / l);
31            ans -= (r - l + 1) * Mertens(n / l);
32        }
33        return mmu[n] = ans;
34    }
35
36    I Totient_Sum(ll n)
37    {
38        if ( n <= limit )
39            return pphi[n];
40        if ( mphi.count(n) )
41            return mphi[n];
42        I ans = I(n) * (n + 1) / 2;
43        for ( ll l = 2, r; l <= n; l = r + 1 )
44        {
45            r = n / (n / l);
46            ans -= I(r - l + 1) * Totient_Sum(n / l);
47        }
48        return mphi[n] = ans;
49    }
50 };

```

24.3 扩展 BSGS

算法约束见动态版本第 48 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Discrete_Log
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; O(sqrt(m)) time/storage. Smallest x>=0, or -1.
6     static ll Solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if ( m == 1 )
12            return 0;
13        ll offset = 0, k = 1;
14        for ( ll g; (g = gcd(a, m)) > 1; )
15        {
16            if ( b == k )
17                return offset;
18            if ( b % g )
19                return -1;
20            b /= g;
21            m /= g;
22            k = (__int128)k * (a / g) % m;
23            ++offset;
24        }
25        ll target = (__int128)b * Number_Theory::Inverse(k, m) % m;
26        ll step = sqrtl(m) + 1;
27        unordered_map<ll, ll> baby;
28        ll cur = 1 % m;
29        for ( ll j = 0; j < step; j++ )
30        {
31            if ( !baby.count(cur) )
32                baby[cur] = j;
33            cur = (__int128)cur * a % m;
34        }
35        ll Inv = Number_Theory::Inverse(Number_Theory::Power(a, step, m), m);
36        cur = target;
37        for ( ll i = 0; i <= step; i++ )
38        {
39            auto it = baby.find(cur);
40            if ( it != baby.end() )
41                return offset + i * step + it->second;
42            cur = (__int128)cur * Inv % m;
43        }
44        return -1;
45    }
46 };

```

第二十五章 计算几何

25.1 整数精确二维几何

算法约束见动态版本第 49 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Integer_Geometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const
12         {
13             return tie(x, y) < tie(b.x, b.y);
14         }
15
16         bool operator==(Point b) const
17         {
18             return x == b.x && y == b.y;
19         }
20     };
21
22     static I Cross(Point a, Point b, Point c)
23     {
24         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
25     }
26
27     static I Dot(Point a, Point b, Point c)
28     {
29         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
30     }
31
32     static I Distance_Squared(Point a, Point b)
33     {
34         return Dot(a, b, b);
35     }
36
37     static int Sign(I x)
38     {
39         return (x > 0) - (x < 0);
40     }
41
42     static bool On_Segment(Point p, Point a, Point b)
43     {
44         return Cross(a, b, p) == 0 && Dot(p, a, b) <= 0;
45     }
46 }
```

```

47 static bool Intersect(Point a, Point b, Point c, Point d)
48 {
49     int x = Sign(Cross(a, b, c)), y = Sign(Cross(a, b, d));
50     int Z_Function = Sign(Cross(c, d, a)), w = Sign(Cross(c, d, b));
51     if ( !x && On_Segment(c, a, b) )
52         return true;
53     if ( !y && On_Segment(d, a, b) )
54         return true;
55     if ( !Z_Function && On_Segment(a, c, d) )
56         return true;
57     if ( !w && On_Segment(b, c, d) )
58         return true;
59     return x * y < 0 && Z_Function * w < 0;
60 }
61
62 static vector<Point> Convex_Hull(vector<Point> p)
63 {
64     sort(p.begin(), p.end());
65     p.erase(unique(p.begin(), p.end()), p.end());
66     int n = (int)p.size();
67     if ( n < 3 )
68         return p;
69     vector<Point> h;
70     for ( Point v : p )
71     {
72         while ( h.size() > 1 && Cross(h[h.size() - 2], h.back(), v) <= 0 )
73             h.pop_back();
74         h.push_back(v);
75     }
76     int low = (int)h.size();
77     for ( int i = n - 2; i >= 0; i-- )
78     {
79         while ( (int)h.size() > low && Cross(h[h.size() - 2], h.back(), p[i]) <= 0 )
80             h.pop_back();
81         h.push_back(p[i]);
82     }
83     h.pop_back();
84     return h; // CCW, no repeated endpoint, no collinear interior points.
85 }
86
87 static I Area_Twice(const vector<Point> &p)
88 {
89     I ans = 0;
90     int n = (int)p.size();
91     for ( int i = 0; i < n; i++ )
92         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
93     return ans;
94 }
95
96 // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
97 static int Contains(const vector<Point> &p, Point q)
98 {
99     int winding = 0, n = (int)p.size();
100     for ( int i = 0; i < n; i++ )
101     {
102         Point a = p[i], b = p[(i + 1) % n];

```

```

103         if ( On_Segment(q, a, b) )
104             return 1;
105         I c = Cross(a, b, q);
106         if ( a.y <= q.y && b.y > q.y && c > 0 )
107             ++winding;
108         if ( a.y > q.y && b.y <= q.y && c < 0 )
109             --winding;
110     }
111     return winding ? 2 : 0;
112 }
113
114 // Input must be the strict CCW hull above. O(log n).
115 static int Convex_Contains(const vector<Point> &p, Point q)
116 {
117     int n = (int)p.size();
118     if ( n < 3 )
119         return Contains(p, q);
120     I a = Cross(p[0], p[1], q), b = Cross(p[0], p[n - 1], q);
121     if ( a < 0 || b > 0 )
122         return 0;
123     if ( !a )
124         return On_Segment(q, p[0], p[1]) ? 1 : 0;
125     if ( !b )
126         return On_Segment(q, p[0], p[n - 1]) ? 1 : 0;
127     int l = 1, r = n - 1;
128     while ( l + 1 < r )
129     {
130         int m = (l + r) / 2;
131         if ( Cross(p[0], p[m], q) >= 0 )
132             l = m;
133         else
134             r = m;
135     }
136     I c = Cross(p[l], p[r], q);
137     return c < 0 ? 0 : c == 0 ? 1 : 2;
138 }
139
140 static I Diameter_Squared(const vector<Point> &p)
141 {
142     int n = (int)p.size();
143     if ( n < 2 )
144         return 0;
145     if ( n == 2 )
146         return Distance_Squared(p[0], p[1]);
147     I ans = 0;
148     int j = 1;
149     for ( int i = 0; i < n; i++ )
150     {
151         int k = (i + 1) % n;
152         while ( Cross(p[i], p[k], p[(j + 1) % n]) > Cross(p[i], p[k], p[j]) )
153             j = (j + 1) % n;
154         ans =
155             max({ans, Distance_Squared(p[i], p[j]), Distance_Squared(p[k], p[j])});
156     }
157     return ans;
158 }

```

```

159
160 struct PolarLess
161 {
162     static int half(Point p)
163     {
164         return p.y < 0 || (p.y == 0 && p.x < 0);
165     }
166
167     bool operator()(Point a, Point b) const
168     {
169         if ( half(a) != half(b) )
170             return half(a) < half(b);
171         I c = Cross({0, 0}, a, b);
172         if ( c )
173             return c > 0;
174         return Distance_Squared({0, 0}, a) < Distance_Squared({0, 0}, b);
175     }
176 };
177 };

```

25.2 浮点点线圆构造

算法约束见动态版本第 52 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Real_Geometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     Real_Geometry(R eps = 1e-12L) : eps(eps)
8     {
9         assert(eps > 0);
10    }
11
12    struct Point
13    {
14        R x = 0, y = 0;
15
16        Point operator+(Point b) const
17        {
18            return {x + b.x, y + b.y};
19        }
20
21        Point operator-(Point b) const
22        {
23            return {x - b.x, y - b.y};
24        }
25
26        Point operator*(R k) const
27        {
28            return {x * k, y * k};
29        }
30
31        Point operator/(R k) const

```

```

32     {
33         return {x / k, y / k};
34     }
35 };
36
37 struct Circle
38 {
39     Point o;
40     R r;
41 };
42 enum class Kind
43 {
44     none,
45     one,
46     two,
47     infinite,
48     degenerate
49 };
50
51 struct Result
52 {
53     Kind kind;
54     vector<Point> p;
55 };
56
57 static R Dot(Point a, Point b)
58 {
59     return a.x * b.x + a.y * b.y;
60 }
61
62 static R Cross(Point a, Point b)
63 {
64     return a.x * b.y - a.y * b.x;
65 }
66
67 static R norm(Point a)
68 {
69     return hypotl(a.x, a.y);
70 }
71
72 static Point perp(Point a)
73 {
74     return {-a.y, a.x};
75 }
76
77 Point Projection(Point p, Point a, Point b) const
78 {
79     Point v = b - a;
80     R d = Dot(v, v);
81     if ( d == 0 )
82         return a;
83     return a + v * (Dot(p - a, v) / d);
84 }
85
86 R Segment_Distance(Point p, Point a, Point b) const
87 {

```

```

88     Point v = b - a;
89     R d = Dot(v, v);
90     if ( d == 0 )
91         return norm(p - a);
92     R t = clamp(Dot(p - a, v) / d, R(0), R(1));
93     return norm(p - (a + v * t));
94 }
95
96 Result Line_Intersection(Point a, Point b, Point c, Point d) const
97 {
98     Point u = b - a, v = d - c;
99     R nu = norm(u), nv = norm(v);
100    if ( nu == 0 || nv == 0 )
101        return {Kind::degenerate, {}};
102    R det = Cross(u, v);
103    if ( fabsl(det) <= eps * nu * nv )
104        return {fabsl(Cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
105                ? Kind::infinite
106                : Kind::none,
107                {}};
108    return {Kind::one, {a + u * (Cross(c - a, v) / det)}};
109 }
110
111 Result Line_Circle(Point a, Point b, Circle c) const
112 {
113     assert(c.r >= 0);
114     Point v = b - a;
115     R len = norm(v);
116     if ( len == 0 )
117         return {Kind::degenerate, {}};
118     Point h = Projection(c.o, a, b);
119     R d = norm(h - c.o), tol = eps * max({R(1), d, c.r});
120     if ( d > c.r + tol )
121         return {Kind::none, {}};
122     if ( fabsl(d - c.r) <= tol )
123         return {Kind::one, {h}};
124     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
125     Point w = v * (t / len);
126     return {Kind::two, {h - w, h + w}};
127 }
128
129 Result Circles(Circle a, Circle b) const
130 {
131     assert(a.r >= 0 && b.r >= 0);
132     Point v = b.o - a.o;
133     R d = norm(v);
134     R tol = eps * max({R(1), d, a.r, b.r});
135     if ( d <= tol )
136     {
137         if ( fabsl(a.r - b.r) > tol )
138             return {Kind::none, {}};
139         if ( a.r == 0 && b.r == 0 )
140             return {Kind::one, {a.o}};
141         return {Kind::infinite, {}};
142     }
143     if ( d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol )

```

```

144         return {Kind::none, {}};
145     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
146     Point h = a.o + v * (x / d);
147     if ( fabsl(d - (a.r + b.r)) <= tol || fabsl(d - fabsl(a.r - b.r)) <= tol )
148         return {Kind::one, {h}};
149     R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
150     Point w = perp(v) * (y / d);
151     return {Kind::two, {h - w, h + w}};
152 }
153
154 R Overlap(Circle a, Circle b) const
155 {
156     R d = norm(a.o - b.o), pi = acosl(-1.L);
157     if ( d >= a.r + b.r )
158         return 0;
159     if ( d <= fabsl(a.r - b.r) )
160         return pi * min(a.r, b.r) * min(a.r, b.r);
161     R x =
162         acosl(clamp((d * d + a.r * a.r - b.r * b.r) / (2 * d * a.r), R(-1), R(1)));
163     R y =
164         acosl(clamp((d * d + b.r * b.r - a.r * a.r) / (2 * d * b.r), R(-1), R(1)));
165     return a.r * a.r * (x - sinl(2 * x) / 2) + b.r * b.r * (y - sinl(2 * y) / 2);
166 }
167 };

```

第二十六章 优化与可持久化

26.1 Max-plus 矩阵幂

算法约束见动态版本第 56 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Max_Plus_Matrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     Max_Plus_Matrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if ( identity )
11             for ( int i = 0; i < n; i++ )
12                 a[i][i] = 0;
13     }
14
15     Max_Plus_Matrix operator*(const Max_Plus_Matrix &b) const
16     {
17         assert(n == b.n);
18         Max_Plus_Matrix c(n);
19         for ( int i = 0; i < n; i++ )
20             for ( int k = 0; k < n; k++ )
21                 if ( a[i][k] != neg )
22                     for ( int j = 0; j < n; j++ )
23                         if ( b.a[k][j] != neg )
24                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
25         return c;
26     }
27
28     Max_Plus_Matrix Power(unsigned long long e) const
29     {
30         Max_Plus_Matrix b = *this, r(n, true);
31         for ( ; e >>= 1, b = b * b )
32             if ( e & 1 )
33                 r = r * b;
34         return r;
35     }
36 };
```

26.2 离散李超树与最优直线编号

算法约束见动态版本第 56 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Li_Chao_Tree
2 {
3     using ll = long long;
4     using I = __int128_t;
5 }
```

```

6   struct Line
7   {
8       ll k = 0, b = 0;
9       int id = -1;
10
11       I value(ll x) const
12       {
13           return I(k) * x + b;
14       }
15   };
16
17   struct Node
18   {
19       Line line;
20       int left = -1, right = -1;
21   };
22
23   vector<ll> xs;
24   vector<Node> a{Node{}};
25
26   Li_Chao_Tree(vector<ll> x) : xs(x)
27   {
28       sort(xs.begin(), xs.end());
29       xs.erase(unique(xs.begin(), xs.end()), xs.end());
30       assert(!xs.empty());
31   }
32
33   bool better(Line u, Line v, ll x) const
34   {
35       return v.id == -1 ||
36              (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
37   }
38
39   void Insert(Line line)
40   {
41       assert(line.id >= 0);
42       Insert(0, 0, (int)xs.size() - 1, line);
43   }
44
45   void Insert(int p, int l, int r, Line line)
46   {
47       int m = (l + r) / 2;
48       if ( better(line, a[p].line, xs[m]) )
49           swap(line, a[p].line);
50       if ( l == r || line.id == -1 )
51           return;
52       bool left = better(line, a[p].line, xs[l]);
53       bool right = better(line, a[p].line, xs[r]);
54       if ( !left && !right )
55           return;
56       int next = left ? a[p].left : a[p].right;
57       if ( next == -1 )
58       {
59           next = (int)a.size();
60           a.push_back(Node{});
61           if ( left )

```

```

62         a[p].left = next;
63     else
64         a[p].right = next;
65 }
66 if ( left )
67     Insert(next, l, m, line);
68 else
69     Insert(next, m + 1, r, line);
70 }
71
72 pair<I, int> Query(ll x) const
73 {
74     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
75     assert(idx < (int)xs.size() && xs[idx] == x);
76     int p = 0, l = 0, r = (int)xs.size() - 1;
77     Line best;
78     while ( p != -1 )
79     {
80         if ( better(a[p].line, best, x) )
81             best = a[p].line;
82         if ( l == r )
83             break;
84         int m = (l + r) / 2;
85         if ( idx <= m )
86         {
87             p = a[p].left;
88             r = m;
89         }
90         else
91         {
92             p = a[p].right;
93             l = m + 1;
94         }
95     }
96     assert(best.id != -1);
97     return {best.value(x), best.id};
98 }
99 };

```

26.3 主席树：静态区间第 k 小

算法约束见动态版本第 58 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Persistent_Kth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7
8     vector<Node> t{Node{}};
9     vector<int> root{0};
10    vector<long long> vals;
11
12    Persistent_Kth(const vector<long long> &a) : vals(a)

```

```

13 {
14     sort(vals.begin(), vals.end());
15     vals.erase(unique(vals.begin(), vals.end()), vals.end());
16     for ( long long x : a )
17     {
18         int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
19         root.push_back(Insert(root.back(), 0, (int)vals.size() - 1, p));
20     }
21 }
22
23 int Insert(int old, int l, int r, int x)
24 {
25     int p = t.size();
26     Node copy = t[old];
27     t.push_back(copy);
28     ++t[p].sum;
29     if ( l < r )
30     {
31         int m = (l + r) / 2;
32         if ( x <= m )
33         {
34             int v = Insert(t[old].l, l, m, x);
35             t[p].l = v;
36         }
37         else
38         {
39             int v = Insert(t[old].r, m + 1, r, x);
40             t[p].r = v;
41         }
42     }
43     return p;
44 }
45
46 long long Kth(int l, int r, int k) const
47 {
48     assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
49     int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
50     while ( L < R )
51     {
52         int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
53         if ( k <= c )
54         {
55             u = t[u].l;
56             v = t[v].l;
57             R = m;
58         }
59         else

```

```
60         {
61             k -= c;
62             u = t[u].r;
63             v = t[v].r;
64             L = m + 1;
65         }
66     }
67     return vals[L];
68 }
69 };
```

第二十七章 图论进阶

27.1 矩形二分图最小权匹配

算法约束见动态版本第 61 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Kuhn_Munkres
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> Solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if ( !n )
10             return {0, {}};
11         int m = cost[0].size();
12         assert(n <= m);
13         vector<I> u(n + 1), v(m + 1);
14         vector<int> p(m + 1), way(m + 1);
15         const I inf = I(1) << 120;
16         for ( int i = 1; i <= n; i++ )
17         {
18             assert((int)cost[i - 1].size() == m);
19             p[0] = i;
20             int j0 = 0;
21             vector<I> dis(m + 1, inf);
22             vector<bool> Used(m + 1);
23             do
24             {
25                 Used[j0] = true;
26                 int i0 = p[j0], j1 = 0;
27                 I delta = inf;
28                 for ( int j = 1; j <= m; j++ )
29                     if ( !Used[j] )
30                     {
31                         I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
32                         if ( cur < dis[j] )
33                         {
34                             dis[j] = cur;
35                             way[j] = j0;
36                         }
37                         if ( dis[j] < delta )
38                         {
39                             delta = dis[j];
40                             j1 = j;
41                         }
42                     }
43                 for ( int j = 0; j <= m; j++ )
44                     if ( Used[j] )
45                     {
46                         u[p[j]] += delta;
```

```

47         v[j] -= delta;
48     }
49     else
50         dis[j] -= delta;
51     j0 = j1;
52 } while ( p[j0] );
53 do
54 {
55     int j1 = way[j0];
56     p[j0] = p[j1];
57     j0 = j1;
58 } while ( j0 );
59 }
60 vector<int> ans(n);
61 for ( int j = 1; j <= m; j++ )
62     if ( p[j] )
63         ans[p[j] - 1] = j - 1;
64 return {-v[0], ans};
65 }
66 };

```

27.2 有向最小树形图

算法约束见动态版本第 62 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Directed_MST
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> Solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while ( true )
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for ( auto a : e )
21                 if ( a.u != a.v && a.w < in[a.v] )
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for ( I x : in )
28             {
29                 if ( x == inf )
30                     return nullopt;

```

```

31         ans += x;
32     }
33     int cnt = 0;
34     for ( int i = 0; i < n; i++ )
35     {
36         int u = i;
37         while ( vis[u] != i && id[u] == -1 && u != root )
38         {
39             vis[u] = i;
40             u = pre[u];
41         }
42         if ( u != root && id[u] == -1 )
43         {
44             id[u] = cnt;
45             for ( int v = pre[u]; v != u; v = pre[v] )
46                 id[v] = cnt;
47             ++cnt;
48         }
49     }
50     if ( !cnt )
51         return ans;
52     for ( int i = 0; i < n; i++ )
53         if ( id[i] == -1 )
54             id[i] = cnt++;
55     for ( auto &a : e )
56     {
57         int v = a.v;
58         a.u = id[a.u];
59         a.v = id[a.v];
60         if ( a.u != a.v )
61             a.w -= in[v];
62     }
63     root = id[root];
64     n = cnt;
65 }
66 }
67 };

```

27.3 无向全局最小割

算法约束见动态版本第 64 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Global_Min_Cut
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> Solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for ( int i = 0; i < n; i++ )
14            group[i] = {i};
15        I best = I(1) << 120;
16        vector<int> side;
17        while ( active.size() > 1 )
18        {
19            vector<I> dis(n);
20            vector<bool> Used(n);
21            int prev = -1;
22            for ( int step = 0; step < (int)active.size(); step++ )
23            {
24                int sel = -1;
25                for ( int v : active )
26                    if ( !Used[v] && (sel == -1 || dis[v] > dis[sel]) )
27                        sel = v;
28                if ( step + 1 == (int)active.size() )
29                {
30                    if ( dis[sel] < best )
31                    {
32                        best = dis[sel];
33                        side = group[sel];
34                    }
35                    for ( int v : active )
36                        if ( v != prev && v != sel )
37                            w[prev][v] = w[v][prev] = w[prev][sel] + w[sel][v];
38                    group[prev].insert(
39                        group[prev].end(), group[sel].begin(), group[sel].end());
40                    active.erase(find(active.begin(), active.end(), sel));
41                    break;
42                }
43                Used[sel] = true;
44                prev = sel;
45                for ( int v : active )
46                    if ( !Used[v] )
47                        dis[v] += w[sel][v];
48            }
49        }
50        return {best, side};
51    }
52 };

```

第二十八章 精确几何进阶

28.1 整数最近点对与 Minkowski 和

算法约束见动态版本第 66 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Geometry_Extra
2 {
3     using G = Integer_Geometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance, O(n log n).
8     static optional<I> Closest_Pair(vector<P> p)
9     {
10         int n = p.size();
11         if ( n < 2 )
12             return nullopt;
13         sort(p.begin(), p.end());
14         for ( int i = 1; i < n; i++ )
15             if ( p[i] == p[i - 1] )
16                 return I(0);
17         vector<P> tmp(n);
18         auto by_y = [](P a, P b)
19         {
20             return tie(a.y, a.x) < tie(b.y, b.x);
21         };
22         function<I(int, int)> Solve = [&](int l, int r) -> I
23         {
24             if ( r - l <= 3 )
25             {
26                 I best = I(1) << 120;
27                 for ( int i = l; i < r; i++ )
28                     for ( int j = i + 1; j < r; j++ )
29                         best = min(best, G::Distance_Squared(p[i], p[j]));
30                 sort(p.begin() + l, p.begin() + r, by_y);
31                 return best;
32             }
33             int m = (l + r) / 2;
34             long long x = p[m].x;
35             I best = min(Solve(l, m), Solve(m, r));
36             merge(p.begin() + l,
37                 p.begin() + m,
38                 p.begin() + m,
39                 p.begin() + r,
40                 tmp.begin() + l,
41                 by_y);
42             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
43             vector<P> strip;
44             for ( int i = l; i < r; i++ )
45                 if ( (I(p[i].x) - x) * (I(p[i].x) - x) < best )
46                     {
```

```

47         for ( int j = (int)strip.size() - 1; j >= 0; j-- )
48         {
49             I dy = I(p[i].y) - strip[j].y;
50             if ( dy * dy >= best )
51                 break;
52             best = min(best, G::Distance_Squared(p[i], strip[j]));
53         }
54         strip.push_back(p[i]);
55     }
56     return best;
57 };
58 return Solve(0, n);
59 }
60
61 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
62 static vector<P> Minkowski(vector<P> a, vector<P> b)
63 {
64     if ( a.empty() || b.empty() )
65         return {};
66     auto plus = [](P u, P v)
67     {
68         return P{u.x + v.x, u.y + v.y};
69     };
70     if ( a.size() < 3 || b.size() < 3 )
71     {
72         vector<P> p;
73         for ( P u : a )
74             for ( P v : b )
75                 p.push_back(plus(u, v));
76         return G::Convex_Hull(p);
77     }
78     auto start = [](vector<P> &p)
79     {
80         auto it = min_element(p.begin(),
81                               p.end(),
82                               [](P u, P v)
83                               {
84                                   return tie(u.y, u.x) < tie(v.y, v.x);
85                               });
86         rotate(p.begin(), it, p.end());
87     };
88     start(a);
89     start(b);
90     int n = a.size(), m = b.size(), i = 0, j = 0;
91     vector<P> result;
92     while ( i < n || j < m )
93     {
94         result.push_back(plus(a[i % n], b[j % m]));
95         if ( i == n )
96         {
97             ++j;
98             continue;
99         }
100         if ( j == m )
101         {
102             ++i;

```

```
103         continue;
104     }
105     P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
106     P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
107     I c = G::Cross({0, 0}, u, v);
108     if ( c >= 0 )
109         ++i;
110     if ( c <= 0 )
111         ++j;
112 }
113 return result;
114 }
115 };
```

28.2 整数三维共线、共面与方向

算法约束见动态版本第 69 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Integer_Geometry_3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, Z_Function;
9     };
10
11     struct Vector
12     {
13         I x, y, Z_Function;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.Z_Function) - b.Z_Function};
19     }
20
21     static Vector Cross(Vector a, Vector b)
22     {
23         return {a.y * b.Z_Function - a.Z_Function * b.y,
24                 a.Z_Function * b.x - a.x * b.Z_Function,
25                 a.x * b.y - a.y * b.x};
26     }
27
28     static I Dot(Vector a, Vector b)
29     {
30         return a.x * b.x + a.y * b.y + a.Z_Function * b.Z_Function;
31     }
32
33     static I orient(Point a, Point b, Point c, Point d)
34     {
35         return Dot(Cross(diff(b, a), diff(c, a)), diff(d, a));
36     }
37
38     static bool collinear(Point a, Point b, Point c)
39     {
40         auto v = Cross(diff(b, a), diff(c, a));
41         return !v.x && !v.y && !v.Z_Function;
42     }
43
44     static bool On_Segment(Point p, Point a, Point b)
45     {
46         return collinear(p, a, b) && Dot(diff(a, p), diff(b, p)) <= 0;
47     }
48 };

```

第二十九章 动态树

29.1 Link-cut tree: 动态森林与路径异或

算法约束见动态版本第 70 页。本节代码独立可抄；采用下列实际接口名。

声明 `Link_Cut_Tree<N>`，然后 `Init(n)` 初始化；`N` 为最大点号。`Set`、`Link`、`Cut`、`Connected`、`Query` 分别对应动态版的小写接口。大对象放在全局或 `static`。

```
1 template <int N> struct Link_Cut_Tree
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int ch[2] {}, fa = 0;
8         U val = 0, sum = 0;
9         bool rev = false;
10    };
11
12    Node tree[N + 1];
13    int st[N + 1];
14
15    void Init(int n)
16    {
17        assert(0 <= n && n <= N);
18        fill(tree, tree + n + 1, Node{});
19    }
20
21    bool Is_Root(int x) const
22    {
23        int f = tree[x].fa;
24        return tree[f].ch[0] != x && tree[f].ch[1] != x;
25    }
26
27    void Push_Up(int x)
28    {
29        tree[x].sum = tree[tree[x].ch[0]].sum ^ tree[x].val ^ tree[tree[x].ch[1]].sum;
30    }
31
32    void Reverse(int x)
33    {
34        if ( !x )
35            return;
36        swap(tree[x].ch[0], tree[x].ch[1]);
37        tree[x].rev = !tree[x].rev;
38    }
39
40    void Push_Down(int x)
41    {
42        if ( !tree[x].rev )
43            return;
44        Reverse(tree[x].ch[0]);
```

```

45     Reverse(tree[x].ch[1]);
46     tree[x].rev = false;
47 }
48
49 void Rotate(int x)
50 {
51     int y = tree[x].fa, z = tree[y].fa;
52     int k = tree[y].ch[1] == x;
53     int w = tree[x].ch[k ^ 1];
54     if ( !Is_Root(y) )
55         tree[z].ch[tree[z].ch[1] == y] = x;
56     tree[x].fa = z;
57     tree[y].ch[k] = w;
58     if ( w )
59         tree[w].fa = y;
60     tree[x].ch[k ^ 1] = y;
61     tree[y].fa = x;
62     Push_Up(y);
63     Push_Up(x);
64 }
65
66 void Splay(int x)
67 {
68     int top = 0, y = x;
69     st[top++] = y;
70     while ( !Is_Root(y) )
71     {
72         y = tree[y].fa;
73         st[top++] = y;
74     }
75     while ( top )
76         Push_Down(st[--top]);
77     while ( !Is_Root(x) )
78     {
79         int y = tree[x].fa, z = tree[y].fa;
80         if ( !Is_Root(y) )
81         {
82             bool same = (tree[y].ch[1] == x) == (tree[z].ch[1] == y);
83             Rotate(same ? y : x);
84         }
85         Rotate(x);
86     }
87 }
88
89 void Access(int x)
90 {
91     int last = 0;
92     for ( int y = x; y; y = tree[y].fa )
93     {
94         Splay(y);
95         // The old right child's fa remains y: it becomes a path-parent.
96         tree[y].ch[1] = last;
97         if ( last )
98             tree[last].fa = y;
99         Push_Up(y);
100        last = y;

```

```

101     }
102     Splay(x);
103 }
104
105 void Make_Root(int x)
106 {
107     Access(x);
108     Reverse(x);
109 }
110
111 int Find_Root(int x)
112 {
113     Access(x);
114     Push_Down(x);
115     while ( tree[x].ch[0] )
116     {
117         x = tree[x].ch[0];
118         Push_Down(x);
119     }
120     Splay(x);
121     return x;
122 }
123
124 bool Connected(int x, int y)
125 {
126     return x == y || Find_Root(x) == Find_Root(y);
127 }
128
129 bool Link(int x, int y)
130 {
131     Make_Root(x);
132     if ( Find_Root(y) == x )
133         return false;
134     tree[x].fa = y;
135     return true;
136 }
137
138 bool Cut(int x, int y)
139 {
140     Make_Root(x);
141     Access(y);
142     if ( tree[y].ch[0] != x || tree[x].ch[1] )
143         return false;
144     tree[y].ch[0] = 0;
145     tree[x].fa = 0;
146     Push_Up(y);
147     return true;
148 }
149
150 void Set(int x, U value)
151 {
152     Access(x);
153     tree[x].val = value;
154     Push_Up(x);

```

```
155     }
156
157     // x and y must be connected. This is a path aggregate, not a subtree sum.
158     U Query(int x, int y)
159     {
160         Make_Root(x);
161         Access(y);
162         return tree[y].sum;
163     }
164 };
```

第三十章 随机平衡树

30.1 FHQ Treap: 可重集合与顺序统计

算法约束见动态版本第 74 页。本节代码独立可抄；采用下列实际接口名。

声明容量模板参数 N，然后 Init() 或 Init(seed)。N 限制累计插入次数，不是当前元素个数；大对象放在全局或 static。接口采用 Insert、Erase、Kth/Query 等命名。

```
1 template <int N> struct Ordered_Treap
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0;
8         ll val = 0;
9         unsigned long long pri = 0;
10    };
11
12    Node tree[N + 1];
13    int tot = 0;
14    mt19937_64 rng;
15    int root = 0;
16
17    void Init(unsigned long long seed = 712367821)
18    {
19        root = 0;
20        tot = 0;
21        tree[0] = Node{};
22        rng.seed(seed);
23    }
24
25    void Push_Up(int p)
26    {
27        tree[p].siz = tree[tree[p].l].siz + tree[tree[p].r].siz + 1;
28    }
29
30    int New_Node(ll x)
31    {
32        assert(tot < N);
33        int p = ++tot;
34        tree[p] = {0, 0, 1, x, rng()};
35        return p;
36    }
37
38    // left: val < x, or val <= x when equal is true. No x-1 overflow.
39    void Split(int p, ll x, int &l, int &r, bool equal = false)
40    {
41        if ( !p )
42        {
43            l = 0;
44            r = 0;
```

```

45         return;
46     }
47     if ( tree[p].val < x || (equal && tree[p].val == x) )
48     {
49         l = p;
50         Split(tree[p].r, x, tree[p].r, r, equal);
51     }
52     else
53     {
54         r = p;
55         Split(tree[p].l, x, l, tree[p].l, equal);
56     }
57     Push_Up(p);
58 }
59
60 int Merge(int l, int r)
61 {
62     if ( !l || !r )
63         return l ? l : r;
64     if ( tree[l].pri > tree[r].pri )
65     {
66         tree[l].r = Merge(tree[l].r, r);
67         Push_Up(l);
68         return l;
69     }
70     tree[r].l = Merge(l, tree[r].l);
71     Push_Up(r);
72     return r;
73 }
74
75 void Insert(ll x)
76 {
77     int p = New_Node(x), l, r;
78     Split(root, x, l, r);
79     root = Merge(Merge(l, p), r);
80 }
81
82 bool Erase(ll x)
83 {
84     int l, mid, r;
85     Split(root, x, l, mid);
86     Split(mid, x, mid, r, true);
87     bool found = mid != 0;
88     if ( mid )
89         mid = Merge(tree[mid].l, tree[mid].r);
90     root = Merge(l, Merge(mid, r));
91     return found;
92 }
93
94 int Size() const
95 {
96     return tree[root].siz;
97 }
98
99 int Count_Less(ll x, bool equal = false) const
100 {

```

```

101     int p = root, ans = 0;
102     while ( p )
103     {
104         if ( tree[p].val < x || (equal && tree[p].val == x) )
105         {
106             ans += tree[tree[p].l].siz + 1;
107             p = tree[p].r;
108         }
109         else
110             p = tree[p].l;
111     }
112     return ans;
113 }
114
115 int Rank(ll x) const
116 {
117     return Count_Less(x) + 1;
118 }
119
120 ll Kth(int k) const
121 {
122     assert(1 <= k && k <= Size());
123     int p = root;
124     while ( true )
125     {
126         int left = tree[tree[p].l].siz;
127         if ( k == left + 1 )
128             return tree[p].val;
129         if ( k <= left )
130             p = tree[p].l;
131         else
132         {
133             k -= left + 1;
134             p = tree[p].r;
135         }
136     }
137 }
138
139 optional<ll> Predecessor(ll x) const
140 {
141     int k = Count_Less(x);
142     if ( !k )
143         return nullopt;
144     return Kth(k);
145 }
146
147 optional<ll> Successor(ll x) const
148 {
149     int k = Count_Less(x, true);
150     if ( k == Size() )
151         return nullopt;
152     return Kth(k + 1);
153 }
154 };

```

30.2 隐式 Treap：序列翻转、加值与区间和

算法约束见动态版本第 76 页。本节代码独立可抄；采用下列实际接口名。

声明容量模板参数 N ，然后 `Init()` 或 `Init(seed)`。 N 限制累计插入次数，不是当前元素个数；大对象放在全局或 `static`。接口采用 `Insert`、`Erase`、`Kth/Query` 等命名。

```

1  template <int N> struct Sequence_Treap
2  {
3      using ll = long long;
4
5      struct Node
6      {
7          int l = 0, r = 0, siz = 0;
8          ll val = 0, sum = 0, tag = 0;
9          unsigned long long pri = 0;
10         bool rev = false;
11     };
12
13     Node tree[N + 1];
14     int tot = 0;
15     mt19937_64 rng;
16     int root = 0;
17
18     void Init(unsigned long long seed = 712367821)
19     {
20         root = 0;
21         tot = 0;
22         tree[0] = Node{};
23         rng.seed(seed);
24     }
25
26     int Size() const
27     {
28         return tree[root].siz;
29     }
30
31     void Push_Up(int p)
32     {
33         tree[p].siz = tree[tree[p].l].siz + tree[tree[p].r].siz + 1;
34         tree[p].sum = tree[tree[p].l].sum + tree[p].val + tree[tree[p].r].sum;
35     }
36
37     void Apply_Add(int p, ll x)
38     {
39         if (!p)
40             return;
41         tree[p].val += x;
42         tree[p].sum += x * tree[p].siz;
43         tree[p].tag += x;
44     }
45
46     void Apply_Reverse(int p)
47     {
48         if (!p)
49             return;
50         swap(tree[p].l, tree[p].r);

```

```

51     tree[p].rev = !tree[p].rev;
52 }
53
54 void Push_Down(int p)
55 {
56     if ( tree[p].rev )
57     {
58         Apply_Reverse(tree[p].l);
59         Apply_Reverse(tree[p].r);
60         tree[p].rev = false;
61     }
62     if ( tree[p].tag )
63     {
64         Apply_Add(tree[p].l, tree[p].tag);
65         Apply_Add(tree[p].r, tree[p].tag);
66         tree[p].tag = 0;
67     }
68 }
69
70 // Split into the first k elements and the rest.
71 void Split(int p, int k, int &l, int &r)
72 {
73     if ( !p )
74     {
75         l = 0;
76         r = 0;
77         return;
78     }
79     Push_Down(p);
80     int left = tree[tree[p].l].siz;
81     if ( k > left )
82     {
83         l = p;
84         Split(tree[p].r, k - left - 1, tree[p].r, r);
85     }
86     else
87     {
88         r = p;
89         Split(tree[p].l, k, l, tree[p].l);
90     }
91     Push_Up(p);
92 }
93
94 int Merge(int l, int r)
95 {
96     if ( !l || !r )
97         return l ? l : r;
98     if ( tree[l].pri > tree[r].pri )
99     {
100         Push_Down(l);
101         tree[l].r = Merge(tree[l].r, r);
102         Push_Up(l);
103         return l;
104     }
105     Push_Down(r);
106     tree[r].l = Merge(l, tree[r].l);

```

```

107     Push_Up(r);
108     return r;
109 }
110
111 // Insert after the first k elements, 0 <= k <= size().
112 void Insert(int k, ll value)
113 {
114     assert(0 <= k && k <= Size());
115     assert(tot < N);
116     int p = ++tot, l, r;
117     tree[p] = {0, 0, 1, value, value, 0, rng(), false};
118     Split(root, k, l, r);
119     root = Merge(Merge(l, p), r);
120 }
121
122 // All range operations use 1-based inclusive intervals.
123 void Erase(int l, int r)
124 {
125     assert(1 <= l && l <= r && r <= Size());
126     int x, y, z;
127     Split(root, r, y, z);
128     Split(y, l - 1, x, y);
129     root = Merge(x, z);
130 }
131
132 void Reverse(int l, int r)
133 {
134     assert(1 <= l && l <= r && r <= Size());
135     int x, y, z;
136     Split(root, r, y, z);
137     Split(y, l - 1, x, y);
138     Apply_Reverse(y);
139     root = Merge(Merge(x, y), z);
140 }
141
142 void Add(int l, int r, ll value)
143 {
144     assert(1 <= l && l <= r && r <= Size());
145     int x, y, z;
146     Split(root, r, y, z);
147     Split(y, l - 1, x, y);
148     Apply_Add(y, value);
149     root = Merge(Merge(x, y), z);
150 }
151
152 ll Query(int l, int r)
153 {
154     assert(1 <= l && l <= r && r <= Size());
155     int x, y, z;
156     Split(root, r, y, z);
157     Split(y, l - 1, x, y);
158     ll ans = tree[y].sum;
159     root = Merge(Merge(x, y), z);
160     return ans;
161 }
162

```

```
163     vector<ll> Values()
164     {
165         vector<ll> ans;
166         vector<int> st;
167         int p = root;
168         while ( p || !st.empty() )
169         {
170             while ( p )
171             {
172                 Push_Down(p);
173                 st.push_back(p);
174                 p = tree[p].l;
175             }
176             p = st.back();
177             st.pop_back();
178             ans.push_back(tree[p].val);
179             p = tree[p].r;
180         }
181         return ans;
182     }
183 };
```

第三十一章 伸展树

31.1 Splay: 可重集合与顺序统计

算法约束见动态版本第 81 页。本节代码独立可抄；采用下列实际接口名。

声明 `Ordered_Splay<N>` 后调用 `Init()`；`N` 为累计新建节点上限。接口采用 `Insert`、`Erase`、`Rank`、`Kth`、`Prev`、`Next`。大对象放在全局或 `static`。

```
1 template <int N> struct Ordered_Splay
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         int ch[2]{}; fa = 0, cnt = 0, siz = 0;
8         ll val = 0;
9     };
10
11     Node tree[N + 1];
12     int tot = 0;
13     int root = 0;
14
15     void Init()
16     {
17         root = 0;
18         tot = 0;
19         tree[0] = Node{};
20     }
21
22     int Size() const
23     {
24         return tree[root].siz;
25     }
26
27     void Push_Up(int x)
28     {
29         tree[x].siz = tree[tree[x].ch[0]].siz + tree[x].cnt + tree[tree[x].ch[1]].siz;
30     }
31
32     int New_Node(ll value, int parent)
33     {
34         assert(tot < N);
35         int x = ++tot;
36         tree[x] = {0, 0, parent, 1, 1, value};
37         return x;
38     }
39
40     void Rotate(int x)
41     {
42         int y = tree[x].fa, z = tree[y].fa;
43         int k = tree[y].ch[1] == x;
44         int w = tree[x].ch[k ^ 1];
```

```

45     if ( z )
46         tree[z].ch[tree[z].ch[1] == y] = x;
47     else
48         root = x;
49     tree[x].fa = z;
50     tree[y].ch[k] = w;
51     if ( w )
52         tree[w].fa = y;
53     tree[x].ch[k ^ 1] = y;
54     tree[y].fa = x;
55     Push_Up(y);
56     Push_Up(x);
57 }
58
59 void Splay(int x)
60 {
61     while ( tree[x].fa )
62     {
63         int y = tree[x].fa, z = tree[y].fa;
64         if ( z )
65         {
66             bool same = (tree[y].ch[1] == x) == (tree[z].ch[1] == y);
67             Rotate(same ? y : x);
68         }
69         Rotate(x);
70     }
71 }
72
73 // Splay the last visited node even when the value does not exist.
74 int Find(ll value)
75 {
76     int x = root, last = 0;
77     while ( x && tree[x].val != value )
78     {
79         last = x;
80         x = tree[x].ch[value > tree[x].val];
81     }
82     if ( x )
83         Splay(x);
84     else if ( last )
85         Splay(last);
86     return x;
87 }
88
89 void Insert(ll value)
90 {
91     int x = root, parent = 0;
92     while ( x && tree[x].val != value )
93     {
94         parent = x;
95         x = tree[x].ch[value > tree[x].val];
96     }
97     if ( x )
98     {
99         ++tree[x].cnt;
100         Push_Up(x);

```

```

101     }
102     else
103     {
104         x = New_Node(value, parent);
105         if ( parent )
106             tree[parent].ch[value > tree[parent].val] = x;
107         else
108             root = x;
109     }
110     Splay(x);
111 }
112
113 bool Erase(ll value)
114 {
115     int x = Find(value);
116     if ( !x )
117         return false;
118     if ( tree[x].cnt > 1 )
119     {
120         --tree[x].cnt;
121         Push_Up(x);
122         return true;
123     }
124     int l = tree[x].ch[0], r = tree[x].ch[1];
125     if ( l )
126         tree[l].fa = 0;
127     if ( r )
128         tree[r].fa = 0;
129     if ( !l )
130     {
131         root = r;
132         return true;
133     }
134     root = l;
135     x = l;
136     while ( tree[x].ch[1] )
137         x = tree[x].ch[1];
138     Splay(x);
139     tree[x].ch[1] = r;
140     if ( r )
141         tree[r].fa = x;
142     Push_Up(x);
143     return true;
144 }
145
146 int Rank(ll value)
147 {
148     int x = root, last = 0, ans = 1;
149     while ( x )
150     {
151         last = x;
152         if ( value <= tree[x].val )
153             x = tree[x].ch[0];
154         else
155         {
156             ans += tree[tree[x].ch[0]].siz + tree[x].cnt;

```

```

157         x = tree[x].ch[1];
158     }
159 }
160 if ( last )
161     Splay(last);
162 return ans;
163 }
164
165 ll Kth(int k)
166 {
167     assert(1 <= k && k <= Size());
168     int x = root;
169     while ( true )
170     {
171         int left = tree[tree[x].ch[0]].siz;
172         if ( k <= left )
173             x = tree[x].ch[0];
174         else if ( k <= left + tree[x].cnt )
175         {
176             Splay(x);
177             return tree[x].val;
178         }
179         else
180         {
181             k -= left + tree[x].cnt;
182             x = tree[x].ch[1];
183         }
184     }
185 }
186
187 optional<ll> Predecessor(ll value)
188 {
189     int x = root, last = 0, answer = 0;
190     while ( x )
191     {
192         last = x;
193         if ( tree[x].val < value )
194         {
195             answer = x;
196             x = tree[x].ch[1];
197         }
198         else
199             x = tree[x].ch[0];
200     }
201     if ( last )
202         Splay(last);
203     if ( !answer )
204         return nullopt;
205     Splay(answer);
206     return tree[answer].val;
207 }
208
209 optional<ll> Successor(ll value)
210 {
211     int x = root, last = 0, answer = 0;
212     while ( x )

```

```
213     {
214         last = x;
215         if ( tree[x].val > value )
216         {
217             answer = x;
218             x = tree[x].ch[0];
219         }
220         else
221             x = tree[x].ch[1];
222     }
223     if ( last )
224         Splay(last);
225     if ( !answer )
226         return nullopt;
227     Splay(answer);
228     return tree[answer].val;
229 }
230 };
```

第三十二章 状态序列维护

32.1 状态序列 Treap：分组区间 GCD

算法约束见动态版本第 86 页。本节代码独立可抄；采用下列实际接口名。

声明 `Gcd_Sequence_Treap<N>` 后调用 `Init(seed)`；`N` 为峰值元素数。`Build` 重建序列，`Insert`、`Erase`、`Set`、`Toggle`、`Query` 对应动态版接口。大对象放在全局或 `static`。

```
1 template <int N> struct Gcd_Sequence_Treap
2 {
3     using U = unsigned long long;
4
5     struct Node
6     {
7         int l = 0, r = 0, siz = 0, tag = 0;
8         int cnt[2]{};
9         U val = 0, g[2]{}, pri = 0;
10    };
11
12    Node tree[N + 1];
13    int tot = 0;
14    vector<int> recycled;
15    mt19937_64 rng;
16    int root = 0;
17
18    void Init(U seed = 712367821)
19    {
20        root = 0;
21        tot = 0;
22        tree[0] = Node{};
23        recycled.clear();
24        rng.seed(seed);
25    }
26
27    int Size() const
28    {
29        return tree[root].siz;
30    }
31
32    int New_Node(U value, int tag)
33    {
34        assert(tag == 0 || tag == 1);
35        int p;
36        if ( recycled.empty() )
37        {
38            assert(tot < N);
39            p = ++tot;
40            tree[p] = Node{};
41        }
42        else
43        {
44            p = recycled.back();
```

```

45         recycled.pop_back();
46         tree[p] = Node{};
47     }
48     tree[p].siz = 1;
49     tree[p].val = value;
50     tree[p].tag = tag;
51     tree[p].cnt[tag] = 1;
52     tree[p].g[tag] = value;
53     tree[p].pri = rng();
54     return p;
55 }
56
57 void Push_Up(int p)
58 {
59     int l = tree[p].l, r = tree[p].r;
60     tree[p].siz = tree[l].siz + tree[r].siz + 1;
61     for ( int t = 0; t < 2; t++ )
62     {
63         tree[p].cnt[t] = tree[l].cnt[t] + tree[r].cnt[t] + (tree[p].tag == t);
64         tree[p].g[t] = gcd(tree[l].g[t], tree[r].g[t]);
65         if ( tree[p].tag == t )
66             tree[p].g[t] = gcd(tree[p].g[t], tree[p].val);
67     }
68 }
69
70 void Split(int p, int k, int &l, int &r)
71 {
72     if ( !p )
73     {
74         l = 0;
75         r = 0;
76         return;
77     }
78     int left = tree[tree[p].l].siz;
79     if ( k > left )
80     {
81         l = p;
82         Split(tree[p].r, k - left - 1, tree[p].r, r);
83     }
84     else
85     {
86         r = p;
87         Split(tree[p].l, k, l, tree[p].l);
88     }
89     Push_Up(p);
90 }
91
92 int Merge(int l, int r)
93 {
94     if ( !l || !r )
95         return l ? l : r;
96     if ( tree[l].pri > tree[r].pri )
97     {
98         tree[l].r = Merge(tree[l].r, r);
99         Push_Up(l);
100        return l;

```

```

101     }
102     tree[r].l = Merge(l, tree[r].l);
103     Push_Up(r);
104     return r;
105 }
106
107 // O(n) GCD calls; replaces the current sequence.
108 void Build(const vector<pair<U, int>> &v)
109 {
110     assert(v.size() <= N);
111     tot = 0;
112     tree[0] = Node{};
113     recycled.clear();
114     root = 0;
115     vector<int> st;
116     for ( auto [value, tag] : v )
117     {
118         int p = New_Node(value, tag), last = 0;
119         while ( !st.empty() && tree[st.back()].pri < tree[p].pri )
120         {
121             last = st.back();
122             st.pop_back();
123         }
124         tree[p].l = last;
125         if ( !st.empty() )
126             tree[st.back()].r = p;
127         st.push_back(p);
128     }
129     if ( st.empty() )
130         return;
131     root = st.front();
132     vector<int> order{root};
133     for ( int i = 0; i < (int)order.size(); i++ )
134     {
135         int p = order[i];
136         if ( tree[p].l )
137             order.push_back(tree[p].l);
138         if ( tree[p].r )
139             order.push_back(tree[p].r);
140     }
141     for ( int i = (int)order.size() - 1; i >= 0; i-- )
142         Push_Up(order[i]);
143 }
144
145 // Insert after the first k entries. Range operations below are 1-based.
146 void Insert(int k, U value, int tag)
147 {
148     assert(0 <= k && k <= Size());
149     int p = New_Node(value, tag), l, r;
150     Split(root, k, l, r);
151     root = Merge(Merge(l, p), r);
152 }
153
154 void Erase(int l, int r)
155 {
156     assert(1 <= l && l <= r && r <= Size());

```

```

157     int x, y, z;
158     Split(root, r, y, z);
159     Split(y, l - 1, x, y);
160     root = Merge(x, z);
161     vector<int> st{y};
162     while ( !st.empty() )
163     {
164         int p = st.back();
165         st.pop_back();
166         if ( tree[p].l )
167             st.push_back(tree[p].l);
168         if ( tree[p].r )
169             st.push_back(tree[p].r);
170         recycled.push_back(p);
171     }
172 }
173
174 void Set(int k, U value)
175 {
176     assert(1 <= k && k <= Size());
177     int x, y, z;
178     Split(root, k, y, z);
179     Split(y, k - 1, x, y);
180     tree[y].val = value;
181     Push_Up(y);
182     root = Merge(Merge(x, y), z);
183 }
184
185 void Toggle(int k)
186 {
187     assert(1 <= k && k <= Size());
188     int x, y, z;
189     Split(root, k, y, z);
190     Split(y, k - 1, x, y);
191     tree[y].tag ^= 1;
192     Push_Up(y);
193     root = Merge(Merge(x, y), z);
194 }
195
196 optional<U> Query(int l, int r, int tag)
197 {
198     assert(1 <= l && l <= r && r <= Size());
199     assert(tag == 0 || tag == 1);
200     int x, y, z;
201     Split(root, r, y, z);
202     Split(y, l - 1, x, y);
203     optional<U> answer;
204     if ( tree[y].cnt[tag] )
205         answer = tree[y].g[tag];
206     root = Merge(Merge(x, y), z);
207     return answer;
208 }
209 };

```

第三部分

数学速查与维护

第三十三章 数学速查：条件先于公式

33.1 整除、卷积与筛法

对正整数定义狄利克雷卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

单位元为 $\varepsilon(n) = [n = 1]$, 常函数 $1(n) = 1$, 恒等函数 $\text{id}(n) = n$ 。关键恒等式是 $\mu * 1 = \varepsilon$ 、 $\varphi * 1 = \text{id}$ 。若 $F(n) = \sum_{d|n} f(d)$, 则 $f(n) = \sum_{d|n} \mu(d)F(n/d)$ 。

$$\sum_{i=1}^a \sum_{j=1}^b [\gcd(i, j) = 1] = \sum_{d=1}^{\min(a, b)} \mu(d) \lfloor a/d \rfloor \lfloor b/d \rfloor.$$

整除分块：当左端点为 l 、商为 $q = \lfloor n/l \rfloor$, 右端点为 $r = \lfloor n/q \rfloor$ 。同时包含多个商时，取各自右端点的最小值。杜教筛由卷积恒等式获得

$$M(n) = 1 - \sum_{l=2}^n M(\lfloor n/l \rfloor), \quad \Phi(n) = \frac{n(n+1)}{2} - \sum_{l=2}^n \Phi(\lfloor n/l \rfloor),$$

求和按等商区间合并；第二式求和中没有额外乘以 l 。

33.2 模运算的适用条件

模逆元存在当且仅当 $\gcd(a, m) = 1$ 。费马逆元 a^{p-2} 仅适用于素数模数及 $a \not\equiv 0$ 。对正模数 m, n , 同余式 $x \equiv a \pmod{m}$ 、 $x \equiv b \pmod{n}$ 可合并当且仅当 $\gcd(m, n) \mid (b - a)$ ；新模数为 $\text{lcm}(m, n)$ 。

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p} \quad (p \text{ 为素数, } n_i, k_i \text{ 是 } p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。阶乘与逆阶乘预处理要求上限小于模数；超过模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

33.3 组合计数

容斥： $|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$ 。二项式反演：若 $b_n = \sum_{k=0}^n \binom{n}{k} a_k$, 则 $a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} b_k$ 。

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}, \quad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$, 在模数下尤其有用，但组合数本身仍须正确求模。第二类 Stirling 数满足

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad S(0, 0) = 1.$$

第一类无符号 Stirling 数把系数 k 换为 $n-1$ 。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n, k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^k S(n, j)$	$S(n, k)$

上表默认 $k \geq 1$ ； $n = 0$ 、 $k = 0$ 等边界应按原问题约定单独处理。

33.4 群作用计数

有限群 G 作用在有限集合 X 上时, 轨道数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

对位置置换的 q 色染色, $|\text{Fix}(g)| = q^{c(g)}$, 其中 $c(g)$ 为循环数。环形项链只商去旋转; 手链还商去反射, 二者群不同, 不能共用分母。模数整除 $|G|$ 时不能使用模逆元, 须先在合适的整数模数下求和再做精确除法。

33.5 生成函数与递推

普通生成函数 $A(x) = \sum a_n x^n$ 的乘法对应普通卷积。指数生成函数 $A(x) = \sum a_n x^n / n!$ 的乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ 。形式幂级数求逆要求常数项可逆; $\log A$ 取 $A(0) = 1$, $\exp A$ 取 $A(0) = 0$ 。有限域中积分涉及 $1/i$, 必须避免 i 被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \quad (\log A)' = A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用, 必须另外证明序列确实具有不超过相应阶数的线性递推; 少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

33.6 线性代数与生成树

域上的线性方程组一致时, 解集为一个特解加零空间, 零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值; 模素数消元是精确域运算; 合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数, 非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树; 重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向, 不能直接复用无向封装。

33.7 概率、期望与博弈

期望线性性不要求独立: $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ 。方差可加需要协方差为零; 独立是充分条件。非负整数变量 $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$ 。吸收过程的期望方程 $E_i = 1 + \sum_j p_{ij} E_j$ 需要检查是否几乎必然吸收及期望是否有限; 不能对闭合非吸收分量直接求唯一有限解。有限无环公平组合游戏在正常玩法下

$$\text{SG}(u) = \text{mex}\{\text{SG}(v) : u \rightarrow v\}.$$

独立子游戏的 SG 值异或为零当且仅当先手必败。反常玩法、含环游戏、双方可走步不同的游戏不能直接套这个结论。

33.8 数值算法与几何

排序必须使用严格弱序; 不能以“差小于 ϵ 则相等”定义通用排序比较器。叉积与行列式容易发生抵消; 整数题优先精确整数判定, 浮点构造单独处理误差。对于简单格点多边形 (无孔), Pick 定理 $A = I + B/2 - 1$, 边界格点数 $B = \sum \gcd(|\Delta x|, |\Delta y|)$; 不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则; 奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性, 整数域最后枚举剩余区间; 二分答案要求判定单调性。

来源索引 本章为重新组织的公式速查, 扩展范围依据 <https://oi-wiki.org/math/>。还需补齐 exLucas、二次剩余、原根、Min25、插值、子集卷积、行列式进阶、数值积分等完整实现与验证; 本章公式不能代替这些待完成代码。

第三十四章 来源与覆盖维护

34.1 参考基线

- kuangbin, ACM 常用算法模板, 2018 年版本: <https://kuangbin.github.io/2018/08/01/ACM-template/>。
- WIDA XCPC: <https://github.com/hh2048/XCPC>, 参考快照 484e9e0。
- OI Wiki 数学: <https://oi-wiki.org/math/>。

本项目重新实现的代码与公式速查不能冒用原作者的验证记录。完整覆盖审计应为每个来源条目记录实现文件、替代关系与验证证据, 保留遗漏项直至完成。

34.2 后续覆盖项

尚需完成的范围以覆盖表为准, 包括动态树、平衡树、一般图匹配、最小树形图、上下界流、高阶几何构造、半平面交、三维凸包、数值积分、复杂数论及多项式、搜索与动态规划等。不能用通用模块的存在推断其覆盖了所有原书例题。

算法索引

AC_Automaton, 114
AhoCorasick, 26
Arborescence, 62

Berlekamp-Massey, 179
Binary_Indexed_Tree, 93
Binomial, 38
Bipartite_Matching, 105
BipartiteMatching, 16
Bounded_Circulation, 101
BoundedCirculation, 11
Burnside, 179

Catalan, 178
Combination, 126
CRT, 178

Dijkstra, 13
Dinic, 7
Directed_MST, 150
Dirichlet convolution, 178
Discrete_Log, 136
DiscreteLog, 48
Disjoint_Set, 91
DSU, 2
Du_Jiao, 135
DuJiao, 47

Euler theorem, 178
expectation, 179

Fenwick, 4

Gcd_Sequence_Treap, 173
GcdSequenceTreap, 86
generating functions, 179
Geometry_Extra, 153
GeometryExtra, 66
Global_Min_Cut, 152

Heavy_Light_Decomposition, 109
HLD, 21
Hungarian, 61

inclusion-exclusion, 178
Integer_Geometry, 137

Integer_Geometry_3D, [156](#)

IntegerGeometry, [49](#)

IntegerGeometry3D, [69](#)

Kmp_Match, [112](#)

kmp_match, [24](#)

Kuhn_Munkres, [149](#)

LazySeg, [4](#)

Li_Chao_Tree, [144](#)

LiChao, [56](#)

Linear_Algebra, [132](#)

Linear_Sieve, [124](#)

LinearAlgebra, [44](#)

LinearSieve, [36](#)

Link_Cut_Tree, [157](#)

LinkCutTree, [70](#)

Low_Link, [107](#)

Lowlink, [19](#)

Lucas theorem, [178](#)

Manacher, [113](#)

manacher, [25](#)

Matrix Tree theorem, [179](#)

Max_Plus_Matrix, [144](#)

MaxPlusMatrix, [56](#)

Min_Cost_Flow, [98](#)

MinCostFlow, [9](#)

Minimum_Rotation, [114](#)

minimum_rotation, [26](#)

Mobius inversion, [178](#)

Mod_Int, [125](#)

ModInt, [37](#)

Network_Flow, [96](#)

Number_Theory, [120](#)

NumberTheory, [32](#)

numerical stability, [179](#)

Ordered_Splay, [168](#)

Ordered_Treap, [161](#)

OrderedSplay, [81](#)

OrderedTreap, [74](#)

Persistent_Kth, [146](#)

PersistentKth, [58](#)

Pick theorem, [179](#)

Pollard_Rho, [123](#)

PollardRho, [35](#)

Polya, [179](#)

Polynomial, [39](#), [127](#)
Prefix_Function, [112](#)
prefix_function, [24](#)

rank, [179](#)
Real_Geometry, [140](#)
RealGeometry, [52](#)
Rollback_DSU, [92](#)
RollbackDSU, [3](#)

SCC, [14](#)
Segment_Tree, [94](#)
Sequence_Treap, [164](#)
SequenceTreap, [76](#)
Shortest_Path, [102](#)
Sprague-Grundy, [179](#)
Stirling numbers, [178](#)
StoerWagner, [64](#)
Strong_Component, [103](#)
Suffix_Array, [117](#)
Suffix_Automaton, [118](#)
SuffixArray, [29](#)
SuffixAutomaton, [30](#)

Two_SAT, [105](#)
TwoSAT, [16](#)

Xor_Basis, [95](#)
XorBasis, [6](#)

Z_Function, [113](#)
z_function, [25](#)